Zcash for Bitcoiners

whiteboard notes: how a shielded transfer is verified without being seen

m@rek.onl

NU7 draft: 23 September 2026

Contents

1 Part A — What Bitcoin publishes, and what privacy must mean **1**

2 Part B — The toolbox, from Bitcoin's primitives up **7**

3 Part C — The data model **20**

4 Part D — Keys and addresses **29**

5 Part E — The Action statement **36**

6 Part E′ — Balance and authority, on the running example **46**

7 Part G — The proof system **51**

8 Part F — Delivery and light clients **63**

9 Part H — The transaction and the turnstile **74**

10 Part K — Consensus: why one fork wins, and what the chain must remember **83**

11 Part I — Limits, frontier, takeaways **95**

12 Backup — optional boards **105**

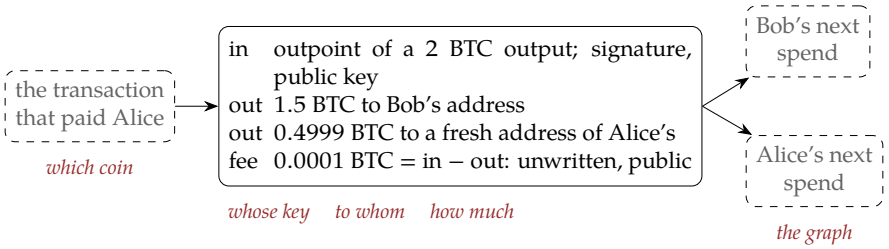
1 Part A — What Bitcoin publishes, and what privacy must mean

Board 1 — Alice pays Bob in Bitcoin: what the chain publishes

all four, read in the clear

Draw

1. Centre of the board: one box with four rows, left labels `in`, `out`, `out`, `fee`; fill the rows from Write below.
2. Left of the box, a dashed box "the transaction that paid Alice"; an arrow from it into the main box.
3. Right of the box, two dashed boxes stacked: "Bob's next spend" above, "Alice's next spend" below; an arrow from the main box's right edge to each.
4. In red, italic, small: "which coin" under the left box; "the graph" under the lower right box; "whose key to whom how much" under the main box's bottom-left corner.



Write

The four rows of the box, in order:

- `in`: outpoint of a 2 BTC output; signature, public key
- `out`: 1.5 BTC to Bob's address
- `out`: 0.4999 BTC to a fresh address of Alice's
- `fee`: 0.0001 BTC = in − out: unwritten, public

Say

Start where the room already is. Alice holds one 2 BTC output and pays Bob 1.5; the change goes to a fresh address of hers. Walk the box row by row: the input names an old output by its outpoint and carries a signature and a public key; each output carries its amount and its spending condition.

The names cost outside information; the graph costs nothing. Nobody wrote "Alice" or "Bob" on the chain, but which coin, whose key, to whom, how much, and the arrows to the previous and the next spend are all in the record. The fee is not even written, and it is public.

Address reuse and change detection are structural. Reuse an address and its history merges. Pay with change and the non-round output to a fresh address is usually yours. Neither is the fault of a careless wallet; both follow from what the record has to contain.

Keep this shape on the board. One 2-coin input, 1.5 to Bob, change back: the shielded running example has exactly this shape, with none of the labels.

Source: *Ironwood Guide*, "Requirements on a shielded payment"

Board 2 — The validator's four checks

all four

Draw

A pseudocode block in three columns: the code on the left, the check's number and name in the middle, what the check reads on the right. Six lines, indented as code; the middle and right columns are filled on the four checked lines only. Number the checks as you write them.

Write

```
1. validate(tx):
2.   for input in tx.inputs:
3.     coin = utxo[input.prev_out]    (1) coin exists     reads the outpoint
4.     check sig(input.sig, coin.script)    (2) authorised     reads script and key
5.     delete utxo[input.prev_out]    (3) not spent twice     reads the outpoint
6.   check sum(in) >= sum(out)    (4) conserved     reads every amount
```

Under the block: fee = Σ in − Σ out.

Say

In Bitcoin the whole state is one table, the set of unspent outputs. A spend is a lookup, a signature check and a deletion. Write the four checked lines and number them; these four numbers are the spine of the workshop, and every later board names the check it serves.

Every check reads a plaintext field of the transaction or of the output it names. Nothing is inferred; everything is looked at. Say what each line reads as you write it: the outpoint, the script and the key, the outpoint again, every amount.

The fee is the fourth check's slack. Sum of inputs minus sum of outputs: not a field, yet public to the satoshi.

Source: *Ironwood Guide*, "Requirements on a shielded payment"; "The note commitment tree"

Board 3 — Hide a field, lose a check

all four

Draw

Keep the pseudocode of the previous board (or rewrite its four checked lines with their numbers). Then, one step at a time, in red:

1. Circle `check sum(in) >= sum(out);` write the first line of Write beside it.
2. Circle `coin = utxo[input.prev_out]` and `delete utxo[input.prev_out]` together; write the second line.
3. Circle `check sig(input.sig, coin.script);` write the third line.
4. Write "Bob's spend, check 2" below the block; draw an arrow to it from the word `script` in line 4; write the fourth line.
5. Box the definition at the bottom of the board.

Write

- hide the amount → check 4 dies: nothing to add up
- hide the input reference → checks 1 and 3 die: nothing to look up, nothing to delete
- hide the key → check 2 dies: nothing to verify against
- hide the receiver → nothing dies here: the output's script is what check 2 reads when Bob spends it, so that spend's check 2 dies

Boxed: *Privacy, for this workshop: hide the sender, the receiver and the amount, while a stranger still performs all four checks.*

Say

Take the plaintext away one field at a time and watch what the node loses. Hide the amount and check 4 dies: nothing to add up. Hide the input reference and checks 1 and 3 die together: nothing to look up, nothing to delete. Hide the key and check 2 dies: nothing to verify against.

Hiding the receiver kills nothing on this transaction. The output's script is what check 2 reads when Bob spends it, so it is that later spend's check 2 that dies. The arrow makes the deferral visible.

This is the definition to leave on the board. Privacy, for this workshop, means hiding the sender, the receiver and the amount while a stranger still performs all four checks. Every mechanism to come answers the same pair of questions: which of the four checks does it restore, and why can the node no longer do it the Bitcoin way?

Source: *Ironwood Guide*, "Requirements on a shielded payment"; "Verification of a transaction"

Board 4 — What a privacy design must supply

all four

Draw

A table, three columns: *the validator must* | *the tool* | *Bitcoin's version*. Four rows, in this order: accept a claim about data it cannot see; know the hidden data cannot change under it; notice the same hidden coin a second time; add up amounts it cannot read. Fill each row across before starting the next.

Write

1. accept a claim about data it cannot see | a zero-knowledge proof: the statement public, the witness (the prover's secret input, not Bitcoin's witness field) private | look at the data
2. know the hidden data cannot change under it | a commitment: hiding, and binding | the output, in the clear
3. notice the same hidden coin a second time | a nullifier: a keyed, one-way tag of the coin, published at spend | delete it from the set
4. add up amounts it cannot read | a homomorphic commitment: sealed envelopes that add | add the numbers

Under the table: *the proof checks the claim; the other three tools make the claim about something fixed, unrepeatable, summable.*

Say

Each dead check names a capability the design must supply. Read the table row by row against Bitcoin's version. Where the Bitcoin node looks at the data, this validator must accept a claim about data it cannot see: a zero-knowledge proof, statement public, witness private. Say at once that "witness" here is the prover's secret input, not Bitcoin's witness field.

The proof alone is not enough: the claim must be about something fixed. Where Bitcoin has the output in the clear, a commitment, hiding and binding, pins the hidden data so it cannot change under the validator.

Where Bitcoin deletes the coin from the set, a nullifier lets the validator notice the same hidden coin a second time: a keyed, one-way tag of the coin, published at spend.

Where Bitcoin adds the numbers, a homomorphic commitment adds sealed envelopes. The next board shows the one line that makes it work.

Emphasise the division of labour. The proof checks the claim; the other three tools make the claim about something fixed, unrepeatable, summable.

Do not say

"The proof hides the sender and the receiver." On this board the proof only checks a claim about hidden data; what hides is the commitment's hiding property and the keyed tag (key re-randomisation joins them on "Four checks, four mechanisms"). Keep the proof in its column.

Source: *Crypto Guide*, "Soundness, zero knowledge, and succinctness"; "Commit and reveal"; *Ironwood Guide*, "The nullifier"

Board 5 — Sealed envelopes that add

conserved

Write

$$([v_1]\,V + [r_1]\,R) + ([v_2]\,V + [r_2]\,R) = [v_1 + v_2]\,V + [r_1 + r_2]\,R$$

Beside it: *holds modulo the group order; the 64-bit range check lives inside the proof.*

Say

The fourth tool in one line: two commitments add without either being opened. Write the sum and point at the right-hand side: the two sealed envelopes combine into an envelope for the sum of the values, with the sum of the randomness.

The room knows this trick. It is the Confidential Transactions trick: amounts hidden, their sum still checkable.

Emphasise the gap and where it is closed. The identity holds only modulo the group order; here the 64-bit range check that closes the gap lives inside the proof.

Source: *Crypto Guide*, "The Pedersen commitment"; *Ironwood Guide*, "Value commitments"; "The Action statement"

Board 6 — Notes versus UTXOs: what a coin is

authorised, conserved

Draw

A table, three columns: (blank) | *Bitcoin UTXO* | *Zcash note*. Three rows, in this order: owner; value; spending condition. Fill each row across.

Write

1. owner | a script, usually one key hash, on chain | an address $(d, \mathsf{pk_d})$, inside the note
2. value | the amount, on chain | $v \in [0, 2^{64})$ zatoshi, inside the note
3. spending condition | script: signatures, timelocks, hashlocks | a key, full stop — no script, no timelocks, no HTLCs

Say

What a coin is, side by side. In Bitcoin the owner is a script, usually one key hash, written on the output, and the value is written beside it. In Zcash both are fields of the note, an address $(d, \mathsf{pk_d})$ and a value v in $[0, 2^{64})$ zatoshi, and the note is never published.

The spending condition is a key, full stop. No script, no timelocks, no HTLCs.

Threshold authority exists and is invisible on chain. It is FROST, later in the workshop; on chain there is one ordinary signature under one per-spend key.

Source: *Ironwood Guide*, "Notes"; "Randomised validating keys"; *FROST Guide*, "Scope"

Board 7 — Notes versus UTXOs: what the chain records

exists, not twice

Draw

A table, three columns: (blank) | *Bitcoin UTXO* | *Zcash note*. Three rows, in this order: on-chain identifier; spent marker; set structure. Fill each row across.

Write

1. on-chain identifier | the outpoint | a commitment cm to the note, stored as cmx
2. spent marker | removed from the UTXO set | a nullifier nf, published at spend
3. set structure | one set: insert, delete | an append-only tree of cmx, depth 32, and a set of nf: append only, both

Under the table: *the note never touches the chain.*

Say

What the chain records, side by side. Bitcoin identifies a coin by its outpoint; Zcash by a commitment cm to the note, stored as cmx. Bitcoin marks a coin spent by removing it from the UTXO set; Zcash by publishing a nullifier nf at spend.

The set structures differ in kind. Bitcoin keeps one set with insert and delete. Zcash keeps two append-only structures: a tree of cmx of depth 32 and a set of nf. Nothing is ever deleted.

Emphasise: the note never touches the chain. Recorded are a commitment to it, an encrypted copy for its recipient, and at spend a tag derived from it.

Source: *Ironwood Guide*, "Notes and note commitments"; "The Merkle hash and the tree"; "Nullifier sets"

Board 8 — The three-line mental model

all four

Write

A shielded spend is three claims about a note nobody sees:

1. *it exists* — a commitment to it is a leaf under a public root, proved with leaf, position and path all private;
2. *it is spent now* — its serial number, the nullifier, is revealed, and the chain refuses to see it twice;
3. *the books balance* — commitments to the amounts add up to the declared net flow.

Under line 1:

$$R_{\text{mem}} = \{ (\underbrace{rt}_{\text{public}} ; \underbrace{(\mathsf{cmx}, i, path)}_{\text{private}}) : \mathsf{Verify}(rt, i, \mathsf{cmx}, path) = \mathsf{accept} \}$$

Say

A shielded spend is three claims about a note nobody sees. Write them as three numbered lines. It exists: a commitment to it is a leaf under a public root, proved with leaf, position and path all private. It is spent now: its serial number, the nullifier, is revealed, and the chain refuses to see it twice. The books balance: commitments to the amounts add up to the declared net flow.

Write the membership relation under line 1. The root is the public part; the commitment, its index and the path are the private witness; the relation holds when the path verifies.

A Bitcoin spend names its coin and signs; a shielded spend names nothing. It proves the first line, reveals the second and signs for the third. And, as in Bitcoin, the spender signs, under a key the proof ties to the note: that is check 2.

Emphasise: everything else in the workshop is machinery for these three lines and for that signature.

Do not say

Letting the position i migrate into line 2. The index is a private witness of line 1 only; the nullifier is a tag derived from the note (the previous board), not from where the note sits in the tree.

Source: *Crypto Guide*, "Privacy-preserving membership via zero-knowledge paths"; "A shielded spend, end to end"; *Ironwood Guide*, "Value commitments and the binding signature"

Board 9 — Four checks, four mechanisms

all four

Draw

A table, four columns: (number) | *the check* | *Bitcoin* | *shielded Zcash*. Four rows in the order (1), (3), (4), (2), the order the workshop takes them. Fill each row across.

Write

1. (1) coin exists | look the outpoint up in the UTXO set | a commitment in an append-only tree; a membership proof under a public root
2. (3) not spent twice | delete it from the set | a nullifier, published once, refused the second time
3. (4) conserved | add the amounts up | value commitments that add; a binding signature over their sum
4. (2) authorised | run the script over the signature | key knowledge proved inside the proof; a signature under a re-randomised key

Say

Each right-hand cell names what replaces one Bitcoin check. Coin exists: a commitment in an append-only tree and a membership proof under a public root. Not spent twice: a nullifier, published once, refused the second time. Conserved: value commitments that add and a binding signature over their sum. Authorised: key knowledge proved inside the proof and a signature under a re-randomised key.

Why the Bitcoin way is gone was settled on the board "Hide a field, lose a check". Point back at it. Each part of the workshop reopens with both questions: which check, and why not the Bitcoin way.

Emphasise the order. Checks 1 and 3 come first, with the data model; checks 4 and 2 follow, on the running example.

Do not say

Glossing the re-randomised key as a multiple of ak. The disguise is additive; its formula comes later, and this table only names it.

Source: *Ironwood Guide*, "Requirements on a shielded payment"; "The Action statement"

Board 10 — The route

all four

Write

The ten parts, as a numbered list:

1. **The toolbox** — each primitive from the Bitcoin instance the room already holds.
2. **The data model** — notes, commitments, the tree, the nullifier: checks 1 and 3.
3. **Keys and addresses** — who can spend, who can see.
4. **The Action statement** — what the proof proves, clause by clause; then a break.
5. **Balance and authority** — checks 4 and 2, on the running example.
6. **The proof system** — from a statement to a few kilobytes.
7. **Delivery and light clients** — how Bob finds his money.

8. **The transaction and the turnstile** — the bytes on the chain, and the pools.
9. **Consensus** — why one fork wins, what the chain must remember.
10. **Limits, frontier, takeaways.**

Under the list: *one example threads it: Alice pays Bob* 1.5 *ZEC from a* 2 *ZEC note.*

Say

The route follows the checks. The toolbox first, each primitive from the Bitcoin instance the room already holds; the data model for checks 1 and 3; keys and addresses; the Action statement clause by clause, then the break; balance and authority for checks 4 and 2; the proof system; delivery; the transaction and the turnstile; consensus; limits and frontier.

One example threads it. Alice pays Bob 1.5 ZEC from a 2 ZEC note: the Bitcoin shape of the first board, without its labels.

Close Part A with the checkpoint. Put the question on the board and take answers before giving yours.

Ask

Which of the four checks does a nullifier serve, and what breaks if two different notes could share one?

- Check 3, not spent twice. The nullifier is "delete from the UTXO set" without naming the coin: published once, refused the second time.
- Two notes, one nullifier: whichever is spent first burns the other, and the second owner's honest spend is refused as a double spend — an attack on earlier designs, in which forged notes could be made to share one.
- How the chain makes sure no two accepted notes can share one — every new note inherits, as its seed, the serial of the note spent beside it — is Part C's business.

Do not say

"A few hundred bytes" for the proof: the route says a few kilobytes, and the proof-system part gives the number. The seed a new note inherits is the serial of the note spent beside it, not a position in the tree.

Source: *Ironwood Guide*, "Nullifiers"

2 Part B — The toolbox, from Bitcoin's primitives up

One board per primitive, or per property of one: what it is in a line, the Bitcoin instance the room already holds, the Zcash instance, the property the protocol spends, and where it returns. The check in each header is the spine; "all four" marks a primitive every check rests on.

Board 11 — Hash functions: one primitive, three notions

all four

Write

1. $H\colon \{0,1\}^* \to \{0,1\}^n$ — fixed, public, compresses
2. preimage: given y, find x' with $H(x') = y$
3. second preimage: given x, find $x' \neq x$ with $H(x') = H(x)$
4. collision: find any $x \neq x'$ with $H(x) = H(x')$ — strongest; commitments, Merkle trees, Fiat–Shamir
5. generic: collision $\approx 2^{n/2}$ (birthday), preimage $\approx 2^n$; $n = 256 \Rightarrow 2^{128}$; RIPEMD-160 $\Rightarrow 2^{80}$
6. Bitcoin: SHA-256d (headers, txids); hash160 = RIPEMD160 $\circ$ SHA256 (addresses)
7. Zcash: keeps all of these (transparent) + three shielded hashes

Say

Start from the primitive the room already trusts. A hash is a fixed public function from arbitrary strings to n bits. Write the three notions in order of strength: preimage, second preimage, collision. In the collision game the adversary picks both inputs, so it is the strongest demand — and the one commitments, Merkle trees and Fiat–Shamir need.

The generic costs set the digest sizes. A collision after about $2^{n/2}$ evaluations, the birthday bound; a preimage after 2^n. That is why a 256-bit digest buys 128-bit collision security, and why RIPEMD-160 sits at 2^{80}.

Bitcoin contrast: nothing is taken away. SHA-256d over every block header and every txid; hash160 inside addresses. Zcash keeps every one of those in its transparent layer and adds three hashes for the shielded one — next board.

Ask

Which of the three notions do commitments and Merkle trees lean on? Collision resistance: the adversary picks both inputs.

Source: *Crypto Guide*, "Security notions: preimage, second-preimage, and collision resistance"; "The birthday bound on generic collision finding"; "The transparent layer's hashes"

Board 12 — Zcash's hashes: SHA-256 is not replaced

all four

Draw

Three columns — *hash*, *kind*, *deployed use* — then four rows, in this order:

1. SHA-256d — bit-oriented, Bitcoin's — header difficulty filter (after Equihash); transparent layer: legacy txids, Base58Check.
2. BLAKE2b — bit-oriented; personalised natively, the key prepended to the input — PRFexpand (the key tree); note-encryption KDF; ZIP-244 digests (txid, sighash).
3. Poseidon — algebraic: $x \mapsto x^5$ over $\mathbb{F}_{p_{\mathsf{Pallas}}}$ — the nullifier PRF, inside the circuit.
4. Sinsemilla — algebraic: curve additions on Pallas — note commitment; Merkle node hash, both inside the circuit.

Write

1. Bitcoin's hashes stay where Bitcoin put them
2. shielded: +1 bit-oriented (off the circuit), +2 algebraic (inside it)

Say

The first row answers the question the room is about to ask. SHA-256d is still there: the header difficulty filter after Equihash, and the transparent layer's legacy txids and Base58Check.

Three hashes are added, and the split is by where the work happens. BLAKE2b, bit-oriented and personalised natively with the key prepended, does the work off the circuit: PRFexpand for the key tree, the note-encryption KDF, the ZIP-244 txid and sighash digests. Poseidon, an algebraic hash built on $x \mapsto x^5$ over $\mathbb{F}_{p_{\mathsf{Pallas}}}$, is the nullifier PRF inside the circuit. Sinsemilla, curve additions on Pallas, is the note commitment and the Merkle node hash, both inside the circuit.

Emphasise the closing line. Bitcoin's hashes stay where Bitcoin put them; the shielded protocol adds one bit-oriented hash for work off the circuit and two algebraic ones for work inside it. Poseidon and Sinsemilla return in Part C; BLAKE2b in Parts D, F and H.

Do not say

"SHA-256 is replaced by BLAKE2b" — it is not replaced; it keeps the header filter and the transparent layer.

Source: *Consensus Guide*, "Equihash and the memoryless model"; *Crypto Guide*, "The transparent layer's hashes"; "PRFs from hash functions: length extension and keyed BLAKE2"; "Arithmetisation-friendly hashing: Poseidon"; *Ironwood Guide*, "Sinsemilla: hash, commitment, and short forms"

Board 13 — Why in-circuit hashing wants algebraic hashes

exists, not twice

Write

1. proof = polynomial constraints over a prime field; field multiplication = 1 constraint; Boolean operation = several + range checks
2. SHA-256, BLAKE2: 32-/64-bit words, XOR, shuffles ⇒ tens of thousands of constraints per evaluation

3. Poseidon round: + constants; x^5 (all elements in a full round, one in a partial); × fixed matrix. 2-to-1 hash ≈ a few hundred constraints. Orchard: width 3, 8 full + 56 partial
4. Sinsemilla, per 10-bit chunk: 1 lookup (table of $2^{10} = 1024$ points) + 2 incomplete additions; collision ⇒ discrete log

Say

Why the shielded layer needs hashes of a different kind. A zero-knowledge proof re-expresses every step of a computation as polynomial constraints over a prime field. A field multiplication is one constraint; a Boolean operation is several, plus range checks.

SHA-256 and BLAKE2 are foreign to the field. They shuffle and XOR 32- and 64-bit words: tens of thousands of constraints per evaluation.

Poseidon keeps the only non-linear step at x^5. Add constants, raise the state to the fifth power — every element in a full round, one element in a partial round — multiply by a fixed matrix. A 2-to-1 hash costs a few hundred constraints. Orchard's instance: state width 3, 8 full and 56 partial rounds.

Sinsemilla is curve arithmetic, native to the circuit. Per 10-bit chunk, one lookup into a table of $2^{10} = 1024$ hashed-to-curve points and two incomplete curve additions; its collision resistance reduces to the discrete logarithm.

Bitcoin contrast: Bitcoin never hashes inside a proof. The first shielded generation did, with SHA-256, and its prover paid for it.

Do not say

Timing claims: "its prover paid for it" stays qualitative; no seconds or minutes for the first generation's prover.

Source: *Crypto Guide*, "Arithmetisation-friendly hashing: Poseidon"; "Sinsemilla: an algebraic hash-based commitment"; *Ironwood Guide*, "Sinsemilla: hash, commitment, and short forms", Remark "Cost inside a proof"; protocol specification, "JoinSplit Statement (Sprout)"

Board 14 — Curves: the same shape, different primes

all four

Write

1. $y^2 = x^3 + b$ over a prime field; prime-order group, cofactor 1; discrete log = the standing assumption
2. secp256k1: $b = 7$, $p = 2^{256} - 2^{32} - 977$, 256-bit prime order
3. Pallas, Vesta: $b = 5$, two 255-bit primes p_{Pallas}, p_{Vesta}; #Pallas $= p_{\mathsf{Vesta}}$, #Vesta $= p_{\mathsf{Pallas}}$; ≈ 126 bits of DL security
4. generators: hashed to the curve (secp256k1's base point: a SEC 2 constant)
5. Pallas: every point in an Action; Vesta: the proof's polynomial commitments

Say

The same shape the room knows. Short Weierstrass $y^2 = x^3 + b$ over a prime field; the points form a group of prime order, cofactor 1; the discrete logarithm in it is the standing assumption.

Bitcoin: secp256k1, $b = 7$ over $p = 2^{256} - 2^{32} - 977$, a 256-bit prime order. The room's keys, ECDSA and BIP-340 live here.

Zcash: Pallas and Vesta, $b = 5$ over two 255-bit primes, with the twist on the board: #Pallas $= p_{\mathsf{Vesta}}$ and #Vesta $= p_{\mathsf{Pallas}}$ — the scalar field of each is the base field of the other. About 126 bits of discrete-log security.

Every generator is hashed to the curve. The secp256k1 base point is a SEC 2 constant; every Pallas and Vesta generator the protocol uses is hashed, so no relation between any two is known to anyone, and there is no ceremony for this stack. Say the caveat in the same breath: the Sapling pool still rests on its Groth16 MPC (Part G).

Division of labour, explained in Part G: Pallas carries every point in an Action — commitments, nullifier base, keys, ephemeral keys, signatures; Vesta carries the proof's polynomial commitments.

Do not say

"No ceremony" without the Sapling caveat: no ceremony for this stack; the Sapling pool still rests on its Groth16 MPC.

Source: *Crypto Guide*, "Instantiation on elliptic curves; the Pasta curves"; "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"; *Math Guide*, "Pallas and Vesta assembled"

Board 15 — Why not secp256k1: two-adicity

all four

Draw

Four columns — *field*, *bits*, *two-adicity of* $p - 1$, *largest power-of-two domain* — then four rows, in this order:

1. Pallas base field — 255 — 32 — 2^{32}.
2. Vesta base field — 255 — 32 — 2^{32}.
3. secp256k1 base field — 256 — 1 — 2.
4. secp256k1 scalar field — 256 — 6 — 64.

Write

1. 2^k-th roots of unity exist iff $2^k \mid p - 1$; largest such k = two-adicity
2. Orchard circuit: 2^{11} rows
3. secp256k1 rows: our own computation, not a volume figure

Say

Why not simply use secp256k1? A proof system of the PLONK family interpolates each circuit column over the 2^k-th roots of unity of its field. Those exist iff $2^k \mid p - 1$; the largest such k is the field's two-adicity.

Read the table. The Pallas and Vesta base fields: 255 bits, two-adicity 32, a domain of 2^{32}. The secp256k1 base field: two-adicity 1, a domain of size 2; its scalar field: 6, a domain of 64.

The Orchard circuit has 2^{11} rows. Neither secp256k1 field offers a domain of that size. The Pasta primes were searched for with two-adicity 32 and the cycle as design goals.

Label the provenance. The secp256k1 rows are the deck's own computation, not a volume figure; say so. Rows, roots of unity and the polynomial identity return in Part G.

Ask

Which secp256k1 field could host 2^{11} rows? Neither: the largest domains are 2 and 64.

Source: *Math Guide*, "Roots of unity and evaluation domains", Remark "Two-adicity and FFT-friendly primes"; "Base fields, scalar fields, and the Pasta cycle"; *Crypto Guide*, "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"

Board 16 — Commitments: a sealed envelope

exists, conserved

Write

1. $c = \mathsf{Commit}(m; r)$, trapdoor r; hiding: two candidate m indistinguishable from c; binding: no c opens to two m
2. not both perfect; deployed: perfectly hiding, binding under DL
3. $\mathsf{Commit}(v; r) := [v]\,V + [r]\,R$, with V, R hashed to Pallas
4. uniform $r \Rightarrow$ uniform point; two openings $\Rightarrow \log_V R$
5. Bitcoin: CT's envelope; an address = hash commitment to a key/script, opened at spend
6. Zcash: cv seals the value change (Pedersen, adds); cm seals the whole note (algebraic hash $+[\mathsf{rcm}]\,H_D$)

Say

A commitment is a sealed envelope. Commitment $c = \mathsf{Commit}(m; r)$ with a random trapdoor r. Hiding: from c, two candidate messages are indistinguishable. Binding: no c opens to two messages. The two cannot both be perfect; the deployed schemes are perfectly hiding and binding under the discrete logarithm.

Pedersen on Pallas, with V and R hashed to the curve. Write $\mathsf{Commit}(v; r) := [v]\,V + [r]\,R$. For uniform r the point is uniform whatever v is — that is hiding. Two openings of one c solve $\log_V R$, which nobody holds — that is binding.

Bitcoin contrast. This envelope is Confidential Transactions'; and an address is already a hash commitment to a public key or a script, opened at spend.

Zcash has two envelopes per Action. The value commitment cv seals an Action's value change — Pedersen, so it adds. The note commitment cm seals a whole note: an algebraic hash plus a blinding term $[\mathsf{rcm}]\,H_D$ of the same Pedersen shape, over its own hashed generator, so the circuit recomputes it cheaply.

Ask

Can the envelope be perfectly hiding and perfectly binding at once? No; the deployed ones are perfectly hiding and binding under the discrete logarithm.

Source: *Crypto Guide*, "Hiding and binding"; "The Pedersen commitment"; "Sinsemilla: an algebraic hash-based commitment"; *Ironwood Guide*, "Sinsemilla: hash, commitment, and short forms"

Board 17 — The envelope adds: the Confidential Transactions bridge

conserved

Write

1. $\mathsf{Commit}(v_1; r_1) + \mathsf{Commit}(v_2; r_2) = \mathsf{Commit}(v_1 + v_2;\ r_1 + r_2)$
2. $$\sum \mathsf{Commit}_{\text{in}} - \sum \mathsf{Commit}_{\text{out}} = \Big[\sum v_{\text{in}} - \sum v_{\text{out}}\Big]V + [\bar{r}]\,R$$
3. V term vanishes iff values balance — modulo the group order; range check on every value closes the wrap
4. CT (Elements): same equation on chain; range proof beside each commitment; receiver told the opening; amounts hidden, keys and graph clear
5. here: 64-bit range check + opening (v, rcv) = clauses of one ZK proof; net trapdoor $\bar{r} \to$ a signing key

Say

The envelope adds. Two commitments sum to a commitment to the summed values under the summed trapdoors. Sum the inputs, subtract the outputs: the V term carries $\sum v_{\text{in}} - \sum v_{\text{out}}$ and vanishes exactly when the values balance — modulo the group order. A range check on every value closes the wrap.

Bitcoin contrast: Confidential Transactions (Elements). The same equation on the chain; a range proof beside each commitment; the receiver is told the opening. Amounts hidden, keys and graph in the clear.

Here the equation moves inside the proof. The 64-bit range check and the opening (v, rcv) are clauses of one zero-knowledge proof, alongside the sender's authorisation and the receiver's note; the net trapdoor $\bar{r}$ becomes a signing key — the board *Preview: a signature under a key nobody holds unless the values balance* shows how. Clause A4 in Part E; the balance walk-through in Part E′.

Source: *Crypto Guide*, "The Pedersen commitment"; "Binding signatures as a proof of knowledge of a discrete logarithm"

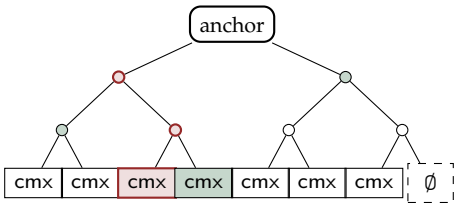
Board 18 — Merkle trees: from the txid tree to the note commitment tree

exists

Draw

1. At the top, a rounded box labelled *anchor*.
2. Under it two circles; under each, two more; under each of those, two small boxes — eight leaves. Label the leaves cm× left to right; draw the eighth dashed and label it $\emptyset$, the empty slot.
3. In red: the third leaf, its parent circle and its grandparent (the left level-one circle) — the spent note and its route to the root.
4. In green: the sibling at each level of that route — the fourth leaf, the first level-two circle, the right level-one circle.

5. Legend under the tree: red = the leaf and its route, private; green = the siblings, private; the anchor alone is public.



red: the leaf and its route, private; green: the siblings, private; the anchor alone is public

Write

1. Bitcoin: txids pairwise up to the header root; SPV proof = leaf + one sibling per level; the leaf (txid) is public
2. Zcash: leaves = cmx in creation order; depth 32, room for 2^{32} notes; node hash Sinsemilla, tagged with its layer; root after each block = that block's *anchor*
3. membership proof: 32 siblings + 32 direction bits, verified by 32 hashes — inside the proof

Say

Bitcoin's tree the room knows. Each block hashes its txids pairwise up to a root in the header; an SPV proof is the leaf plus one sibling per level — and the leaf, the txid, is public.

Zcash's tree. The leaves are cmx in creation order; depth 32, room for 2^{32} notes; the node hash is Sinsemilla, tagged with its layer; the root after each block is that block's anchor.

A membership proof is 32 **siblings and** 32 **direction bits, verified by** 32 **hashes** — recomputed here inside the proof. Point at the tree: red is the leaf and its route, private; green the siblings, private; the anchor alone is public.

Source: *Crypto Guide*, "Authentication paths and membership proofs"; *Ironwood Guide*, "The Merkle hash and the tree"

Board 19 — What changes: append only, historical roots, private path

exists

Write

1. append only: one leaf at a time; frontier ≤ 32 carried labels; root updated in 32 hashes; nothing deleted — "deleted" = a separate set
2. history binding: proof against the root at insertion valid forever; consensus accepts the anchor of any earlier main-chain block
3. private path: (leaf, index, siblings) = witness; anchor = statement
4. $$R_{\text{mem}} = \{\, (rt;\ (\mathsf{cmx}, i, path)) : \mathsf{Verify}(rt, i, \mathsf{cmx}, path) = \mathsf{accept} \,\}$$
5. Bitcoin SPV names the txid; this proof names only the root

Say

Append only. One leaf at a time; the frontier — at most 32 carried labels — updates the root in 32 hashes. Nothing is ever deleted; the "deleted" half of the state is a separate set.

History binding. A proof formed against the root at insertion stays valid against that root forever, and consensus accepts the anchor of any earlier main-chain block.

Private path. Revealing (leaf, index, siblings) would say which note is spent; so the path is a witness of the zero-knowledge proof and only the anchor is a statement. Write the relation: the statement is the root rt, the witness is $(\mathsf{cmx}, i, path)$.

Bitcoin contrast: an SPV proof names the txid it proves; this proof names only the root and hides the leaf among every leaf beneath it. The tree, anchors and the nullifier set in Part C; clause A3 in Part E.

Source: *Crypto Guide*, "Append-only and incrementally updatable trees"; "Privacy-preserving membership via zero-knowledge paths"; *Ironwood Guide*, "The note commitment tree"

Board 20 — Pseudorandom functions: a key that fakes randomness

not twice

Write

1. F_k pseudorandom: secret uniform k; no efficient adversary with oracle access (any inputs, adaptively) tells F_k from a random function
2. algorithm public; only k hidden; adversary sees input–output pairs, never k
3. Bitcoin: HMAC-SHA512 in BIP-32 child derivation
4. Zcash, off circuit: BLAKE2b personalised = PRFexpand; $\mathsf{sk} \to \mathsf{ask}, \mathsf{nk}, \mathsf{r_{ivk}}$; later the note's randomness from its seed
5. Zcash, in circuit: $F_{\mathsf{nk}}(\rho) = \mathsf{PoseidonHash}(\mathsf{nk}, \rho)$ over $\mathbb{F}_{p_{\mathsf{Pallas}}}$; width 3, S-box x^5, 8 full + 56 partial rounds

Say

A PRF is a key that fakes randomness. A keyed function F_k is pseudorandom if, for a secret uniform k, no efficient adversary with oracle access — any inputs, adaptively — tells F_k from a truly random function. The algorithm is public; only k is hidden; the adversary sees input–output pairs of her choosing and never k.

Bitcoin: HMAC-SHA512 inside BIP-32 child derivation — a PRF the room already trusts to make child keys look unrelated.

Zcash, off the circuit: BLAKE2b, personalised, as PRFexpand — one call per sub-secret, $\mathsf{sk} \to \mathsf{ask}, \mathsf{nk}, \mathsf{r_{ivk}}$, and later the note's randomness from its seed. It builds the key ladder in Part D.

Zcash, inside the circuit: Poseidon over $\mathbb{F}_{p_{\mathsf{Pallas}}}$, keyed by its first input — nk first, ρ second, as written on the board; state width 3, S-box x^5, 8 full and 56 partial rounds.

Source: *Crypto Guide*, "Pseudorandom functions and permutations"; "PRFs from hash functions: length extension and keyed BLAKE2"; "Arithmetisation-friendly hashing: Poseidon"

Board 21 — What a nullifier needs from a PRF

not twice

Write

1.
$$\mathsf{nf} = \mathsf{Extract}([\, F_{\mathsf{nk}}(\rho) + \psi \,]\, G^{\mathsf{nf}} + \mathsf{cm})$$

2. Extract: the x-coordinate; ρ, ψ: per-note randomness fixed at creation; G^{nf}: a hashed generator
3. uniqueness: all inputs fixed once the note exists ⇒ deterministic; two notes, one tag ⇒ DL relation on Pallas; no PRF property used (forger holds nk)
4. unlinkability: without nk, $[F_{\mathsf{nk}}(\rho) + \psi]\, G^{\mathsf{nf}}$ masks cm — Poseidon PRF; DDH on Pallas only the specification's aim, with an entropy assumption
5. Bitcoin: spent marker = the outpoint, unique and public

Say

Preview the nullifier and name what it needs. Write the formula. The function Extract is the x-coordinate; ρ and ψ are per-note randomness fixed at creation (Part C); G^{nf} is a hashed generator.

Uniqueness. Every input — nk, ρ, ψ, cm — is fixed once the note exists, so the tag is deterministic; two distinct notes sharing a tag yield a discrete-log relation on Pallas. No PRF property is used here: the forger holds nk.

Unlinkability. Without nk the term $[F_{\mathsf{nk}}(\rho) + \psi]\, G^{\mathsf{nf}}$ masks cm with a value that looks random — by Poseidon's PRF security. DDH on Pallas is only the specification's backup aim, under an extra entropy assumption.

Bitcoin contrast: the spent marker is the outpoint itself, unique and public; a nullifier must be unique *and* say nothing. The two properties are separate — the checkpoint at the end of the part turns on that. The nullifier in full in Part C; clause A5 in Part E.

Ask

Which of the two properties uses the PRF? Only unlinkability; uniqueness is discrete-log structure, and the forger holds nk anyway.

Do not say

The seed ρ as a tree position: it is per-note randomness fixed at creation, and the leaf index never enters the nullifier.

Source: *Ironwood Guide*, "The nullifier"; "Nullifier uniqueness"

Board 22 — Key agreement: the ECDH of silent payments

delivery, not a check

Write

1. base point P; secret a, public $[a]P$; $\text{Agree}(a, Q) = [a]Q$; $[a]([b]P) = [b]([a]P)$
2. BIP-352: sender's input key + receiver's scan key → shared secret → tweaks the output key; nothing encrypted
3. Zcash address: $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$, $\mathsf{g_d}$ hashed from the diversifier
4. sender: ephemeral esk (from the note's seed, PRFexpand); publishes $\mathsf{epk} = [\mathsf{esk}]\,\mathsf{g_d}$
5. $\mathsf{s} = [\mathsf{esk}]\,\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{epk}$
6. cofactor 1 ⇒ only on-curve, non-identity check on epk

Say

The ECDH the room knows from silent payments. On a prime-order curve with base point P: a secret a, a public $[a]P$, and $\text{Agree}(a, Q) = [a]Q$, so that $[a]([b]P) = [b]([a]P)$.

Bitcoin (BIP-352): the sender's input key and the receiver's scan key agree on a secret that tweaks the output key. Nothing is encrypted; the receiver scans for outputs that decode under the shared secret.

Zcash: the address carries a static key $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$ over a per-address base $\mathsf{g_d}$, itself hashed from the diversifier. The sender draws an ephemeral esk — from the note's seed, by PRFexpand — and publishes $\mathsf{epk} = [\mathsf{esk}]\,\mathsf{g_d}$. Both sides reach $\mathsf{s} = [\mathsf{esk}]\,\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{epk}$.

Static–ephemeral, and why it suits a chain. The recipient was never online, and the sender learns nothing beyond the address. Cofactor 1 leaves only the on-curve, non-identity check on epk. Note encryption step by step in Part F.

Source: *Crypto Guide*, "Elliptic-curve Diffie–Hellman"; "Shielded-note delivery as deployed hybrid encryption"; *Ironwood Guide*, "Encryption to the recipient"

Board 23 — Encryption on top: KDF, AEAD, wrong-key rejection

delivery, not a check

Write

1.
$$K_{\text{enc}} = \text{BLAKE2b-256}(\text{“Zcash_OrchardKDF”};\ \mathsf{s} \,\|\, \mathsf{epk})$$

2. AEAD: ChaCha20-Poly1305; 256-bit key; nonce all zero; associated data empty; 128-bit tag; each key encrypts exactly once
3. wrong key ⇒ $\perp$, except with probability $\approx 2^{-128}$; not key-committing ⇒ recompute cm and compare
4. Bitcoin: nothing encrypted on chain; here recipient, value, memo exist on chain only inside the ciphertext
5. two ciphertexts per Action: to the recipient; to the sender's own outgoing key

Say

KDF. One personalised hash call over the shared secret and epk; folding epk in binds the key to this ephemeral.

AEAD. ChaCha20-Poly1305 under the 256-bit key, nonce all zero and associated data empty — safe because each key encrypts exactly once — with a 128-bit tag.

Wrong-key rejection is the property the wallet spends. Decryption under an independent key returns $\perp$ except with probability about 2^{-128}. This is what lets a wallet try every output on the chain with its own key. The scheme is not key-committing, so the wallet also recomputes cm from the plaintext and compares.

Bitcoin never encrypts anything on chain: scripts and amounts are plaintext by design. Here the recipient, the value and the memo exist on chain only inside the ciphertext. Two ciphertexts per Action: one to the recipient, one to the sender's own outgoing key. Trial decryption and compact blocks in Part F.

Ask

Why is an all-zero nonce safe? Each key encrypts exactly once.

Do not say

Asked where the sender's outgoing key comes from: not from ivk — Part D. Asked whether that second ciphertext is how the sender finds her change: no — Part F.

Source: *Crypto Guide*, "Key-derivation functions"; "The ChaCha20-Poly1305 AEAD"; *Ironwood Guide*, "Encryption to the recipient"

Board 24 — Signatures: ECDSA, Schnorr, RedPallas

authorised

Write

1. Bitcoin: ECDSA (r, s), secp256k1, DER + sighash-type byte; Taproot: BIP-340 Schnorr, $s = k + e\,d$, $e = H(x(R) \,\|\, x(X) \,\|\, m)$
2. Zcash transparent: the same ECDSA (sighash as message, DER, low-s); no Taproot, no BIP-340
3. Zcash shielded: RedPallas = Schnorr over Pallas, base point P a parameter; $X = [x]P$
4. $$\begin{aligned} \text{Sign:} \quad & r = H(T \,\|\, X \,\|\, m) \text{ for 80 fresh random bytes } T,\ R = [r]P, \\ & c = H(R \,\|\, X \,\|\, m), \quad s = r + c\,x; \\ \text{Verify:} \quad & [s]P = R + [c]X \end{aligned}$$

   H: BLAKE2b-512, personalised, reduced mod the group order
5. two instantiations, differing only in P: $G^{\mathsf{SpendAuth}}$ and the value-commitment base R

Say

Bitcoin's two signatures. ECDSA (r, s) over secp256k1, DER plus a sighash-type byte; under Taproot, BIP-340 Schnorr, $s = k + e\,d$ with $e = H(x(R) \,\|\, x(X) \,\|\, m)$.

Zcash, transparent: the same ECDSA — sighash as message, DER, low-s. No Taproot, no BIP-340 in the transparent layer.

Zcash, shielded: RedPallas — Schnorr over Pallas with the base point P a parameter of the scheme. Keys $X = [x]P$. Write Sign and Verify: the nonce r is hashed from 80 fresh random bytes T with X and m; $R = [r]P$; $c = H(R \,\|\, X \,\|\, m)$; $s = r + c\,x$; verify $[s]P = R + [c]X$. The hash H is BLAKE2b-512, personalised, reduced modulo the group order.

Two instantiations, differing only in P: the spend-authorisation base $G^{\mathsf{SpendAuth}}$ and the value-commitment base R. The next three boards use one each.

Source: *Crypto Guide*, "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"; "RedDSA: re-randomisable Schnorr for Orchard"

Board 25 — The need: one spend key per wallet, not per coin

authorised

Write

1. Bitcoin: one key per output; reuse = a choice; each reuse links two payments
2. Zcash: one spend validating key per wallet, $\mathsf{ak} = [\mathsf{ask}]\,G^{\mathsf{SpendAuth}}$
3. $$\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk}), \qquad \mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$$
4. un-randomised spend ⇒ publishes ak for every note the wallet ever spends: worse than address reuse, structural
5. fresh disguise per spend; Schnorr key = $[x]P$, homomorphic ⇒ shiftable without the secret

Say

Bitcoin: one key per output. Reuse is a wallet's choice, and each reuse links two payments.

Zcash: one spend validating key per wallet. The key $\mathsf{ak} = [\mathsf{ask}]\,G^{\mathsf{SpendAuth}}$, and every diversified address shares it: write ivk as a commitment to $\mathsf{Extract}(\mathsf{ak})$ and nk, and $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$.

So an un-randomised spend would publish ak — the same key for every note the wallet ever spends. Linkage worse than address reuse, and structural rather than chosen.

Hence a fresh disguise per spend. The public key must be shiftable without the secret; a Schnorr key is the homomorphic image $[x]P$ of its secret, so it can be. The ladder that makes every address share ak: Part D.

Do not say

Multiplicative re-randomisation: the disguise on the next board is additive, never $[\alpha]\,\mathsf{ak}$.

Source: *Crypto Guide*, "Key re-randomisation and unlinkability"; *Ironwood Guide*, "Viewing keys"; "Diversified addresses"

Board 26 — Key re-randomisation, additive

authorised

Write

1. fresh uniform α per spend — added, never multiplied in
2. $$\mathsf{rsk} := \mathsf{ask} + \alpha, \qquad \mathsf{rk} := \mathsf{ak} + [\alpha]\,G^{\mathsf{SpendAuth}} = [\mathsf{rsk}]\,G^{\mathsf{SpendAuth}}$$
3. consistency: $(\mathsf{rsk}, \mathsf{rk})$ an ordinary key pair
4. unlinkability: uniform $\alpha \Rightarrow \mathsf{rk}$ uniform, whatever ak — perfect, no assumption; α fresh and secret
5. signing needs ask and α; circuit proves rk re-randomises the ak in the note's address; node checks the signature under rk
6. BIP-32 non-hardened child = pubkey + [tweak] G; tweak recomputable from the xpub — α never published

Say

Additive, never multiplicative. For a fresh uniform α per spend: $\mathsf{rsk} := \mathsf{ask} + \alpha$ and $\mathsf{rk} := \mathsf{ak} + [\alpha]\,G^{\mathsf{SpendAuth}} = [\mathsf{rsk}]\,G^{\mathsf{SpendAuth}}$.

Consistency. The pair $(\mathsf{rsk}, \mathsf{rk})$ is an ordinary key pair; a signature under rsk verifies under rk.

Unlinkability. For uniform α the key rk is uniform on the group whatever ak is — perfect, no assumption, provided α is fresh and stays secret.

Who does what. Signing under rk needs both ask and α. The circuit proves rk is a re-randomisation of the ak bound into the note's address; the node checks the signature under rk.

Bitcoin bridge: a non-hardened BIP-32 child is pubkey + [tweak] G too — but anyone holding the parent xpub (key, chain code, index) recomputes its tweak; this one is uniform and never published. Clauses A6/A7 in Part E; the signature on the wire in Part E′.

Ask

Who can recompute rk from ak? Nobody without α — unlike the BIP-32 tweak, which anyone holding the xpub recomputes.

Do not say

Multiplicative re-randomisation: $\mathsf{rk} = \mathsf{ak} + [\alpha]\,G^{\mathsf{SpendAuth}}$, never $[\alpha]\,\mathsf{ak}$.

Source: *Crypto Guide*, "Key re-randomisation and unlinkability"; *Ironwood Guide*, "Randomised validating keys"

Board 27 — Preview: a signature under a key nobody holds unless the values balance

conserved

Write

1. $$\mathsf{bvk} = \sum \mathsf{cv} - [\mathsf{valueBalance}]\,V \quad \text{(public)}$$

2. balanced ⇒ V terms cancel, $\mathsf{bvk} = [\sum \mathsf{rcv}]\,R$, author knows $\mathsf{bsk} = \sum \mathsf{rcv}$
3. not balanced ⇒ $\mathsf{bvk} = [\beta]\,R$ would reveal $\log_R V$ — nobody holds it
4. binding signature: RedPallas under bsk, verified under bvk, over the sighash; knowing the log = "the values balance"
5. range checks in the proof: balance mod group order → balance over the integers

Say

Sum the envelopes and fold in the public balance. Write bvk; it is public.

Every group element is some multiple of R; what matters is who can compute the multiplier. If the hidden values balance, the V terms cancel, $\mathsf{bvk} = [\sum \mathsf{rcv}]\,R$, and the author knows $\mathsf{bsk} = \sum \mathsf{rcv}$. If not, writing bvk as any multiple $[\beta]\,R$ would reveal $\log_R V$, which nobody holds.

The binding signature. RedPallas under bsk, verified under bvk, over the sighash. A Schnorr signature proves knowledge of a discrete logarithm; here knowing it means "the values balance". Range checks in the proof lift balance modulo the group order to balance over the integers.

Bitcoin's check 4 reads the amounts; here they stay sealed and a signature certifies their sum. The running example's balance, in a toy group, in Part E′.

Source: *Crypto Guide*, "Binding signatures as a proof of knowledge of a discrete logarithm"

Board 28 — Zero-knowledge proofs, first contact

all four

Write

1. relation R of pairs (x, w): statement x public, witness w private; claim: "I know w with $(x, w) \in R$"
2. complete: honest prover with w convinces
3. sound: no prover convinces on a false x — *argument* if only efficient provers are excluded (price of succinctness)
4. zero knowledge: a simulator without w reproduces the verifier's whole view; nothing beyond x leaks
5. knowledge sound: whoever convinces can be rewound into surrendering w
6. "some spendable note exists" — true for everyone; "I hold one" — the claim

Say

Statement, witness, relation. A relation R of pairs (x, w): the statement x is public, the witness w private; the claim is "I know w with $(x, w) \in R$".

Four properties, written in order. Complete: an honest prover holding w convinces. Sound: no prover convinces on a false x — an argument if only efficient provers are excluded, the price of succinctness. Zero knowledge: a simulator without w reproduces the verifier's whole view; nothing beyond x leaks. Knowledge sound: whoever convinces can be rewound into surrendering w.

Knowledge soundness is the one this protocol needs. "Some spendable note exists in the tree" is true for everyone; "I hold one" is the claim.

Do not say

"The proof hides sender and receiver": zero knowledge says nothing beyond x leaks, no more; the board *What the proof hides, and what it does not* says what keeps the public fields unlinkable.

Source: *Crypto Guide*, "Languages, relations, and witnesses"; "Interactive protocols, proofs, and arguments"; "Knowledge soundness and extractors"; "The simulation paradigm and zero knowledge"

Board 29 — A signature on a statement

all four

Write

1. Fiat–Shamir: challenge = $H(\text{statement} \,\|\, \text{transcript so far})$; one string carries the exchange; anyone re-derives the challenges
2. proof of knowledge + Fiat–Shamir = signature
3. Schnorr = proof of knowledge of x in $X = [x]G$, message folded into the challenge hash

4. Bitcoin: one equation per signature; here: one fixed circuit's verifying key against the statement, per proof

Say

Interaction removed by Fiat–Shamir. Each challenge becomes a hash of the statement and the transcript so far, so one string carries the whole exchange and anyone can re-derive the challenges.

Proof of knowledge plus Fiat–Shamir equals signature. The room's Schnorr signature is exactly this: a proof of knowledge of x in $X = [x]G$, with the message folded into the challenge hash.

Bitcoin contrast: Bitcoin verifies one equation per signature; this verifier runs one fixed circuit's verifying key against the statement, per proof.

Source: *Crypto Guide*, "The Fiat–Shamir transform: from interactive to non-interactive"; "SNARKs: succinct non-interactive arguments of knowledge"

Board 30 — A SNARK: succinct, hence an argument

all four

Write

1. SNARK = the same move on a whole statement: "this note is in the tree, this nullifier is its, these values balance"
2. proof size and verifier time sublinear in the computation
3. Halo 2 as deployed: the proof is succinct; the verifier's linear part is shared across a batch
4. succinct ⇒ argument: proof shorter than the witness cannot pin it down against an unbounded prover; soundness against efficient provers, under an assumption

Say

A SNARK is the same move on a whole statement — "this note is in the tree, this nullifier is its, these values balance" — with the proof size and the verifier's time sublinear in the computation.

Halo 2, as deployed: the proof is succinct; the verifier's linear part is shared across a batch of proofs (Part G).

Succinctness forces an argument. A proof shorter than the witness cannot pin the witness down against an unbounded prover, so soundness holds against efficient provers, under an assumption. The statement in Part E; the proof system in Part G.

Do not say

Timing claims: "sublinear" is the claim; no millisecond figure for verification, no seconds for proving.

Source: *Crypto Guide*, "SNARKs: succinct non-interactive arguments of knowledge"; *Ironwood Guide*, "The Action circuit and the Halo 2 proof"

Board 31 — Proof sizes, with units

all four

Draw

Five columns — *generation*, *system*, *one proof covers*, *bytes*, *running example* — then three rows, in this order:

1. Sprout — BCTV14 — one JoinSplit: 2 in, 2 out — 296 B — 1 JoinSplit, 296 B.
2. Sapling — Groth16 — one Spend or one Output description — 192 B each — 1 spend + 2 outputs, 576 B.
3. Orchard — Halo 2 — the whole bundle, a Actions — $2720 + 2272\,a$ B — $a = 2$ in the Ironwood pool, 7264 B.

Write

1. $a = 1$: 4992 B; Alice: one proof for both Actions, 4992 + 2272 = 7264 B
2. curves: BN254 → BLS12-381 → Pallas/Vesta; pairing proofs constant-size; growth to Halo 2 = the price of no trusted setup
3. Sprout's row: its original system; it later moved to Groth16

Say

Sizes with units, one row per generation. Sprout, BCTV14: one proof per JoinSplit, 296 B. Sapling, Groth16: one per Spend and one per Output description, 192 B each. Orchard, Halo 2: one proof for the whole bundle, $2720 + 2272\,a$ bytes — 4992 B for $a = 1$.

The running example in each. Sprout: 1 JoinSplit, 296 B. Sapling: 1 spend + 2 outputs, 576 B. Ironwood: $a = 2$, 7264 B — one proof for both Actions, not two: 4992 bytes plus 2272 for the second.

Curves, and the price of no setup. BN254, then BLS12-381, then Pallas/Vesta. Pairing proofs are constant-size; the growth to Halo 2 is the price of no trusted setup — for this stack; the Sapling pool still rests on its Groth16 MPC. Sprout's row is its original system; it later moved to Groth16. Where the bytes go in Part G; the transaction's size in Part H.

Do not say

"A few hundred bytes" — say the numbers with units. "No trusted setup" without the Sapling caveat. A minimum-actions constant as the reason for $a = 2$: the two outputs fix it, and one dummy spend pads the spend list (Part E).

Source: *Ironwood Guide*, "The transaction format"; *Crypto Guide*, "SNARKs: succinct non-interactive arguments of knowledge"; protocol specification, "JoinSplit Descriptions"; "Constants"

Board 32 — What the proof hides, and what it does not

all four

Write

1. transcript reveals nothing beyond the public inputs: anchor, cv, nf, rk, cmx, flags — that is all of ZK
2. necessary, not sufficient: the public fields must not link on their own; the proof cannot help them
3. cm, cv: hiding — uniform for a uniform trapdoor
4. nf: keyed PRF — random-looking without nk
5. rk: re-randomisation — uniform for a uniform α
6. epk, ciphertexts: key agreement + KDF + AEAD
7. Bitcoin signature hides its key's secret, nothing else; here every field is a disguise, the proof certifies they are consistent

Say

What zero knowledge buys, exactly. The transcript reveals nothing beyond the public inputs of the Action — anchor, cv, nf, rk, cmx, flags. That is the whole of zero knowledge.

Necessary, not sufficient. The public fields must not link on their own, and the proof cannot help them.

Field by field, what keeps them unlinkable. The commitments cm, cv: hiding — uniform for a uniform trapdoor. The nullifier nf: the keyed PRF — random-looking without nk. The key rk: re-randomisation — uniform for a uniform α. The ephemeral key epk and the ciphertexts: key agreement plus the KDF and AEAD.

Bitcoin contrast: a Bitcoin signature hides its key's secret and nothing else; the transaction around it is plaintext by design. Here every published field is a disguise, and the proof certifies that the disguises are consistent.

Do not say

"The proof hides sender and receiver": the proof hides the witness; the hiding of the published fields is the commitments', the PRF's, the re-randomisation's and the encryption's work.

Source: *Ironwood Guide*, "The Action circuit and the Halo 2 proof"; "Privacy"; *Crypto Guide*, "Hiding and binding"; "Key re-randomisation and unlinkability"

Board 33 — The instantiation table

all four

Draw

Four columns — *primitive*, *Bitcoin*, *Zcash*, *in the Action; part* — then nine rows, in this order; ask the room for the Bitcoin cell before writing the Zcash one:

1. hash — SHA-256d, hash160 — BLAKE2b; Poseidon; Sinsemilla — keys, nf, cmx; C, D.
2. curve — secp256k1 — Pallas (Action), Vesta (proof) — every point; G.
3. commitment — CT Pedersen; key/script hash — cv Pedersen; cm Sinsemilla — cv, cmx; C, E.
4. Merkle tree — txid tree — commitment tree, depth 32 — anchor, A3; C, E.
5. PRF — HMAC-SHA512 — PRFexpand; Poseidon — nf, A5; C, D.
6. key agreement — BIP-352 ECDH — ECDH to g_d — epk; F.
7. encryption — none — ChaCha20-Poly1305 — ciphertexts; F.
8. signature — ECDSA; BIP-340 — ECDSA (transparent); RedPallas — rk, two signatures; E, E′.
9. ZK proof — none — Halo 2 zk-SNARK per bundle — the proof; E, G.

Say

Close Part B by filling the table row by row. Every row is a board already on the wall; the Bitcoin column is the room's, the Zcash column is this part's, and the final column says where the primitive lands in the Action and which part returns to it.

The last column is the map for the rest of the workshop. The data model in Part C, the key ladder in Part D, the statement in Part E, balance in Part E′, delivery in Part F, the proof system in Part G.

Ask

Checkpoint — why can't the nullifier simply be the hash of the note commitment?

- The commitment is public: it is the leaf. Anyone could hash every leaf and match the results against published nullifiers — each spend would point at its mint, and the transaction graph would return.
- Uniqueness would survive (distinct commitments, distinct hashes); unlinkability would not. The two properties are separate, and the second needs a secret.
- Deployed: $\mathsf{nf} = \mathsf{Extract}([\,F_{\mathsf{nk}}(\rho) + \psi\,]\,G^{\mathsf{nf}} + \mathsf{cm})$ — keyed by nk, so only its holder can compute it; the discrete-log structure keeps uniqueness provable, and Poseidon's PRF security gives unlinkability.

Do not say

The seed ρ as a tree position, when writing the deployed formula in the answer.

Source: *Crypto Guide*, "The transparent layer's hashes"; "Instantiation on elliptic curves; the Pasta curves"; "Shielded-note delivery as deployed hybrid encryption"; "RedDSA: re-randomisable Schnorr for Orchard"; *Ironwood Guide*, "The Merkle hash and the tree"; "The nullifier"

3 Part C — The data model

Board 34 — A note: the shielded unit of value

exists

Draw

The table, one row at a time (field, job, in Bitcoin):

1. Header: *field* — *job* — *in Bitcoin*.
2. $(d, \mathsf{pk_d})$ — the recipient's diversified address: an 88-bit diversifier and a transmission key (Part D) — scriptPubKey.
3. v — the value, $0 \le v < 2^{64}$ zatoshi — amount.
4. ρ — a per-note uniqueness seed, fixed at creation (where it comes from: the nullifier, later in this part) — none.
5. ψ — nullifier salt, derived from the note seed rseed — none.
6. rcm — commitment trapdoor, derived from rseed; for an Ironwood note, last and from every other field (Part F) — none.

Write

Above the table, the private object:

$$\mathbf{n} := (d,\ \mathsf{pk_d},\ v,\ \rho,\ \psi,\ \mathsf{rcm})$$

Below it: "Bitcoin publishes every field of an output in clear; every field of a note stays with its owner."

Say

In Bitcoin the coin is the output, and the output is public. The node reads amount and scriptPubKey straight off the chain, and every later check — exists, authorised, not twice, conserved — reads those clear fields. Here the unit of value is a note, and no field of it is published; every check has to be rebuilt on something else, and this part builds that something.

Six fields, two jobs. The address $(d, \mathsf{pk_d})$ and the value v are the content, the same information a scriptPubKey and an amount carry. The other three are machinery: ρ and ψ serve the nullifier, rcm serves the commitment. The salt ψ and the trapdoor rcm derive from one note seed rseed; in an Ironwood note rcm is derived last, from every other field (Part F).

Type ρ carefully. It is a per-note uniqueness seed fixed at creation; where it comes from is the nullifier board later in this part. Say only that here.

The contrast to emphasise: Bitcoin publishes every field of an output in clear; every field of a note stays with its owner.

Do not say

"ρ is the note's position in the tree" — it is a seed fixed at creation, and the leaf index never enters the nullifier (later in this part).

Source: *Ironwood Guide*, "Notes"; "The note seed: Ironwood derivations (lead byte 0x03)"

Board 35 — A note: content and machinery

exists

Write

Two columns under the note tuple:

1. content: $(d, \mathsf{pk_d})$, v
2. machinery: ρ, ψ → the nullifier; rcm → hiding
3. Alice: $v = 2$ ZEC $= 2 \cdot 10^8$ zatoshi, Ironwood pool, lead byte 0x03

Say

Content and machinery. The address and the value are the content. The seeds ρ and ψ exist only to make the note's eventual nullifier unique and unpredictable; the trapdoor rcm makes the commitment hiding.

Alice's note, the running example. Two ZEC, $2 \cdot 10^8$ zatoshi, at one of her own addresses, in the Ironwood pool — its plaintext carries lead byte 0x03 (Part F).

Bitcoin publishes every field of an output; the note itself never touches the chain. What does touch the chain comes on the next boards.

Source: *Ironwood Guide*, "Notes"; "The note seed: Ironwood derivations (lead byte 0x03)"

Board 36 — Vocabulary: one protocol, two pools

exists

Write

1. Orchard *protocol* = the cryptography: curves, Sinsemilla, Action circuit, notes, commitments, nullifiers, keys, note encryption
2. *pool* = one note commitment tree + one nullifier set + one public balance row
3. Ironwood pool: current; all new value enters here; lead byte 0x03; tree began empty at activation
4. Orchard pool: legacy; sealed spend-only by three rules
5. one asset (ZEC), one chain, one transaction format
6. Bitcoin: P2WPKH beside P2PKH, plus a public total per type

Say

Fix the words before the mechanism. The Orchard protocol is the shared cryptography: the curves, Sinsemilla, the Action circuit, notes, commitments, nullifiers, keys, note encryption. A pool

is state: one note commitment tree, one nullifier set, one public balance row (the chain value pool balance).

Two pools run that protocol. The Ironwood pool is current: all new value enters here, its note plaintexts carry lead byte `0x03` (Part F), and its tree began empty at activation. The Orchard pool is the legacy carrier, sealed to spend-only by three rules (next board).

One asset, one chain, one transaction. ZEC is the only asset; one transaction format carries both pools' components (Part H).

Bitcoin analogue: an output type on the same chain — P2WPKH beside P2PKH — if Bitcoin also tracked the total value held in each type. The running example lives in the Ironwood tree; use the two pool names consistently from here on.

Do not say

"the pool is auditable" — what is public is the pool's balance row, not what happens inside the pool.

Source: *Ironwood Guide*, "The Orchard protocol and its two pools"; "Chain state and pool rules"

Board 37 — The seal: three rules

conserved

Write

Numbered, with the rule's name first:

1. **No issuance**: coinbase carries an *empty* Orchard component
2. **No circulation**: every Orchard-pool Action has $\mathsf{enableCrossAddress} = 0$ (in the verifying key, selected by branch)
3. **No entry**: per transaction, $v_{\text{balance}}^{\mathsf{Orchard}} \geq 0$

Say

Three consensus rules seal the Orchard pool, binding every transaction regardless of its version. No issuance: a coinbase transaction must carry an empty Orchard component. No circulation: every Orchard-pool Action has $\mathsf{enableCrossAddress} = 0$, so a spend may pay only its own receiver. No entry: per transaction the pool's value balance is at least zero — net value may leave the pool, never enter.

Emphasise where rule two lives. It is in the Action-circuit verifying key, which consensus selects by branch, so no transaction version bypasses it.

Why seal it. First, supply integrity after the Orchard soundness bug (ZIP-257); second, legacy notes are unrecoverable under a quantum break (Part I). Consensus stops their circulation and drains the pool through a public turnstile (Part H).

Bitcoin contrast: a soft fork that narrows what an output type may do — still spendable, but only back to its own key or out of the type; no longer created by coinbase, no longer funded from outside (Part E).

Do not say

"the turnstile makes the pool auditable" — the turnstile is a public exit; it does not audit what the pool holds.

Source: *Ironwood Guide*, "Chain state and pool rules"

Board 38 — Sinsemilla: a hash built from curve additions

exists

Write

$$\mathsf{Acc}_0 := Q(D), \qquad \mathsf{Acc}_{i+1} := (\mathsf{Acc}_i \mathbin{\dot{+}} P[m_{i+1}]) \mathbin{\dot{+}} \mathsf{Acc}_i, \qquad \mathsf{Sinsemilla}_D(M) := \mathsf{Acc}_\ell,$$
$$\mathsf{SinsemillaCommit}_r(D, M) := \mathsf{Sinsemilla}_{D\|\text{-M}}(M) + [r]\, H_D$$

Beneath: "M in 10-bit chunks $m_1, \ldots, m_\ell$; $\dot{+}$ is incomplete addition; short forms return Extract".

Say

The chain's record of a note is a commitment built on this hash, so start with the hash. Sinsemilla is built from curve additions, not from bit operations.

The mechanism. Split the message M into 10-bit chunks. Start the accumulator at $Q(D)$, a generator fixed by the domain D; for each chunk, add that chunk's generator $P[m_{i+1}]$ to the accumulator, then add the accumulator again. The hash is the final accumulator. The commitment form adds $[r]\,H_D$ under a domain suffixed "-M". The addition $\dot{+}$ is incomplete — the plain formula, valid when its inputs are neither equal nor opposite — and the short forms return Extract of the point.

Nothing up any sleeve. The generators $Q(D)$, $P[0], \ldots, P[2^{10}-1]$ and H_D are GroupHash outputs: no known discrete-log relation among them.

Emphasise the assumption. Collision resistance reduces to discrete log on Pallas — nothing beyond what the curve already needs.

Source: *Ironwood Guide*, "Sinsemilla: hash, commitment, and short forms"

Board 39 — Why Sinsemilla: in-circuit cost

exists

Write

1. per 10-bit chunk: one table lookup + two additions
2. bit-oriented hash: field arithmetic + range checks; dominates the circuit
3. Bitcoin: nobody recomputes SHA-256 inside a proof

Say

Bitcoin can afford SHA-256 because nobody recomputes it inside a proof. Here the spender must recompute the commitment inside one, so the hash's in-circuit cost is the whole question.

The cost. One table lookup and two additions per 10-bit chunk, native to a circuit with lookups. A bit-oriented hash must be re-expressed through field arithmetic and range checks, and dominates a circuit.

Source: *Ironwood Guide*, "Sinsemilla: hash, commitment, and short forms", Remark "Cost inside a proof"; *Crypto Guide*, "Arithmetisation-friendly hashing: Poseidon"

Board 40 — The note commitment: what the chain stores

exists

Draw

Two rows under the formulas:

1. on chain — cmx, one field element (plus a ciphertext, Part F)
2. private — d, $\mathsf{pk_d}$, v, ρ, ψ, rcm: recipient, amount, seeds, trapdoor

Write

$$\mathsf{cm} := \mathsf{NoteCommit}_{\mathsf{rcm}}(\mathsf{g_d}^{\star} \,\|\, \mathsf{pk_d}^{\star} \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{255}(\rho) \,\|\, \mathrm{LE}_{255}(\psi)) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),$$
$$\mathsf{cmx} := \mathsf{Extract}(\mathsf{cm}) \in \mathbb{F}_{p_{\mathsf{Pallas}}}$$

Say

What the chain stores about a note is one field element. The commitment is the Sinsemilla commitment under its own domain, in the Pallas group; the star is the 32-byte point encoding. The tree consumes a field element, hence Extract: the chain sees cmx.

Emphasise: recipient, amount, seeds and trapdoor all stay private; the chain holds one field element per note (plus a ciphertext, Part F).

Source: *Ironwood Guide*, "Note commitments"; "Sinsemilla: hash, commitment, and short forms"

Board 41 — Hiding, binding, and the CT contrast

exists

Write

Beneath the commitment formula of the previous board:

1. hiding: $[\mathsf{rcm}]H_D$ masks the hash — perfectly for a uniform rcm, computationally (PRF) for the seed-derived one
2. binding: one cm opened as two notes ⇒ a discrete-log relation
3. CT: commit to the amount + a range proof beside it; here: recipient + amount, opened only inside the proof (Part E)

Say

Hiding and binding. The term $[\mathsf{rcm}]H_D$ masks the hash — perfectly for a uniform rcm, computationally, as a PRF output, for the seed-derived one. Binding: opening one cm as two notes yields a discrete-log relation.

Contrast with Confidential Transactions. CT commits to the amount and publishes a range proof beside it; this commitment covers recipient and amount, and is opened only inside the zero-knowledge proof (Part E).

Do not say

"the proof hides the recipient and the amount" — the commitment's hiding does; the proof adds nothing beyond its statement (Part E).

Source: *Ironwood Guide*, "Note commitments"; "Sinsemilla: hash, commitment, and short forms"; *Crypto Guide*, "Hiding and binding"

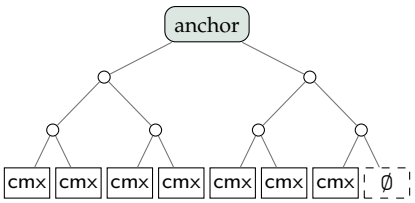
Board 42 — Two structures that only grow

exists, not twice

Draw

First the table (set, operations), then the tree.

1. Bitcoin — the UTXO set — insert on output, **delete** on spend.
2. each pool — the note commitment tree — append every cmx; a root per block.
3. (same row group) — the nullifier set — insert every nf; never delete.
4. Eight small boxes in a row, each labelled cmx; draw the eighth dashed and label it ∅.
5. A circle above each adjacent pair of boxes, joined to both.
6. A circle above each adjacent pair of circles, joined to both.
7. A rounded box at the top labelled *anchor*, joined to the two circles below it.
8. Caption: "public cmx leaves; one tree per pool".



Write

Nothing beyond the table and the figure.

Say

Bitcoin keeps one structure that shrinks; each pool keeps two that only grow. The UTXO set inserts on output and deletes on spend. The note commitment tree appends every cmx and yields a root per block; the nullifier set inserts every nf and never deletes.

Split the jobs. The tree expresses existence of value, as the UTXO set does; stopping a double spend is not its job. The nullifier set is the "deleted" half, and nothing maps a nullifier back to a leaf.

Emphasise: Bitcoin's set forgets a spent coin; these two forget nothing. The leaves are public cmx values, and there is one tree per pool.

Source: *Ironwood Guide*, "The Merkle hash and the tree"; "Nullifier sets"; protocol specification, "Note Commitment Trees"; "Nullifier Sets"

Board 43 — The tree by the numbers

exists

Write

1. depth 32 ⇒ 2^{32} leaves, creation order, one tree per pool
2. node = Sinsemilla short hash of (10-bit layer tag ‖ 255-bit child ‖ 255-bit child) = 520 bits = 52 chunks
3. empty position = 2, the x-coordinate of no Pallas point
4. Bitcoin txid tree: rebuilt per block, SHA-256d, depth follows the transaction count

Say

Depth 32**.** Room for 2^{32} leaves, filled in creation order, one tree per pool for the pool's whole life.

Nodes are layer-tagged Sinsemilla short hashes: a 10-bit layer tag and two 255-bit children, 520 bits, 52 chunks.

Empty positions hold the sentinel 2, the x-coordinate of no Pallas point, so no leaf can collide with "empty".

Bitcoin contrast: the txid tree is rebuilt per block with SHA-256d and its depth follows the transaction count; this tree is one, fixed in depth, and only ever extended.

Source: *Ironwood Guide*, "The Merkle hash and the tree"; protocol specification, "Note Commitment Trees"

Board 44 — Minting: what an output writes to the chain

exists, conserved

Write

1. output side writes: $\mathsf{cmx}_{\text{new}}$, epk, C_{enc} (to the recipient), C_{out} (to the sender's outgoing key)
2. value commitment:
$$\mathsf{cv} := [\, v_{\text{old}} - v_{\text{new}} \,]\, V + [\mathsf{rcv}]\, R$$
3. Action 1 mints Bob's note: $v = 1.5$ ZEC $= 1.5 \cdot 10^8$ zatoshi, ρ_{new} := the nullifier of Alice's note
4. on chain: cmx, cv, epk, C_{enc}, C_{out} — opaque strings

Say

In Bitcoin an output writes amount and scriptPubKey in clear, and the UTXO set indexes them. Here an Action's output side appends $\mathsf{cmx}_{\text{new}}$ to the tree and attaches an ephemeral key epk and two ciphertexts: C_{enc} to the recipient, C_{out} to the sender's own outgoing key (Part F). Nothing readable about the new note is written.

The value is hidden twice. Once inside cm, bound to the recipient; and the Action's net value $v_{\text{old}} - v_{\text{new}}$ a second time inside its value commitment cv, whose homomorphism lets a validator check conservation without any amount (Part E′).

The running example. Alice's Action 1 — the deck's label; wire order is shuffled (Part E) — mints Bob's note: 1.5 ZEC, $1.5 \cdot 10^8$ zatoshi, with ρ_{new} set to the nullifier of Alice's note. On chain: cmx, cv, and the delivery fields epk, C_{enc}, C_{out} — opaque strings.

Emphasise: the delivery fields are opaque to the validator; only their owners open them (Part F).

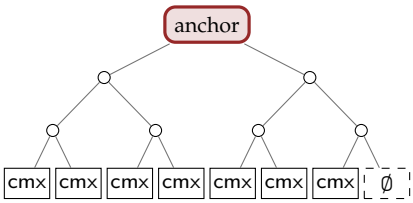
Source: *Ironwood Guide*, "Actions, bundles, and dummy notes"; "Note encryption"; "Value commitments"

Board 45 — Resting: the frontier and the anchor

exists

Draw

1. Redraw the tree of Board 42 (strokes 4–7).
2. Box the *anchor* node in red, thick.
3. Caption: "a Bitcoin input names one outpoint; this spend names a tree".



Write

1. append on the right ⇒ no old root changes; an anchor names one fixed tree, for ever
2. block's anchor = its tree's final root; a transaction's anchor = an *earlier* block's
3. frontier = newest leaf + one sibling root per filled subtree along its path
4. any main-chain block since the pool's activation: no window

Say

A Bitcoin input names one outpoint; this spend names a tree. Appending on the right alters no old root, so an anchor names one fixed historical tree, for ever.

Whose root. A block's anchor is its tree's final root, one per main-chain block; a transaction's must name an *earlier* block's: no spend of an unmined note, no mempool chains.

The frontier. A wallet seeds its tree from the frontier: the newest leaf plus one sibling root per filled subtree along its path — enough to append and recompute the root. Its own notes' paths it keeps as retained nodes and subtree roots (Part F).

Emphasise: no window. Consensus accepts the anchor of any main-chain block since the pool's activation; proving is decoupled from broadcast.

Source: *Ironwood Guide*, "The Merkle hash and the tree"; "Anchors"; "Authentication paths"; *Wallet Guide*, "Frontiers as scan seeds"; protocol specification, "Action Transfers and their Descriptions"

Board 46 — Which anchor a wallet picks

exists

Write

1. consensus: no depth mandated; wallet policy
2. deployed default: newest checkpoint at or below $H - 3$, H the target height; per pool, oldest taken
3. ZIP-315: input availability; fixed depth hides trusted status (untrusted 10 confirmations, trusted 3)
4. the Guide: anchor's tree = candidate set; newest admissible = largest; a deep anchor fingerprints
5. old anchor ≠ soundness hole; second spend is the nullifier's job

Say

Consensus mandates no depth; the choice is wallet policy. The deployed default is the newest *checkpoint* at or below $H-3$, with H the target height. A checkpoint is a tree state the wallet stored at a block boundary, one series per pool; the pick is made per pool and the oldest taken, so checkpoint gaps and that cross-pool minimum can make the anchor older.

Two rationales, attributed separately. ZIP-315's stated one: input availability, and a fixed depth hides whether the spent input was trusted — untrusted notes wait 10 confirmations, trusted ones 3. The Guide's own argument: the anchor's tree is the candidate set, so the newest admissible anchor names the largest one, and an idiosyncratically deep anchor would fingerprint the spender.

An old anchor is no soundness hole. Membership in an older, smaller tree proves the note was committed even earlier. Stopping a second spend is the nullifier's job, not the anchor's.

Bitcoin has no such choice: an input names its outpoint, whatever the tip.

Do not say

Attributing the anonymity-set argument to ZIP-315 — it is the Guide's; ZIP-315 states input availability and hiding the input's trusted status.

Source: *Ironwood Guide*, "Anchors"; *Wallet Guide*, "The confirmation policy"

Board 47 — When the anchor vanishes: reorganisations

exists

Write

1. reorg strikes the anchoring block ⇒ anchor invalid; deeper is rarer, never impossible
2. automatic rollback depth = node policy, not consensus; ZIP-218 recommends 600 blocks for NU7
3. deployed wallets: truncate to a checkpoint below the fork → rescan → rebuild; same nullifiers
4. specified, wallet support pending: re-anchor with txid unchanged (ZIP-374 v2; the proof pins the anchor)

Say

A reorganisation that strikes the anchoring block from the main chain invalidates the anchor. Deeper anchors make this rarer; no depth rules it out.

Automatic rollback depth is node policy, not consensus. ZIP-218 recommends 600 blocks for NU7. Recovery beyond the node's retained rollback window requires a different recovery procedure.

What deployed wallets do. Truncate to a checkpoint below the fork, rescan, and rebuild the transaction. The rebuilt spend reveals the same nullifiers, because each is a fixed function of its note.

What is specified, with wallet support pending. Re-anchoring a signed transaction with its txid unchanged: ZIP-374 v2 makes v6 anchors deferrable, and the proof rather than the signature pins them.

Bitcoin contrast: a reorged transaction stays valid wherever its inputs survive; here the tree it named may no longer exist.

Source: *Ironwood Guide*, "Anchors"; *Wallet Guide*, "Reorganisation detection and recovery"; "PCZT v2: deferred anchors and the Ironwood bundle"

Board 48 — Not a mixer

exists

Write

1. candidate set = every leaf of the Ironwood tree under the anchor
2. no participants, no coordinator, no round
3. CoinJoin: set = one transaction's co-signers; the graph remains
4. narrows the set: metadata — dummy or spent leaves, timing, fees, transparent components — not the proof

Say

The candidate set of Alice's spend is every leaf of the Ironwood tree under her anchor: everyone's notes since the pool's activation, when the tree began empty.

No participants, no coordinator, no round, nobody to wait for: the other leaves' owners never take part.

CoinJoin contrast. There the anonymity set is one transaction's co-signers, and the graph remains — every output is a child of the whole input set.

What narrows the set here is metadata, not the proof: publicly known dummy or spent leaves, timing, fees, transparent components (Part H). Tree size alone does not measure it.

Source: *Ironwood Guide*, "The note commitment tree"; "Anchors"; "Chain state and pool rules"

Board 49 — The nullifier: a keyed serial number

not twice

Write

$$\mathsf{nf} := \mathsf{Extract}(\,[\,F_{\mathsf{nk}}(\rho) + \psi\,]\,G^{\mathsf{nf}} + \mathsf{cm}\,) \in \mathbb{F}_{p_{\mathsf{Pallas}}},$$
$$G^{\mathsf{nf}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard, K})$$

Then, beneath:

1. keyed by nk; seeded by ρ; salted by ψ; bound to cm; F_{nk} = Poseidon keyed by nk
2. $\rho_{\text{new}} := \mathsf{nf}_{\text{old}}$ (the note spent in the same Action)
3. dummy spend: $\rho = \mathsf{Extract}(\text{random Pallas point})$; still publishes nf

Say

Bitcoin's spent marker is the outpoint, the coin's own name; this one names nothing. Write the formula, then read it aloud: keyed by nk, the owner's nullifier key; seeded by ρ; salted by ψ; bound to cm. The PRF F_{nk} is Poseidon keyed by nk, an algebraic hash the circuit evaluates natively.

Where ρ comes from. Set $\rho_{\text{new}} := \mathsf{nf}_{\text{old}}$, the nullifier of the note spent in the same Action — every note is born from a spend and inherits its nullifier. This answers the question left open on Board 34.

The dummy case. A dummy spend — an Action with no real input — seeds ρ from Extract of a random Pallas point, and still publishes a nullifier.

Emphasise: the nullifier is a keyed serial number, not a pointer; it names no leaf.

Do not say

"ρ is the leaf position" — it is the parent note's nullifier (or a random point's Extract for a dummy). Naming a minimum-actions constant as the reason for the dummy — the Action count is set in Part E.

Source: *Ironwood Guide*, "The nullifier"; *Crypto Guide*, "Arithmetisation-friendly hashing: Poseidon"; protocol specification, "Dummy Notes (Orchard)"

Board 50 — Uniqueness and unlinkability, separately

not twice

Write

1. duplicate nf rejected per pool ⇒ accepted ρ unique within the pool
2. leaf index never enters nf (private witness of membership only)
3. uniqueness: ingredients fixed at creation; two notes, one nf ⇒ discrete-log relation on Pallas
4. unlinkability: without nk the masking term looks random — Poseidon as a PRF

Say

The chain's rule on ρ closes the loop. Duplicate nullifiers are rejected per pool, so accepted notes' ρ values are unique within the pool — forged notes cannot be made to share a nullifier.

The leaf index never enters. Position is a private witness of the membership check, not an input to nf.

Recall Part B, now with every input in view. Uniqueness, because each ingredient is fixed once the note exists and two notes sharing one nf would yield a discrete-log relation on Pallas. Unlinkability, because without nk the masking term looks random — under Poseidon as a PRF. Decisional Diffie–Hellman on Pallas is only the specification's backup aim, under an extra entropy assumption.

Bitcoin's marker is unique because it is the coin's name; this one is unique because two names would break discrete log.

Do not say

"the leaf position makes the nullifier unique" — position never enters; uniqueness rests on ingredients fixed at creation and on discrete log on Pallas.

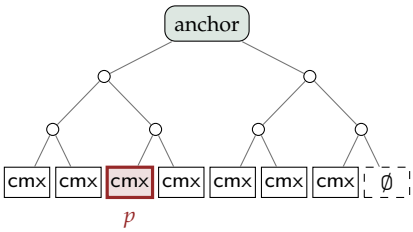
Source: *Ironwood Guide*, "Nullifier uniqueness and unlinkability"

Board 51 — Alice's leaf: the position is a secret

exists, not twice

Draw

1. Redraw the tree of Board 42 (strokes 4–7).
2. Box the third leaf from the left in red, thick; write p in red beneath it.
3. Caption: "position p and the path stay inside the proof".



Write

1. private witness: p and its 32 siblings; public: the anchor
2. nf fixed at mint: nk, ρ, ψ, cm all set; only a holder of nk computes it
3. node at spend: nf $\notin$ set $\Rightarrow$ record nf
4. SPV's Merkle path to a public root, run inside the zero-knowledge proof

Say

Alice's 2 ZEC note sits at position p, recorded by her wallet when trial decryption succeeded; p and its 32 siblings are private witness, only the anchor public.

Her nullifier was fixed at mint — nk, ρ, ψ, cm are all set — and only a holder of nk can compute it.

At spend she reveals nf**;** the node checks it is not in the set, then records it. Nothing links nf to the leaf at p.

The SPV proof you know — a Merkle path to a public root — runs here inside the zero-knowledge proof. That is the one thing to leave on the board.

Ask

Alice's restored wallet has found its old notes. How does it learn which are spent?

Answer. The chain never marks a leaf spent, so the tree cannot say. With nk the wallet computes each note's nullifier and discards those already in the pool's nullifier set — compact blocks carry every nf for this, and to supply ρ when detecting the note minted beside it (Part F). With the incoming keys alone it sees notes arrive but cannot tell which have left: spent status is a capability of nk (Part D). Bitcoin: a restored wallet asks the UTXO set.

Do not say

"the proof hides the sender and the receiver" — the proof runs the membership check; hiding comes from the commitment, the keyed nullifier PRF and key re-randomisation (Parts B, E).

Source: *Ironwood Guide*, "Authentication paths"; "The nullifier"; "Verification of a transaction"; "Trial decryption and note acceptance"

4 Part D — Keys and addresses

Board 52 — BIP-32 becomes ZIP-32: one seed, two trees

authorised

Write

1. one seed: 32–252 bytes, at least 256 bits of entropy
2. $(\mathsf{sk}_i, c_i) :=$ halves of $\mathsf{PRFexpand}_{c_{\mathrm{par}}}(\mathtt{0x81} \,\|\, \mathsf{sk}_{\mathrm{par}} \,\|\, i)$, $\quad i \geq 2^{31}$
3. transparent: $m/44'/133'/\mathit{account}'/\mathit{change}/\mathit{index}$ — two lowest levels non-hardened, xpub at the account
4. Orchard: $m_{\mathsf{Orchard}}/32'/133'/\mathit{account}'$ — hardened only, chain code discarded at the account, no change level

Say

One seed roots both trees, as one BIP-32 seed roots a Bitcoin wallet. A seed of 32 to 252 bytes with at least 256 bits of entropy roots the transparent tree and the shielded tree, so one backup recovers every key.

Transparent keys keep BIP-44 as the room knows it. The path is

$$m/44'/133'/account'/change/index,$$

the two lowest levels non-hardened, an xpub at the account.

Orchard keys stop at the account, and every level is hardened. Each level applies the formula on the board; the chain code is discarded at the account. The xpub mode — a public key deriving child public keys — is undefined, so no Orchard extended full viewing key exists.

Emphasise: there is no change level. BIP-44's change branch separates addresses on a public graph. A shielded change output is indistinguishable on chain from any other output, so a change subtree would separate nothing. Where change goes instead is the next two boards.

Source: *Wallet Guide*, "Wallet seeds"; "Hardened-only derivation"; "The standard account path"; ZIP-32, "Key path levels"

Board 53 — Two scopes per account: external and internal

authorised

Write

1. $\mathsf{r_{ivk}}^{\mathsf{int}} := \mathsf{ToScalar}(\mathsf{PRFexpand}_{\mathsf{r_{ivk}}}(\mathsf{0x83} \,\|\, \mathsf{ak} \,\|\, \mathsf{nk}))$
2. $\mathsf{fvk}^{\mathsf{int}} := (\mathsf{ak},\ \mathsf{nk},\ \mathsf{r_{ivk}}^{\mathsf{int}})$
3. same ask spends, same nk nullifies; ivk, dk, ovk re-derived
4. $\mathsf{fvk} \to \mathsf{fvk}^{\mathsf{int}}$; $(\mathsf{dk}, \mathsf{ivk}) \to$ external scope only

Say

Only $\mathsf{r_{ivk}}$ changes. The map ToScalar reduces the 512-bit PRF output modulo the Pallas scalar-field order. The same ask spends and the same nk nullifies in both scopes; ivk, dk and ovk are re-derived, so an internal address does not decrypt under the external ivk.

The internal key follows from the full viewing key. A full viewing key sees both scopes. An external $(\mathsf{dk}, \mathsf{ivk})$ — a payment processor — sees the external scope only, never the change: neither the internal scope nor any ovk follows from it.

The Bitcoin contrast. BIP-44 separates change by a path level on a public graph; here the separation is a second full viewing key, and the change output looks like any other output on chain.

Do not say

- "The wallet recovers its change through ovk." Change is a note to an internal address; the full viewing key sees it because $\mathsf{fvk}^{\mathsf{int}}$ follows from fvk, and the internal ivk decrypts it.
- "The pair $(\mathsf{dk}, \mathsf{ovk})$ comes from ivk." No ovk follows from $(\mathsf{dk}, \mathsf{ivk})$; the pair comes from fvk (the ladder board).

Source: *Wallet Guide*, "Internal keys: change and auto-shielding"; ZIP-32, "Orchard internal key derivation"; ZIP-316, "Deriving Internal Keys"

Board 54 — The internal scope: change and auto-shielding

authorised

Write

1. internal addresses ← change, auto-shielded funds
2. shielding = moving transparent funds into the pool (automatic, to an internal address)
3. Alice's change: 0.4999 ZEC → internal address, index 0

Say

What BIP-44 puts in the path sits here in the key. Internal addresses take change and auto-shielded funds.

Define shielding before using the word. Shielding is moving transparent funds into the pool; wallets do it automatically, to an internal address.

The running example. The deployed wallet library (librustzcash) sends change to the internal address at index 0: Alice's 0.4999 ZEC goes there.

The Bitcoin contrast. A Bitcoin change output sits on the change branch of a public graph; here the change output is indistinguishable from any other, and only the internal scope's keys find it.

Do not say

- "Change is recovered via ovk." Change is an incoming note to the internal address, seen through the internal scope, not through the outgoing key.

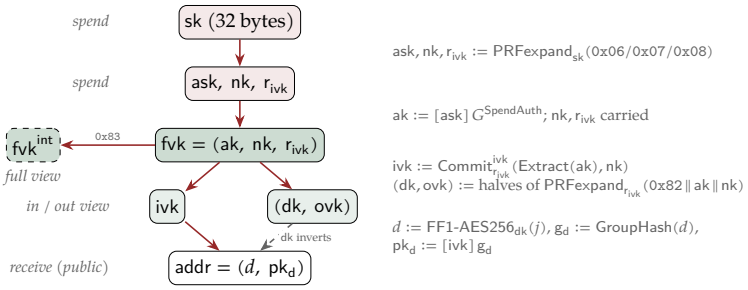
Source: *Wallet Guide*, "Internal keys: change and auto-shielding"; *Ironwood Guide*, "Chain state and pool rules"

Board 55 — The ladder: one secret, graded capabilities

authorised / not twice

Draw

1. A column of three boxes down the middle, top to bottom: "sk (32 bytes)"; "ask, nk, r_{ivk}"; "fvk = (ak, nk, r_{ivk})".
2. Under fvk, two boxes side by side: ivk on the left, (dk, ovk) on the right; under those, centred, a white box "addr = (d, pk_d)".
3. A dashed box "fvk^{int}" to the left of fvk.
4. Red arrows, one-way: sk → middle box → fvk; fvk → ivk; fvk → (dk, ovk); ivk → addr; and fvk → fvk^{int} leftwards, labelled 0x83.
5. One grey dashed arrow (dk, ovk) → addr, labelled "dk inverts".
6. Tier labels in the left margin: *spend* beside the top two boxes; *full view* under fvk^{int}; *in / out view* beside the ivk row; *receive (public)* beside the address.
7. The operation beside each rung, from Write below.



Write

1. $ask, nk, r_{ivk} := PRFexpand_{sk}(0x06/0x07/0x08)$
2. $ak := [ask]\, G^{SpendAuth};\ nk, r_{ivk}$ carried
3. $ivk := Commit^{ivk}_{r_{ivk}}(Extract(ak), nk)$
4. $(dk, ovk) :=$ halves of $PRFexpand_{r_{ivk}}(0x82 \,\|\, ak \,\|\, nk)$
5. $d := \text{FF1-AES256}_{dk}(j),\ g_d := GroupHash(d),\ pk_d := [ivk]\, g_d$

Say

Red arrows are one-way. A rung is handed down without the powers above it.

In Bitcoin, BIP-32 has two rungs, xprv and xpub, and the public ledger supplies the rest. Here every rung between is a separate secret — full view, incoming view, outgoing view — and only the address at the bottom is public.

Emphasise: (dk, ovk) come from fvk by a PRF, not from ivk. Point at the two downward arrows leaving fvk: the ivk branch and the (dk, ovk) branch are siblings.

The circuit re-walks fvk → ivk → pk_d (A7) with g_d as a witness. The step $j \to d \to g_d$ that produces the witness is never checked in the circuit; that is the dashed grey edge, and the diversified-addresses board says why.

Do not say

- "The pair (dk, ovk) is derived from ivk." Both branches leave fvk; the incoming viewing key yields neither.

Source: *Ironwood Guide*, "Key capabilities and scope"; "Viewing keys"; "The Action statement"; protocol specification, "Orchard Key Components"

Board 56 — What holds the rungs apart

authorised / not twice

Draw

Three columns — *step* | *operation* | *why it is one-way* — one row at a time:

1. $\mathsf{sk} \to (\mathsf{ask}, \mathsf{nk}, \mathsf{r_{ivk}})$ | $\mathsf{PRFexpand_{sk}}(\mathtt{0x06/0x07/0x08})$ | PRF (BLAKE2b)
2. $\mathsf{ask} \to \mathsf{ak}$ | $[\mathsf{ask}]\,G^{\mathsf{SpendAuth}}$ | discrete log on Pallas
3. $\mathsf{fvk} \to \mathsf{ivk}$ | $\mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$ | hiding under $\mathsf{r_{ivk}}$
4. $\mathsf{fvk} \to (\mathsf{dk}, \mathsf{ovk})$ | halves of $\mathsf{PRFexpand_{r_{ivk}}}(\mathtt{0x82} \,\|\, \mathsf{ak} \,\|\, \mathsf{nk})$ | PRF
5. $\mathsf{fvk} \to \mathsf{fvk}^{\mathsf{int}}$ | $\mathsf{PRFexpand_{r_{ivk}}}(\mathtt{0x83} \,\|\, \mathsf{ak} \,\|\, \mathsf{nk})$ | PRF
6. $\mathsf{ivk} \to \mathsf{pk_d}$ | $[\mathsf{ivk}]\,\mathsf{g_d}$ | discrete log on Pallas
7. $j \to d$ | $\mathsf{FF1\text{-}AES256_{dk}}(j)$ | *not* one-way: dk inverts it

Write

1. the scalar ivk: a commitment, not a hash — binds *and* hides (ak, nk)
2. one $\mathsf{r_{ivk}}$ keys both $\mathsf{Commit}^{\mathsf{ivk}}$ and the 0x82 PRF ⇒ joint assumption once dk or ovk is exposed
3. a separate nk: spent status without spend authority

Say

Three kinds of wall. A PRF (BLAKE2b) at the top and at the 0x82 and 0x83 splits; a discrete logarithm on Pallas at ask → ak and at ivk → $\mathsf{pk_d}$; a hiding commitment at fvk → ivk.

The scalar ivk is a commitment, not a hash, because it must bind and hide (ak, nk). An ivk holder learns neither. Since $\mathsf{r_{ivk}}$ also keys the 0x82 PRF, hiding rests on a joint assumption once dk or ovk is exposed.

Why a separate nk. Spent status can be delegated without spend authority: the viewing key holds nk, never ask.

The last row is deliberately not one-way. The diversifier key inverts FF1, so the dk holder maps any of its addresses back to an index. Bitcoin has one wall, xprv to xpub; the rest of its ladder is public.

Do not say

- "The pair (dk, ovk) comes from ivk." Row 4 starts at fvk.

Source: *Ironwood Guide*, "Key capabilities and scope"; *Wallet Guide*, "Diversifier indices"; protocol specification, "Orchard Key Components"

Board 57 — Graded capabilities: spend, full view, spent status

authorised / not twice

Draw

Four columns — *capability* | *holds* | *can* | *cannot* — one row at a time:

1. spend | ask (from sk) | authorise spends, both scopes | —
2. full view | $\mathsf{fvk} = (\mathsf{ak}, \mathsf{nk}, \mathsf{r_{ivk}})$ | everything below, both scopes; every address and its index | spend: ask is a discrete log away
3. spent status | nk (inside fvk) | the nullifier of a note it holds | spend; find notes (needs ivk)

Write

1. Bitcoin: only the spend row is secret (xprv)

Say

Each row holds strictly less than the one above. Spend authority is ask, from sk, and it covers both scopes. Full view is fvk: everything below it, both scopes, every address and its index — but not spending, because ask is a discrete logarithm away from ak. Spent status is nk alone: it computes the nullifier of a note it is handed, and can neither spend nor find notes, which needs ivk.

Bitcoin keeps only the spend row secret (xprv). Full view and spent status are what its public ledger shows to everyone.

Emphasise the middle row. A full viewing key sees everything and moves nothing.

Source: *Ironwood Guide*, "Key capabilities and scope"; *Wallet Guide*, "Orchard key components of an account"

Board 58 — Graded capabilities: incoming, outgoing, receive

exists

Draw

Same four columns, three more rows:

1. incoming view | $(\mathsf{dk}, \mathsf{ivk})$ | detect and decrypt incoming; derive all 2^{88} addresses and invert them to indices | nullifiers, so spent status; outgoing; the internal scope
2. outgoing view | ovk | open C_{out} of what this account and scope sent (a sender may set $\mathsf{ovk} = \bot$: then no ovk opens it) | incoming; the internal ovk
3. receive | $(d, \mathsf{pk_d})$ | be paid | anything else — the only public item

Write

1. every row but the last is a distinct secret

Say

Every row but the last is a distinct secret. Bitcoin's ledger collapses everything between xprv and the address into what anyone can read.

The incoming view sees arrivals, not departures. The pair $(\mathsf{dk}, \mathsf{ivk})$ detects and decrypts incoming notes and enumerates all 2^{88} addresses, inverting each to its index; it has no nullifiers, so no spent status, no outgoing view, and no internal scope.

The outgoing view may open nothing. The key ovk opens C_{out} of what this account and scope sent, but a sender may set $\mathsf{ovk} = \bot$, and then no ovk opens it.

Emphasise: the address is the only public item. It lets one be paid and does nothing else.

Do not say

- "The pair $(\mathsf{dk}, \mathsf{ovk})$ comes from ivk." The incoming row holds no ovk and cannot reach one.

Source: *Ironwood Guide*, "Key capabilities and scope"; "The outgoing ciphertext"; *Wallet Guide*, "Raw encodings"; ZIP-316, "Deriving Internal Keys"

Board 59 — Viewing keys: xpub is the bridge, not the analogue

exists / not twice

Write

1. xpub = watch-only: enumerates addresses; the public ledger supplies incoming, outgoing, spent status — to anyone
2. $(\mathsf{dk}, \mathsf{ivk})$ = watch-only for incoming: enumerates *and* decrypts
3. three differences: (1) nothing visible without the key; (2) spent status = nk; (3) outgoing = ovk
4. "viewing" = a capability, not publicity

Say

Start from what the room has: an xpub is a watch-only wallet. It enumerates the addresses, and the public ledger supplies incoming, outgoing and spent status — to anyone, xpub or not.

The pair $(\mathsf{dk}, \mathsf{ivk})$ is the watch-only wallet for incoming. It enumerates the addresses *and* decrypts. Then the three differences, in order: Bitcoin shows everything regardless, here nothing is visible without the key, because the outputs are ciphertext; spent status is a second secret, nk, which computes the nullifiers that the chain's set is matched against; outgoing is a third, ovk, which opens C_{out}. The xpub gets both free only because Bitcoin's ledger is public.

Emphasise: "viewing" names a capability, not publicity. Every viewing key is secret material disclosed deliberately — to an auditor, a processor — and validation needs none of them.

Do not say

- "The pool is auditable." A viewing key discloses one account, to whoever its holder chooses; nothing about the pool is readable without one, and the validator holds none.

Source: *Ironwood Guide*, "Key capabilities and scope"; "Verification of a transaction"; *Wallet Guide*, "Unified viewing keys"

Board 60 — Diversified addresses: one key, 2^{88} addresses

exists

Write

1. $d := \mathsf{FF1\text{-}AES256}_{\mathsf{dk}}(j)$
2. $\mathsf{g_d} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-gd},\ d)$
3. $\mathsf{pk_d} := [\mathsf{ivk}]\,\mathsf{g_d}, \quad \mathsf{addr} := (d,\ \mathsf{pk_d})$
4. $2^{88} \approx 3.1 \times 10^{26}$ addresses per key; raw receiver $11 + 32 = 43$ bytes
5. random 88-bit draws collide near 2^{44}; FF1 never collides, and dk inverts it

Say

Every index works. Each j of the $2^{88} \approx 3.1 \times 10^{26}$ gives an address: GroupHash is total (a fixed fallback covers the identity), so there is no rejection loop. A raw receiver is $11 + 32 = 43$ bytes, and all of them are spent by one ask and found by one ivk, at no extra scanning cost.

A keyed permutation, not a counter and not a hash. Consecutive indices scatter over $\{0,1\}^{88}$ and never collide — random 88-bit draws would, near 2^{44} — and dk inverts the map, so the wallet stores nothing per address.

Wallet-side only, never in the circuit. That is why AES is fine here: the circuit takes $\mathsf{g_d}$ as a witness (Board 55) and never sees j.

The Bitcoin contrast. An xpub also enumerates addresses; the difference is what an address reveals, which is the next board.

Source: *Ironwood Guide*, "Diversified addresses"; *Wallet Guide*, "Diversifier indices"; "Raw encodings"; *Crypto Guide*, "Format-preserving encryption and FF1"; ZIP-32, "Orchard key path"

Board 61 — Unlinkable, even to the payer

exists

Write

1. $(d, \mathsf{pk_d})$ and $(d', \mathsf{pk_d}')$: $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d},\ \mathsf{pk_d}' = [\mathsf{ivk}]\,\mathsf{g_d}'$ — same ivk?
2. deciding it = DDH on Pallas, GroupHash a random oracle
3. only (dk, ivk) can check: re-derive at the decrypted index
4. never a chosen-diversifier oracle

Say

Two addresses, one question: same ivk? Deciding it is a DDH problem on Pallas with GroupHash as a random oracle: given two addresses of different owners and a third derived from one of them, no one can tell which. Alice, who paid one of Bob's addresses, learns nothing about the other.

Only the holder of (dk, ivk) can check, by re-deriving the address at the decrypted index under that ivk.

One rule for implementers. Never offer a chosen-diversifier oracle — an address for an attacker's d under your ivk: it breaks unlinkability for every address of that ivk.

The Bitcoin contrast, and the thing to emphasise. Under one xpub, addresses link on the public graph once spent together (Part A), and a leaked xpub links all beneath it, spent or not. Here an address reveals only itself.

Source: *Wallet Guide*, "Diversifier indices"; *Ironwood Guide*, "Diversified addresses"; protocol specification, "$\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ and $\mathsf{DiversifyHash}^{\mathsf{Orchard}}$ Hash Functions"

Board 62 — Unified addresses (ZIP-316): one string, several receivers

exists

Write

1. typecodes: 0x00 P2PKH, 0x01 P2SH, 0x02 Sapling, 0x03 Orchard — at most one each, at least one shielded
2. ascending typecode → F4Jumble (whole payload, HRP included) → Bech32m
3. sender: Orchard > Sapling > transparent; may refuse a type, never reorder
4. Orchard receiver → the Ironwood pool

Say

One string carries at most one receiver per type, with at least one shielded. Transparent P2PKH or P2SH (`0x00 / 0x01`), Sapling (`0x02`), Orchard (`0x03`). Items are encoded in ascending typecode, passed through F4Jumble — an unkeyed four-round Feistel permutation of the whole payload, HRP included — then Bech32m, so a substituted receiver changes the whole string, not one region of it.

What a sender does. Use the most preferred receiver it supports: Orchard over Sapling over transparent. A wallet may refuse a type, never reorder.

Which pool. An Orchard receiver names the protocol, not a pool, and all new value enters the Ironwood pool, so a payment to it lands in the Ironwood tree; the receiver has no string form of its own.

The same container with viewing-key items is a unified full or incoming viewing key.

The Bitcoin contrast. One Bitcoin address encodes one script type; here one address lets the sender's wallet choose the pool.

Source: *Wallet Guide*, "Unified addresses"; "String encoding: F4Jumble and Bech32m"; "Unified viewing keys"; *Ironwood Guide*, "Key capabilities and scope"; ZIP-316, "Encoding of Unified Addresses"

Board 63 — Bob's address in the example

exists

Write

1. $\mathsf{addr}_B = (d_B,\ \mathsf{pk_d})$, 43 bytes
2. $\mathsf{g_d} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-gd},\ d_B)$, $\quad \mathsf{epk} := [\mathsf{esk}]\,\mathsf{g_d}$
3. $\mathbf{n} := (\mathsf{addr}_B,\ 1.5\ \text{ZEC},\ \rho,\ \psi,\ \mathsf{rcm})$
4. on chain: inside cm, inside C_{enc}, as epk under a fresh esk; lead byte `0x03`
5. change 0.4999 ZEC → Alice's internal address, index 0

Say

Bob hands Alice a unified address; her wallet takes its Orchard receiver. That is $\mathsf{addr}_B = (d_B, \mathsf{pk_d})$, 43 bytes. From it she derives $\mathsf{g_d}$, then epk under a fresh esk, and builds the note **n** with 1.5 ZEC.

That is all it learns. No ivk, nk or ak, no index j, no link to Bob's other addresses. On chain the address appears only blinded: inside cm, inside C_{enc}, and as epk under a fresh esk.

Which tree. The note is minted in the Ironwood tree, lead byte `0x03`; Bob's one ivk trial-decrypts both pools.

Her change, 0.4999 **ZEC, goes to her internal address at index** 0: no address of hers appears either.

The Bitcoin contrast. Bitcoin displays the script beside the amount; here neither appears in the clear.

Ask

Which key does an accountant need to audit a company's incoming *and* outgoing shielded payments — and what can they not do with it?

- One unified full viewing key per account. Inside it, fvk yields ivk (incoming), nk (which of those notes are spent) and ovk (outgoing) — for the external scope and, through the `0x83` step, the internal one, so change is covered too.
- Not spend: ask is a discrete logarithm away from ak. Not wander into a sibling account: only hardened derivation exists, and no Orchard extended full viewing key.
- Not read what was sent under some other ovk, or under none: the sender may set $\mathsf{ovk} = \perp$, and then no ovk opens C_{out}. Nor anything received by an account they were not given.

Do not say

- "The proof hides the receiver." The commitment, the ciphertext and the fresh esk blind the address; the proof shows the address was used consistently (A7).
- "The pair (dk, ovk) comes from ivk." The accountant's ovk comes from fvk.

- "Change is recovered via ovk." Change is an incoming note in the internal scope; the full viewing key covers it through the 0x83 step.

Source: *Ironwood Guide*, "Encryption to the recipient"; "Notes and note commitments"; "Key capabilities and scope"; "The outgoing ciphertext"; *Wallet Guide*, "Unified addresses"; "Internal keys: change and auto-shielding"; "Orchard key components of an account"; "Unified viewing keys"

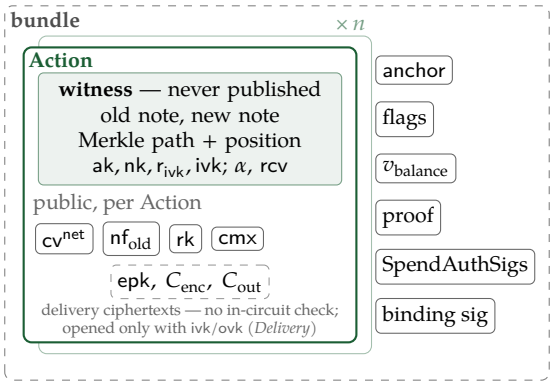
5 Part E — The Action statement

Board 64 — Anatomy of an Action

all four

Draw

1. A large dashed rectangle; write *bundle* in its top-left corner.
2. Inside, a solid box labelled *Action* top-left, with a faint second box just behind it; write × *n* at that box's corner.
3. Top of the Action box, a shaded block: *witness — never published*: old note, new note; Merkle path + position; ak, nk, r_{ivk}, ivk; α, rcv.
4. Below it, a row of four small boxes: cv^{net}, nf_{old}, rk, cmx; label the row *public, per Action*.
5. Under the row, a dashed box: epk, C_{enc}, C_{out}; caption it *delivery ciphertexts — no in-circuit check; opened only with* ivk/ovk (*Delivery*).
6. Down the right edge, inside the bundle but outside the Action, six boxes top to bottom: anchor, flags, $v_{balance}$, proof, SpendAuthSigs, binding sig.



Write

- one Action = one old note spent + one new note created
- witness (never published): old note, new note, path + position, ak, nk, r_{ivk}, ivk, α, rcv
- rim, per Action: cv^{net}, nf_{old}, rk, cmx — the four the proof binds
- dashed: epk, C_{enc}, C_{out} — ciphertexts, read by no clause

Say

Start from the shape. One Action spends one old note and creates one new one; the shape is fixed, so an Action never shows which it "really" is.

The witness stays inside. Both notes, the path, the keys, α, rcv — never published.

Four fields on the rim are what the proof binds. Per Action: cv^{net}, nf_{old}, rk, cmx.

The dashed box is delivery, not verification. The fields epk, C_{enc}, C_{out} are ciphertexts read by no clause; only ivk or ovk opens them (*Delivery and light clients*).

Bitcoin contrast. A Bitcoin input names its coin and shows its script; a Bitcoin output shows its amount and its script. Here every field is a commitment, a serial number, a disguised key, or a ciphertext.

Do not say

"The proof hides the sender and receiver." The fields hide them — each is a commitment, a serial number, a disguised key, or a ciphertext; the proof establishes the statement about them and nothing more.

Source: *Ironwood Guide*, "Actions, bundles, and dummy notes"; "The outgoing ciphertext"; "The Action statement"

Board 65 — The bundle around it

all four

Draw

Three columns: *per bundle* | *once* | *role*. Rows, in order:

1. anchor — one root, shared by every Action — the tree the spends prove membership in
2. flags — enableSpends, enableOutputs, enableCrossAddress — consensus gates (A8–A10)
3. v_{balance} — signed, cleartext — net value leaving the pool
4. proof — one aggregate Halo 2 proof — covers all n Actions at once
5. binding sig — one signature — proves the values balance
6. (a gap, then) per Action — SpendAuthSig under rk — the owner authorised this spend

Write

- one Action = 820 public bytes, 660 of them ciphertext
- Alice: 2 Actions, one 7264-byte proof
- node checks: proof, anchor, nullifier set, two signatures — never a note

Say

The bundle is written once; the Action is repeated. One anchor shared by every Action; three flags driving the consensus gates A8–A10; a signed, cleartext v_{balance}, the net value leaving the pool; one aggregate Halo 2 proof covering all n Actions at once; one binding signature proving the values balance. Per Action, one SpendAuthSig under rk: the owner authorised this spend.

Sizes. Each Action is 820 public bytes, 660 of them ciphertext; Alice's bundle carries two Actions and one 7264-byte proof.

What the node touches. The node checks the proof, the anchor, the nullifier set, and the two signatures — and never sees a note. Both signatures and v_{balance} return in *Balance and authority*.

Do not say

"A few hundred bytes." Alice's proof is 7264 bytes for two Actions, and each Action adds 820 public bytes.

Source: *Ironwood Guide*, "Verification of a transaction"; "Action descriptions and bundles"

Board 66 — Why a spend and an output are fused

all four

Write

$$n_{\text{act}}(s, o) = \max(\max(s, o),\, 2) \qquad \text{Alice: } s = 1,\ o = 2 \ \Rightarrow\ n_{\text{act}} = 2$$

- spend list and output list each padded to n_{act}; floor 2
- Alice: 1 spend, 2 outputs (Bob, change) ⇒ one dummy spend

Say

A Bitcoin transaction shows its true input and output counts. Here the count is n_{act}, and the fee is computed from that public count (*The transaction and the turnstile*).

Uniformity. Every Action looks like every other: one spend slot, one output slot, whatever the payment needed.

Padding. The spend list and the output list are each extended to n_{act}; the floor is two. Alice's one spend and two outputs (Bob, change) pad with one dummy spend.

Scope. This count, and the shuffle on the next board, are the Ironwood component's (cross-address enabled). A sealed Orchard-pool bundle pads to $s + o$ and pairs each spend with an output to the spent note's own receiver.

Do not say

"The dummy is there because of the minimum of two Actions." Alice's $n_{\text{act}} = 2$ comes from her two outputs; the dummy pads the spend list up to that count. The floor of two is a separate rule that gives the same number here.

Source: *Wallet Guide*, "Padding, dummies, and shuffling"; *Ironwood Guide*, "Actions, bundles, and dummy notes"

Board 67 — Dummies and the shuffle

all four

Write

- dummy spend: value 0, fresh random spending key, $\rho = \mathsf{Extract}(\text{random Pallas point})$; still publishes a nullifier
- dummy output: value 0, fresh random address, $\rho := \mathsf{nf}_{\text{old}}$ like any output's
- shuffle spends and outputs *independently*, then zip
- Alice: spend side {2 ZEC note, dummy}; output side {Bob, change}

Say

A slot with nothing real in it holds a dummy. A dummy spend is a zero-value note under a fresh random spending key, its ρ the Extract of a random Pallas point; it still publishes a nullifier. A dummy output is a zero-value note to a fresh random address, its $\rho := \mathsf{nf}_{\text{old}}$ like any output's.

Shuffle, then zip. Spends and outputs are shuffled independently; which spend shares an Action with which output is random.

Alice's bundle. Her 2 ZEC note and a dummy on the spend side, Bob's note and her change on the output side; which pair forms which Action is the shuffle's choice.

Bitcoin contrast. A Bitcoin input is an input and an output is an output; here a dummy spend looks exactly like a real one and, like a real one, publishes a nullifier the pool records.

Do not say

Writing ρ as the note's position in the tree. The seed ρ is a per-note uniqueness value: a dummy's is the Extract of a random point, an output's is the nullifier of the note spent beside it. Neither is a leaf index.

Source: *Wallet Guide*, "Padding, dummies, and shuffling"; *Ironwood Guide*, "Actions, bundles, and dummy notes"; protocol specification, "Dummy Notes (Orchard)"

Board 68 — The statement, in the validator's language

all four

Write

- *"Some leaf under this anchor is a note I own; this is its serial number; this commitment hides its value change; this signing key is a fresh disguise of the owner's — and I have shown you neither the leaf, the note, nor the owner's key."*
- public: anchor, $\mathsf{cv}^{\mathsf{net}}$, nf_{old}, rk, cmx, three flags — 8 names, 10 field elements
- witness: both notes, path + position, ak, nk, $\mathsf{r}_{\mathsf{ivk}}$, ivk, α, rcv
- relation: ten clauses A1–A10; one verifying key, selected by branch; a proof of nine fails

Say

Say the statement in the validator's words first. Read the quoted sentence off the board.

Public inputs. The anchor, $\mathsf{cv}^{\mathsf{net}}$, nf_{old}, rk, cmx and the three flags — eight names, ten field elements (the two Pallas points enter by their coordinates). Here $\mathsf{cv}^{\mathsf{net}}$ and nf_{old} are the data model's cv and nf, marked *net* (this Action's value change) and *old* (the note spent).

Witness. Both notes, the path and position, ak, nk, $\mathsf{r}_{\mathsf{ivk}}$, ivk, α, rcv.

Relation. Ten clauses, A1–A10, over those inputs. The node's verifying key fixes exactly these ten, selected by branch; a proof of nine is a proof for a different circuit, and it fails.

Bitcoin contrast — the one thing to emphasise. Bitcoin's validator runs *your* script. This validator runs one verifier keyed to one fixed circuit; the transaction supplies inputs, never logic.

Source: *Ironwood Guide*, "The Action statement"; "The Action circuit and the Halo 2 proof"; protocol specification, "Action Statement (Orchard)"

Board 69 — The ten clauses, ordered by the four checks

all four

Draw

Three columns: *Bitcoin's check* | *clause* | *what the proof establishes*. Rows, in order:

1. coin exists (UTXO lookup) — A3 — a Merkle path from cm_{old} to the anchor
2. not spent twice (entry deleted) — A5 — nf_{old} is this note's serial, keyed by nk
3. conserved ($\sum$ in $\geq \sum$ out) — A4 — cv^{net} hides $v_{\text{old}} - v_{\text{new}}$, both below 2^{64}
4. authorised (script + signature) — A6, A7 — rk disguises ak; ak, nk sit behind the address
5. (a gap, then) well-formed — A1, A2 — both commitments open to notes; $\rho_{\text{new}} = \mathsf{nf}_{\text{old}}$
6. gated — A8, A9 — spends and outputs switched on by the flags
7. sealed — A10 — cross-address transfers switched off, or not

Write

- 4 Bitcoin checks → 7 clauses (A1–A7); 3 consensus gates (A8–A10)
- stays with the node: "has this serial been seen?"

Say

Order the clauses by Bitcoin's checks, not by their numbers. Fill the top four rows first: existence is a Merkle path (A3), no double spend is a keyed serial (A5), conservation is a hidden difference in range (A4), authorisation is a disguised key behind the address (A6, A7).

Then the three rows below the gap. Well-formed (A1, A2): both commitments open to notes, and $\rho_{\text{new}} = \mathsf{nf}_{\text{old}}$. Gated (A8, A9): spends and outputs switched on by the flags. Sealed (A10): cross-address transfers switched off, or not.

Emphasise. Four of Bitcoin's checks become seven clauses; three more are consensus gates the flags drive. The remaining check — "has this serial been seen?" — is global history and stays with the node.

Source: *Ironwood Guide*, "The Action statement"; "Requirements on a shielded payment"; "Verification of a transaction"

Board 70 — Alice's two Actions against the ten clauses

all four

Draw

Three columns: *clause* | *Action 1:* 2 *ZEC → Bob* 1.5 | *Action 2: dummy → change* 0.4999. Rows, in order:

1. A1 — Alice's 2 ZEC note opens cm_{old} — a zero-value dummy opens cm_{old}
2. A2 — Bob's note; $\rho_{\text{new}} := \mathsf{nf}$ of Alice's note — change note; $\rho_{\text{new}} := \mathsf{nf}$ of the dummy
3. A3 — a 32-sibling path to the Ironwood anchor — gated off: $v_{\text{old}} = 0$
4. A4 — cv^{net} commits to +0.5 ZEC — cv^{net} commits to −0.4999 ZEC
5. A5 — nf of Alice's note, under her nk — nf of the dummy, under its random nk
6. A6 — $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$, Alice's ak — the same, the dummy's ak
7. A7 — Alice's ak, nk sit behind her address — the dummy's key, behind its address
8. A8, A9 — both flags set: both products vanish — the same
9. A10 — $\delta = 0$ (Ironwood): nothing constrained — the same

Write

$$+0.5 - 0.4999 = +0.0001 \text{ ZEC} = 10\,000 \text{ zatoshi} = \text{fee} = v_{\text{balance}}$$

Say

Walk the two columns clause by clause. Action 1 spends Alice's 2 ZEC note and creates Bob's 1.5 ZEC note; Action 2 spends a dummy and creates her 0.4999 ZEC change.

One row is switched off, one changes sign. Clause A3 is gated off for the dummy ($v_{\text{old}} = 0$); clause A4 commits to +0.5 ZEC in one Action and −0.4999 ZEC in the other. Clause A2 chains $\rho_{\text{new}} := \mathsf{nf}$ of whatever is spent beside the new note — Alice's note for Bob's, the dummy for the change.

The rest has the same shape in both. A serial under the spender's nk (A5), a disguise of the spender's ak (A6), that ak, nk behind the address (A7); both flags set, so both products vanish (A8, A9); $\delta = 0$ in the Ironwood component, so A10 constrains nothing.

The two nets sum to the fee. The sum $+0.5 - 0.4999 = +0.0001$ ZEC $= 10\,000$ zatoshi is the fee, and the bundle's v_{balance} (*Balance and authority*).

Do not say

Reading ρ as where the note sits in the tree. In the A2 row the new note's ρ is the nullifier of the note spent in the same Action; the 32-sibling path of A3 is a private witness and never enters it.

Source: *Ironwood Guide*, "The Action statement"; *Wallet Guide*, "Padding, dummies, and shuffling"; "The conventional fee"

Board 71 — What Alice's public fields carry

all four

Draw

Three columns: *field* | *Action 1* | *Action 2*. Rows, in order:

1. $\mathsf{cv}^{\mathsf{net}}$ — +0.5 ZEC under a fresh rcv — −0.4999 ZEC under a fresh rcv
2. $\mathsf{nf}_{\mathsf{old}}$ — Alice's note's serial: the note is retired — the dummy's serial, recorded all the same
3. rk — a fresh disguise of Alice's ak — a fresh disguise of the dummy's ak
4. cmx — Bob's 1.5 ZEC note — Alice's 0.4999 ZEC change note, at her internal address
5. epk, C_{enc} — to Bob's pk_{d}; opened with his ivk — to Alice's internal pk_{d}; found with $\mathsf{ivk}^{\mathsf{int}}$
6. C_{out} — under Alice's ovk: she can re-read it — no ovk for change (wallet default): unrecoverable
7. (a gap, then) bundle — one Ironwood anchor; flags $(1, 1, 1)$; $v_{\text{balance}} = +10\,000$ zatoshi; one proof; two SpendAuthSigs; one binding signature

Write

- which is "Action 1" on the wire: the shuffle decides
- 2 nullifiers, 2 commitments: every two-Action bundle

Say

Read the wire, field by field. The commitment $\mathsf{cv}^{\mathsf{net}}$ is +0.5 ZEC in one Action and −0.4999 ZEC in the other, each under a fresh rcv. The serial $\mathsf{nf}_{\mathsf{old}}$ retires Alice's note in one; in the other it is the dummy's, recorded all the same. The key rk is a fresh disguise of Alice's ak, or of the dummy's. The commitment cmx is Bob's 1.5 ZEC note, or Alice's 0.4999 ZEC change note at her internal address.

Delivery. The pair epk, C_{enc} goes to Bob's pk_{d}, opened with his ivk, and to Alice's internal pk_{d}, found with $\mathsf{ivk}^{\mathsf{int}}$. The ciphertext C_{out} under Alice's ovk lets her re-read what she sent Bob; the change has no ovk (wallet default) and its C_{out} is unrecoverable.

Bundle. One Ironwood anchor; flags $(1, 1, 1)$; $v_{\text{balance}} = +10\,000$ zatoshi; one proof; two SpendAuthSigs; one binding signature.

Emphasise. Which of the two is "Action 1" on the wire is decided by the shuffle. Two nullifiers, two commitments: the shape of every two-Action bundle.

Do not say

"Alice recovers her change through ovk." She finds it with $\mathsf{ivk}^{\mathsf{int}}$; the change's C_{out} is written without an ovk and is unrecoverable.

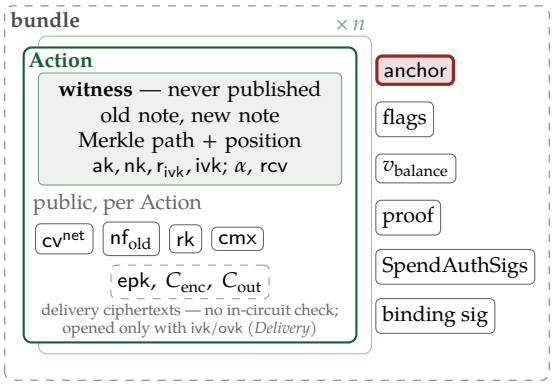
Source: *Ironwood Guide*, "The outgoing ciphertext"; *Wallet Guide*, "Padding, dummies, and shuffling"; "Internal keys: change and auto-shielding"; "The transaction plan"

Board 72 — Coin exists — A3, the path to the anchor

exists

Draw

1. Keep Board 1's box up (or redraw it: bundle, Action, witness block, four rim fields, six bundle fields).
2. Box the anchor field on the right edge in red.



Write

- fold $\mathsf{Extract}(\mathsf{cm}_{\text{old}})$ up 32 witnessed siblings to root
- $v_{\text{old}} \cdot (\text{root} - \text{anchor}) = 0$
- witness: siblings, position bits; public: anchor, one per bundle

Say

Bitcoin looks the coin up in the UTXO set, and so names it. This clause says only "some leaf under this anchor": the candidate set is the Ironwood tree under the anchor.

Mechanism. Fold $\mathsf{Extract}(\mathsf{cm}_{\text{old}})$ up 32 witnessed siblings to root; then $v_{\text{old}} \cdot (\text{root} - \text{anchor}) = 0$. The witness is the siblings and the position bits; the public part is the anchor, one per bundle.

The dummy gate. A zero-value dummy skips the check by arithmetic, not by a flag.

Ask

"Which of the four checks?" — coin exists.

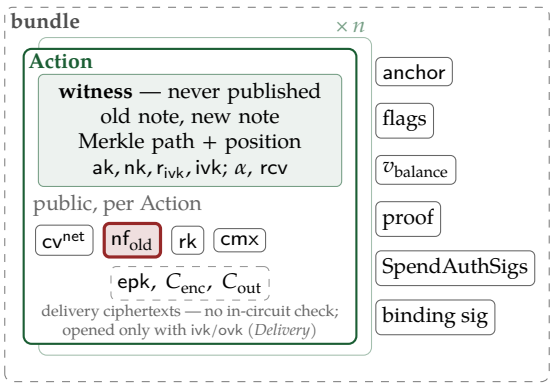
Source: *Ironwood Guide*, "Authentication paths"; "Anchors"; "The Action statement"

Board 73 — No double spend — A5, the nullifier

not twice

Draw

Same box as Board 1; box the nf_{old} field on the rim in red.



Write

$$\mathsf{nf}_{\text{old}} = \mathsf{Extract}([\,\mathsf{Poseidon}_{\mathsf{nk}}(\rho_{\text{old}}) + \psi_{\text{old}}\,]\,G^{\mathsf{nf}} + \mathsf{cm}_{\text{old}})$$

- three lines: membership (A3), serial (A5), balance (A4)

Say

Bitcoin deletes the entry; here an unlinkable serial is added.

Mechanism. Clause A5 binds the published serial to the spent note under the witnessed nk; otherwise a spender could invent a fresh serial per spend and the set would never repeat.

What the proof cannot do. "Seen before?" is global history: the node checks the set, the proof cannot.

The opening's three lines. Membership (A3), serial (A5), balance (A4, *Balance and authority*).

Ask

"Which check?" — not spent twice.

Do not say

Calling ρ the note's tree position. The formula takes the note's seed ρ and ψ under nk; the leaf index is a private witness of A3 and never enters the nullifier.

Source: *Ironwood Guide*, "The nullifier"; "Verification of a transaction"

Board 74 — Authorised — the linkage problem first

authorised

Write

$$\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk}), \qquad \mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d} \quad \text{for every } d$$

- one ak per wallet, not per coin
- publish ak + signature ⇒ every spend, from every address, linked
- needed: a fresh key per spend that still proves the same secret

Say

One spend key ak **per wallet, not per coin.** Every diversified address shares it through ivk (*Keys and addresses*).

The obvious design fails. Publish ak and a signature under it, and every spend the wallet ever makes is stamped with one key: every note it spends, from every address, linked.

Bitcoin contrast. Bitcoin's version of this is address reuse, and it is optional. Here it would be structural.

Needed. A key that is fresh for every spend and still proves the same secret.

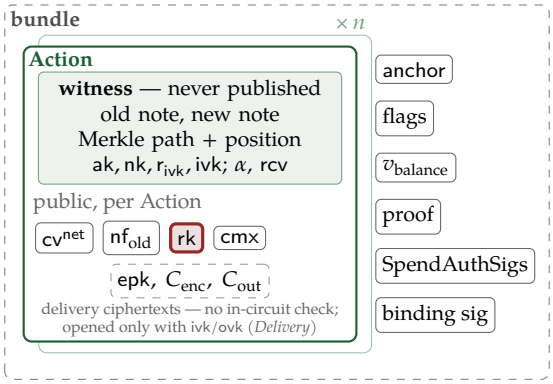
Source: *Crypto Guide*, "Key re-randomisation and unlinkability"; *Ironwood Guide*, "Randomised validating keys"

Board 75 — Authorised — A6, a disguised key

authorised

Draw

Same box as Board 1; box the rk field on the rim in red.



Write

$$\mathsf{rk} = \mathsf{ak} + [\alpha]\,G^{\mathsf{SpendAuth}}, \qquad \mathsf{rsk} = \mathsf{ask} + \alpha$$

- circuit checks the shift; node checks a signature under rk
- uniform α ⇒ uniform rk, whatever ak

Say

Mechanism. Clause A6: the published key is ak shifted by a fresh, secret α. The circuit checks the shift; the node checks a signature under rk (*Balance and authority*).

Why it is unlinkable. Uniform α makes rk uniform whatever ak is.

The Bitcoin bridge. Schnorr with a per-spend tweak: a non-hardened BIP-32 child whose tweak is uniform and never published.

Ask

"Which check?" — authorised.

Do not say

The multiplicative form $\mathsf{rk} = [\alpha]\,\mathsf{ak}$. The re-randomisation is additive: $\mathsf{rk} = \mathsf{ak} + [\alpha]\,G^{\mathsf{SpendAuth}}$ and $\mathsf{rsk} = \mathsf{ask} + \alpha$. Write it that way, never a variant.

Source: *Ironwood Guide*, "Randomised validating keys"; *Crypto Guide*, "Key re-randomisation and unlinkability"

Board 76 — Authorised — A7, one identity behind the address

authorised

Write

$$\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk}), \qquad \mathsf{pk}_{\mathsf{d_{old}}} = [\mathsf{ivk}]\,\mathsf{g}_{\mathsf{d_{old}}}$$

- one identity: signs (A6), nullifies (A5), is paid (A7)
- circuit: the key is the owner's; signature under rk: the spender holds it

Say

Clause A6 alone proves a disguise of some ak. Clause A7 ties that ak, and the nk of A5, to the spent note's address by the ladder of *Keys and addresses*.

One identity. It signs (A6), nullifies (A5), and is paid (A7): the same secret behind all three, and none of them published.

Authority is split in two. The circuit proves the key is the owner's; the signature under rk proves the spender holds it (*Balance and authority*).

Bitcoin contrast. In Bitcoin a P2PKH address *is* the key's hash, so "this key matches this coin" is one hash comparison in the clear. Here the address is a commitment to the key, and the match is proved.

Source: *Ironwood Guide*, "The Action statement"; "Authorisation of spends"

Board 77 — Well-formed — A1 and A2, both notes really are notes

well-formed

Write

$$\mathsf{cm}_{\mathrm{old}} = \mathsf{NoteCommit}_{\mathsf{rcm}_{\mathrm{old}}}(\mathbf{n}_{\mathrm{old}}), \qquad \mathsf{Extract}(\mathsf{NoteCommit}_{\mathsf{rcm}_{\mathrm{new}}}(\mathbf{n}_{\mathrm{new}})) = \mathsf{cmx},$$
$$\text{with } \rho_{\mathrm{new}} := \mathsf{nf}_{\mathrm{old}}.$$

- note = address, value, ρ, ψ; commitment binding ⇒ one leaf, one note
- chain stores only cmx

Say

A Bitcoin output is plaintext: well-formed means it parses. Here the note is hidden, so "this commitment opens to a note" must be proved.

Clause A1. The object being spent opens as a note — address, value, ρ, ψ — under the witnessed rcm; the commitment is binding, so one leaf cannot be reopened as a second note.

Clause A2. The created note is committed the same way, and the chain stores only cmx.

Emphasise: the chain $\rho_{\mathrm{new}} := \mathsf{nf}_{\mathrm{old}}$ is enforced here. The new note's seed is the serial this Action reveals, so accepted seeds are unique within the pool and two notes cannot share a nullifier.

Do not say

Treating ρ as a tree position. The seed ρ is part of the note — fixed at creation as the nullifier of the note spent beside it — not where the note lands in the tree.

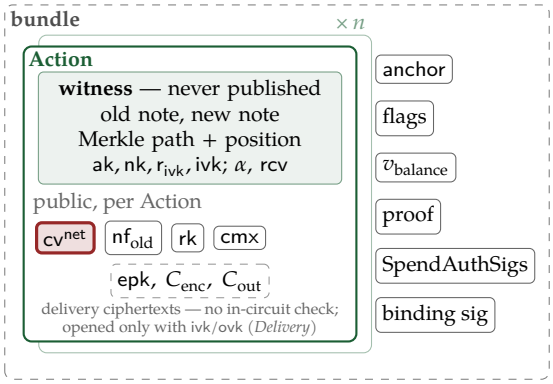
Source: *Ironwood Guide*, "Note commitments"; "Nullifier chaining"; "The Action statement"

Board 78 — Conserved — A4, the value commitment

conserved

Draw

Same box as Board 1; box the $\mathsf{cv}^{\mathsf{net}}$ field on the rim in red.



Write

$$\mathsf{cv}^{\mathsf{net}} = [v_{\text{old}} - v_{\text{new}}]\,V + [\mathsf{rcv}]\,R, \qquad v_{\text{old}}, v_{\text{new}} \in [0, 2^{64}) \text{ (witness typing)}, \quad \text{sign} \in \{-1, +1\}$$

- group order $\approx 2^{254}$: a value near the order is a small negative one

Say

Bitcoin checks $\sum$ in $\geq \sum$ out on amounts it can read; here they are hidden. Clause A4 commits to this Action's value *change* under a fresh trapdoor rcv; the bundle sums these (*Balance and authority*).

The range is the point. The sum lives in a group of order about 2^{254}, where a value near the order is a small negative one. Unbounded, the books would "balance" while minting.

Elements contrast. Elements' range proof exists for the same reason; here it lives inside the Action proof, with the opening.

Ask

"Which check?" — conserved.

Source: *Ironwood Guide*, "Value commitments"; "Value conservation"

Board 79 — Gated — A8 and A9, the flags

gated

Write

$$v_{\text{old}} \cdot (1 - \mathsf{enableSpends}) = 0, \qquad v_{\text{new}} \cdot (1 - \mathsf{enableOutputs}) = 0$$

- flag clear ⇒ only value 0 passes
- coinbase: enableSpends = 0; Alice: both flags set

Say

Bitcoin's coinbase has one null input by format. Here the same rule is arithmetic on a hidden value — the only way to gate what the node cannot read.

Mechanism. The flags are bundle-level bits the node supplies as public inputs; the circuit only multiplies. With a flag clear, only a zero value passes: a cleared enableSpends admits dummies and nothing else.

Coinbase. A coinbase must clear enableSpends: its Actions spend dummies and create the block's shielded reward. Ordinary bundles set both flags, as Alice's does.

Source: *Ironwood Guide*, "The Action statement"; "Chain state and pool rules"; "Bundle flags"

Board 80 — Sealed — A10, the cross-address gate

sealed

Write

$$\delta \cdot (x(\mathsf{g_d}_{\text{old}}) - x(\mathsf{g_d}_{\text{new}})) = 0, \quad \delta \cdot (y(\mathsf{g_d}_{\text{old}}) - y(\mathsf{g_d}_{\text{new}})) = 0,$$
$$\delta \cdot (x(\mathsf{pk_d}_{\text{old}}) - x(\mathsf{pk_d}_{\text{new}})) = 0, \quad \delta \cdot (y(\mathsf{pk_d}_{\text{old}}) - y(\mathsf{pk_d}_{\text{new}})) = 0$$

- $\delta :=$ disableCrossAddress, a public input; on the wire negated as enableCrossAddress, bit 2 of flagsIronwood (reserved, 0, in flagsOrchard)
- Ironwood `0b111`: $\delta = 0$ (Alice); Orchard pool `0b011`: $\delta = 1$

Say

One more public input. The flag $\delta :=$ disableCrossAddress joins the instance. On the wire it travels negated, as enableCrossAddress, bit 2 of the Ironwood flags byte; in the Orchard byte that bit is reserved and 0.

Mechanism. With $\delta = 1$ the four products force the new note to pay the spent note's own receiver, as full points. With $\delta = 0$ they vanish.

The two pools. The Ironwood component sets the bit (`0b111`): Alice's bundle has $\delta = 0$. An Orchard-pool component must clear it (`0b011`): $\delta = 1$.

Bitcoin contrast. The shape is A3's product gate again, on four more rows. A Bitcoin output type cannot be told "pay only yourself"; this one can.

Source: *Ironwood Guide*, "The Action statement", Remark "Meaning of A10"; "Bundle flags"

Board 81 — The seal: three rules, and two checks that are not clauses

sealed

Write

- **No issuance** — coinbase: empty Orchard component
- **No circulation** — every Orchard-pool Action: enableCrossAddress $= 0$
- **No entry** — per transaction: $v_{\text{balance}}^{\text{Orchard}} \geq 0$
- not clauses: canonical encodings; rk $\neq$ identity

Say

Three rules, by name. No issuance: a coinbase carries an empty Orchard component. No circulation: every Orchard-pool Action has enableCrossAddress $= 0$, so A10 confines its output to the spent note's own receiver — enforced by the verifying key consensus selects by branch, so it binds whatever the transaction version. No entry: per transaction, $v_{\text{balance}}^{\text{Orchard}} \geq 0$; value may leave the pool, never enter.

How a sealed-pool note pays anyone. By *leaving*: the value exits through a positive v_{balance}, and the output slot holds a zero-value note to the spent note's own receiver.

Not clauses. Before the proof, the node parses: every field element and point canonically encoded, and rk not the identity. Consensus rules, not relation.

Bitcoin contrast. A Bitcoin soft fork narrows what a script may do, in the clear; this seal narrows what a hidden note may do, by a public flag the circuit reads.

Ask

Checkpoint, then the break — leave the question on the board. "Which clause would you delete to counterfeit, and what stops you?" — Delete A3 and spend a note nobody minted; delete A4's

range and wrap a subtraction into new value; delete A1 and reopen one leaf as two notes; delete A5 and spend one note under a fresh serial each time. What stops you: the circuit is fixed, and consensus selects its verifying key by branch. A proof of a weaker statement is a proof for a different circuit, and no node holds that key.

Source: *Ironwood Guide*, "Chain state and pool rules"; "Actions, bundles, and dummy notes"; "Verification of a transaction". Checkpoint: *Ironwood Guide*, "The Action statement"; "The Action circuit and the Halo 2 proof"

6 Part E′ — Balance and authority, on the running example

Board 82 — Conservation: the envelopes add

conserved

Draw

The table, header first, one note per row, computed values as given:

1. Header: note | v (units) | r | $C(v;r)$.
2. Alice's, 2 ZEC | 20 000 | 4 321 | 56 457.
3. Bob's, 1.5 ZEC | 15 000 | 1 234 | 53 037.
4. change, 0.4999 ZEC | 4 999 | 9 876 | 12 555.

Write

1. Commitment: $C(v;r) := [v]\,V + [r]\,R$; nobody knows $\log_V R$.
2. Toy group: $\mathbb{Z}_q$, $q = 65\,537$, $V = 7$, $R = 11$; one unit $= 10^4$ zatoshi = the fee.
3. Then the table above.
4. Cancellation: $C_A - C_B - C_c - [1]\,V = 56\,395 = [\,4\,321 - 1\,234 - 9\,876\,]\,R = [58\,748]\,R$.

Say

Bitcoin's node adds up amounts it can read; here the amounts sit inside commitments. Part B's $\mathsf{Commit}(v;r)$ is written $C(v;r)$ from here on: $[v]\,V + [r]\,R$, with V and R hashed to Pallas so that nobody knows $\log_V R$. On the board the stand-in is the additive group $\mathbb{Z}_q$ with $q = 65\,537$, $V = 7$, $R = 11$; one unit is 10^4 zatoshi, which is the fee. Logs are free in this toy; on Pallas they are not.

Write the three notes and subtract. Alice's note minus Bob's, minus the change, minus one unit of fee under V: the V-components cancel because the values do. What survives is a commitment to zero under the net randomness, $[58\,748]\,R$.

The one thing to emphasise: only the author who drew the three r knows that net randomness. That knowledge is what the next boards turn into a signature.

Source: *Ironwood Guide*, "Value commitments"; *Crypto Guide*, "The Pedersen commitment"

Board 83 — Conservation: per Action, not per output

conserved

Write

1. Bitcoin: three amounts, read and subtracted.
2. Confidential Transactions: one commitment per input and per output; the two sums differ by the fee.
3. Here, one commitment per Action: $\mathsf{cv} := [v_{\text{old}} - v_{\text{new}}]\,V + [\mathsf{rcv}]\,R$.
4. Alice: two envelopes for three notes, plus v_{balance} beside them.

Say

Bitcoin subtracts three amounts it reads. Confidential Transactions keep that shape with one commitment per input and one per output, and the two sums must differ by the fee.

Here the unit is the Action, so the commitment is already a net. Each Action commits to its spend minus its output, $v_{\text{old}} - v_{\text{new}}$, under a fresh trapdoor rcv. Alice's bundle therefore publishes two envelopes for three notes and declares the net flow v_{balance} beside them.

Source: *Ironwood Guide*, "Value commitments"; *Crypto Guide*, "The Pedersen commitment"

Board 84 — Cancellation is modular: why the Action range-checks

conserved

Write

1. Wrap-around: $C(q+3;\,5) = C(3;\,5) = 76$; q units = 6.5537 ZEC.
2. Counterfeit change: $4\,999 + q = 70\,536$ units = 7.0536 ZEC from a 2 ZEC note.
3. Same envelope: $C(70\,536;\,9\,876) = 12\,555 = C(4\,999;\,9\,876)$.
4. Toy cancellation accepts it: $20\,000 - 15\,000 - 70\,536 - 1 = -65\,537 \equiv 0 \pmod{q}$.
5. Witness typing: $v_{\text{old}}, v_{\text{new}} \in [0, 2^{64})$, sign $\in \{-1, +1\}$; Pallas order: 255 bits.

Say

The sum cancels modulo the group order, and a value at the order commits like zero. Show it: $C(q+3;\,5)$ equals $C(3;\,5)$, both 76, and q units is 6.5537 ZEC.

So the toy accepts a counterfeit. Change of $4\,999 + q = 70\,536$ units, 7.0536 ZEC, out of a 2 ZEC note: the commitment does not move, 12 555 either way, and the value sum is $-65\,537$, which is $0 \bmod q$.

The witness typing closes the gap inside the Action proof. Old and new values lie in $[0, 2^{64})$ and the sign is ± 1, so every net value is a small signed integer; the Pallas order has 255 bits, and a bundle's bounded sum of bounded values cannot reach it.

The contrast. Elements attaches a range proof to each output. Here the range check is one clause of the statement the Action proof already carries. Bitcoin reads each amount and bounds it.

Source: *Ironwood Guide*, "The Action statement"; "Value conservation"; *Crypto Guide*, "The Pedersen commitment"

Board 85 — The binding signature: a key only a balanced author knows

conserved

Write

1. Per Action (A4): $\mathsf{cv} = [v_{\text{old}} - v_{\text{new}}]\,V + [\mathsf{rcv}]\,R$.
2. Sum over the bundle: $\sum_i \mathsf{cv}_i = [\sum_i (v_{\text{old}} - v_{\text{new}})_i]\,V + [\mathsf{bsk}]\,R, \quad \mathsf{bsk} := \sum_i \mathsf{rcv}_i$.
3. Subtract the declared flow: $\mathsf{bvk} := \sum_i \mathsf{cv}_i - [v_{\text{balance}}]\,V$.
4. Balanced $\iff \mathsf{bvk} = [\mathsf{bsk}]\,R$.

Say

Sum the Action commitments and subtract the declared net flow. The V-component carries the net value difference, the R-component carries the sum of trapdoors, bsk. Taking away $[v_{\text{balance}}]\,V$ leaves a pure R-multiple exactly when the books balance.

The key bvk **is public: every node recomputes it from the wire.** Its secret bsk is a sum of trapdoors only the author drew, and it is a discrete log of bvk only when the books balance; otherwise finding one means finding $\log_R V$.

The binding signature is RedPallas under bvk **on base** R**.** That is a Schnorr proof of knowledge of bsk, hence of balance, revealing no amount and not bsk itself.

The contrast. Bitcoin's check is arithmetic on amounts it reads; this one is a signature check under a key derived from commitments.

Source: *Ironwood Guide*, "The binding signature"; "Value conservation"; *Crypto Guide*, "Binding signatures as a proof of knowledge of a discrete logarithm"

Board 86 — The one public amount: v_{balance}

conserved

Write

1. Field v_{balance}: cleartext, signed, one per pool; positive = value leaving the pool.
2. Fully shielded: $\sum_i (v_{\text{old}} - v_{\text{new}})_i$ = fee, so v_{balance} = fee.
3. Bitcoin's implicit $\sum$ in $-\sum$ out = fee becomes a public, signed number.
4. Alice: $v_{\text{balance}} = +10\,000$ zatoshi.
5. Transparent outputs: paid from a positive v_{balance}. Shielding: a negative one.

Say

One amount in the bundle is cleartext: v_{balance}, signed, one per pool, positive when value leaves the pool. Fully shielded, the only value leaving is the fee, so the sum of the per-Action nets is the fee and v_{balance} is the fee. Bitcoin's implicit "inputs minus outputs equals fee" becomes a public, signed number. Alice's reads +10 000 zatoshi.

The transparent leg enters through the same field. Transparent outputs are paid from a positive v_{balance}; shielding funds a negative one, its Actions spending dummies and creating real notes. The crossing amount is public; the receiving note is not.

The contrast. Bitcoin's fee is reconstructed by looking up every input. Fully shielded, as Alice's is, the balancing field is the one amount the transaction states, and it is the fee; with a transparent leg the fee is again a difference of public amounts, as in Bitcoin.

Do not say

"The pool is auditable" — v_{balance} is one transaction's crossing, and the pool's balance row is the public sum of such crossings; nothing inside the pool is audited.

Source: *Ironwood Guide*, "The balancing value"; "Chain state and pool rules"; *Wallet Guide*, "The conventional fee"

Board 87 — Authorisation on the wire: two signatures, one scheme

authorised

Draw

The table, header first, one signature per row:

1. Header: (blank) | per | base P and key X | knowing x means.
2. SpendAuthSig | Action | $G^{\mathsf{SpendAuth}}$, $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$ | authority: $\mathsf{rsk} = \mathsf{ask} + \alpha$.
3. binding signature | pool | R, $\mathsf{bvk} = \sum \mathsf{cv} - [v_{\text{balance}}]\, V$ | balance: $\mathsf{bsk} = \sum \mathsf{rcv}$.

Write

1. RedPallas: Schnorr on Pallas, base point P a parameter.
2. Key: $X = [x]\, P$.
3. Signature: (N, s), $32 + 32 = 64$ bytes.
4. Verifies iff $[s]\, P = N + [c]\, X, \quad c = H(N \,\|\, X \,\|\, m)$.
5. Note: N is Part B's R, renamed; R stays the commitment base.
6. Then the table above.

Say

Both signatures in the bundle are one scheme. RedPallas is Schnorr on Pallas with the base point a parameter: key $X = [x]\, P$, a 64-byte signature (N, s), verified by $[s]\, P = N + [c]\, X$ with $c = H(N \,\|\, X \,\|\, m)$. The nonce commitment is Part B's R renamed to N, so that R can stay the commitment base.

Two instantiations, two bases, two secrets. Per Action, under $G^{\mathsf{SpendAuth}}$ with key $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$, knowing the secret $\mathsf{rsk} = \mathsf{ask} + \alpha$ means authority. Per pool, under R with key $\mathsf{bvk} = \sum \mathsf{cv} - [v_{\text{balance}}]\, V$, knowing the secret $\mathsf{bsk} = \sum \mathsf{rcv}$ means balance.

Do not say

Multiplicative re-randomisation — the key is $\mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$ and the secret $\mathsf{ask} + \alpha$, never $[\alpha]\, \mathsf{ak}$.

Source: *Ironwood Guide*, "The RedPallas signature scheme"; "The transaction format"; *Crypto Guide*, "RedDSA: re-randomisable Schnorr for Orchard"

Board 88 — Authorisation on the wire: who checks what

authorised

Write

1. Circuit: rk is a shift of the ak welded into the spent note's address (A6, A7) — hidden pedigree.
2. Node: SpendAuthSig verifies under rk — public approval.
3. Signing needs $\mathsf{ask} + \alpha$: both, or nothing.
4. One ak behind every address of the wallet ⇒ a fresh rk per spend.

Say

Division of labour. The circuit checks the key's pedigree, which must stay hidden: rk is a shift of the ak welded into the spent note's address, clauses A6 and A7. The node checks the signature, which is public: the holder of that key approved this transaction. Signing needs $\mathsf{ask} + \alpha$ — both, or nothing.

The contrast. Bitcoin uses the coin's own key, in the script or hashed there, revealed at its one spend; reuse links only if the address was reused. Here one ak stands behind every address of the wallet, so an un-randomised spend would publish it at every note the wallet ever spends.

Hence a fresh, uniformly distributed rk **per spend**, unlinkable to ak and to every other rk.

Do not say

Multiplicative re-randomisation — the shift is additive, $\mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$, never $[\alpha]\, \mathsf{ak}$.

Source: *Ironwood Guide*, "Randomised validating keys"; *Crypto Guide*, "Key re-randomisation and unlinkability"

Board 89 — What the signatures sign

authorised

Write

1. Message m = the ZIP-244 signature digest: a tree of personalised BLAKE2b hashes, one node per transaction component.
2. Covers all effecting data (everything under the txid): per Action nf, cmx, cv, rk, ciphertexts; the flags; v_{balance}; the branch identifier.
3. Sighash type fixed at ALL: no ANYONECANPAY, SINGLE, NONE.
4. From v6 the anchor is authorizing data: pinned by the proof (A3), committed by the block (Part H).
5. No transparent inputs ⇒ signature digest = txid.

Say

Every shielded signature in the transaction signs one message. That message is the ZIP-244 signature digest, a tree of personalised BLAKE2b hashes, one node per transaction component.

It commits to all effecting data — everything under the txid. Each Action's nf, cmx, cv, rk and ciphertexts, the flags, v_{balance}, and the branch identifier of the network upgrade. No Action can be altered or transplanted into another transaction.

For the shielded signatures the sighash type is fixed at ALL. Nothing like ANYONECANPAY, SINGLE or NONE: one message, all of it, for the spend-authorisation and binding signatures alike.

One field the digest used to cover no longer is. From v6 the anchor joins the proofs and signatures in authorizing data. The proof pins it as a public input, clause A3, and the block commits to authorizing data (Part H); a transaction can be re-anchored without changing its txid — specified (ZIP-374 v2), wallet support pending (Part H).

The contrast. Without transparent inputs, the signature digest is the txid. Bitcoin's sighash byte chooses coverage per input; so do Zcash's transparent inputs, but every shielded signature covers all of it.

Do not say

"The signature pins the anchor" — from v6 the digest does not cover the anchor; the proof pins it as a public input.

Source: *Ironwood Guide*, "Transaction digests and signatures"; *Wallet Guide*, "Authorisation of a transaction"

Board 90 — Alice's two Actions

conserved, authorised

Draw

The table, header first, then one row per field, top to bottom:

1. Header: (blank) | Action 1 | Action 2.
2. spent note, v_{old} | Alice's, 2 ZEC | dummy, 0: A3 gated off.
3. created note, v_{new} | Bob's, 1.5 ZEC | change, 0.4999 ZEC, internal address.

4. $v_{\text{old}} - v_{\text{new}}$ | +0.5 ZEC | −0.4999 ZEC.
5. nf | Alice's note's | the dummy's.
6. rk, SpendAuthSig | $\mathsf{ak} + [\alpha_1]\, G^{\mathsf{SpendAuth}}$, Alice signs | a throwaway key's, it signs.
7. toy cv | $C(+5\,000;\ 3\,141) = 4\,014$ | $C(-4\,999;\ 2\,718) = 60\,442$.

Write

1. Action count: $n_{\text{act}} = \max(\max(1, 2), 2) = 2$.
2. Fee: $5\,000 \cdot \max(2, 2) = 10\,000$ zatoshi $= v_{\text{balance}}$.

Say

Two outputs force two Actions, so one dummy spend pads the spend list. The count is $\max(\max(1, 2), 2) = 2$: one real spend, two outputs, floor of two. The dummy is a zero-value note under a fresh random spending key, its ρ the Extract of a random point; it still publishes a nullifier and still signs.

Read the table column by column. Action 1 spends Alice's 2 ZEC note and creates Bob's 1.5 ZEC note, net +0.5 ZEC, nullifier Alice's note's, signed under $\mathsf{ak} + [\alpha_1]\, G^{\mathsf{SpendAuth}}$. Action 2 spends the dummy, with A3 gated off, and creates the 0.4999 ZEC change at Alice's internal address, net −0.4999 ZEC, nullifier the dummy's, signed under a throwaway key. The toy commitments are 4 014 and 60 442.

The fee is $5\,000 \cdot \max(2, 2) = 10\,000$ zatoshi, and that is v_{balance}. The change goes to Alice's internal address, derived from her own keys and never published.

Do not say

- A minimum-Actions constant as the reason for the dummy — the two outputs already force the count, $\max(1, 2) = 2$, before the floor of two applies.
- The dummy's ρ as a tree position — it is the Extract of a random point, a per-note uniqueness seed.
- "Alice recovers her change via ovk" — the change is an ordinary note to her internal address; how she finds it is Part F's.

Source: *Wallet Guide*, "Padding, dummies, and shuffling"; "The conventional fee"; "Internal keys: change and auto-shielding"; protocol specification, "Dummy Notes (Orchard)"

Board 91 — The bundle balances, however it is shuffled

conserved

Write

1. Pairing shown: $\sum \mathsf{cv} = 4\,014 + 60\,442 = 64\,456$.
2. Key: $\mathsf{bvk} = 64\,456 - [1]\, V = 64\,449 = [5\,859]\, R$.
3. Secret: $\mathsf{bsk} = 3\,141 + 2\,718 = 5\,859$.
4. Other pairing: $\mathsf{cv} = 8\,484$ and 55 972; same $\sum \mathsf{cv} = 64\,456$, same bvk, same bsk.

Say

Check the pairing shown in the toy. The two commitments sum to 64 456; subtract one unit of fee under V and bvk is 64 449, which is $[5\,859]\, R$; and bsk is $3\,141 + 2\,718 = 5\,859$. The key and the secret agree.

Now pair the other way: Alice's note with the change, the dummy with Bob. The commitments become 8 484 and 55 972, but the sum is the same 64 456, the same bvk, the same bsk. Balance is a property of the bundle, not of the pairing.

That is why the shuffle costs nothing. Spends and outputs are padded, shuffled independently, and zipped; each Action draws its own rcv and α, the dummy included, so nothing on the wire says which Action is the dummy.

The contrast. Bitcoin: one input naming the 2-coin output, two outputs, and the change tell — one round amount and one not. Here: two Actions of one shape and a fee.

Ask

Checkpoint: what does the validator learn from Alice's bundle?

Answer. That the four checks pass: the anchor is a main-chain root and the proof verifies (exists); two nullifiers are absent from the set (not twice); two SpendAuthSigs verify under $\mathsf{rk}_1, \mathsf{rk}_2$ (authorised); the binding signature verifies under the recomputed bvk (conserved). The metadata: two Actions; $v_{\text{balance}} = +10\,000$ zatoshi, hence the fee; the anchor; two nullifiers, two cmx, two ciphertext pairs; 1 640 bytes of Actions, a 7 264-byte proof, three 64-byte signatures; when it arrived. Not: who, to whom, the amounts, which leaf, or which Action is the dummy — or whether either is.

Do not say

- "A few hundred bytes" — the proof is 7 264 bytes for two Actions, the Actions 1 640, the signatures three times 64.
- "The proof hides sender and receiver" — say what the validator does not learn, as listed; the hiding is the commitments', the keyed nullifier's and the re-randomised key's, not the proof's.

Source: *Wallet Guide*, "Padding, dummies, and shuffling"; "Authorisation of a transaction"; *Ironwood Guide*, "The binding signature". Checkpoint: *Ironwood Guide*, "Verification of a transaction"; "Construction of a transaction"; "The transaction format"

7 Part G — The proof system

Board 92 — A zk-SNARK, precisely enough

all four checks

Write

1. Relation R: public statement x, private witness w.
   $\text{Prove}(x, w) \to \pi, \qquad \text{Verify}(x, \pi) \in \{0, 1\}$
2. Knowledge sound: an extractor pulls a w with $(x, w) \in R$ out of any convincing prover.
3. Zero knowledge: a simulator given only x reproduces π.
4. Schnorr = proof of knowledge of $\log_G X$. SNARK = the same, for any relation.

Say

In Bitcoin the validator holds everything it checks. The witness sits in the transaction; the node re-runs the script and checks the signature against data in front of it. Here the validator holds only the public statement x and a short string π; the witness w never leaves the prover.

Knowledge soundness is the property to insist on. Plain soundness would allow "a spendable note exists" — other people's notes exist. The extractor is the point: from any prover that convinces the verifier, a w with $(x, w) \in R$ can be pulled out, so the prover must *hold* one.

Zero knowledge is the other half. A simulator given only x reproduces π: the proof adds nothing beyond the statement. What that does and does not buy is the next board.

The contrast to emphasise. A Schnorr signature is already a proof of knowledge, for the one relation "I know $\log_G X$". A SNARK is the same guarantee for any relation — here the ten clauses of the Action statement.

Do not say

"The proof hides the sender and the receiver." The proof hides the witness; whether the published fields can be linked is a separate question (next board).

Source: *Crypto Guide*, "SNARKs: succinct non-interactive arguments of knowledge"; "Knowledge soundness and extractors"; "The simulation paradigm and zero knowledge"

Board 93 — Zero knowledge: what π hides, and what it does not

all four checks

Write

1. Whatever a verifier learns from π, a simulator produces from x alone.
2. Published: nf, rk, cv, cmx.
3. Unlinkable because of: commitment hiding, the keyed nullifier PRF, key re-randomisation — not π.
4. Zero knowledge: necessary, not sufficient. A perfect proof beside nf = cmx hides nothing.

Say

The proof reveals nothing beyond the public inputs. That is the whole content of zero knowledge: whatever a verifier learns from π, a simulator produces from x alone.

But the public inputs are published. The nullifier, the re-randomised key, the value commitment, the note commitment all go on chain. That they cannot be linked back to the note and the key rests on other assumptions — commitment hiding, the keyed nullifier PRF, key re-randomisation, from Parts C and E — not on π.

Zero knowledge is necessary, not sufficient. Put a perfect proof beside a nullifier equal to cmx and it hides nothing.

The contrast. Bitcoin's witness is the secret's public shadow: a signature, a script preimage. Here the witness never leaves the prover, and the shadow is the public-input tuple, unlinkable for reasons the proof does not supply.

Do not say

"The proof hides the sender and the receiver." Unlinkability of the published tuple is bought by hiding commitments, the keyed PRF and re-randomisation, not by π.

Source: *Crypto Guide*, "The simulation paradigm and zero knowledge"; *Ironwood Guide*, "The Action circuit and the Halo 2 proof"; "Privacy"

Board 94 — Succinct, and non-interactive

all four checks

Write

1. Succinct: proof size logarithmic in the circuit — commitments and openings, never the polynomials. One linear MSM, shared across a batch.
   challenge := Hash(everything sent so far) (Fiat–Shamir)
2. Proof of knowledge + Fiat–Shamir = signature.

Say

Succinct means the polynomials never travel. The proof is logarithmic in the circuit: commitments and openings only. Halo 2's verifier still pays one linear multi-scalar multiplication, shared across a batch; it is a SNARK in the proof-size sense.

Non-interactive means the coin is a hash. Wherever a live verifier would toss a coin, the prover computes the challenge as a hash of everything sent so far, statement and every commitment included. A commitment is an input of the hash, so it is fixed before the challenge that tests it exists. The proof becomes one static object; verification recomputes the hashes and runs the same checks.

The contrast. A full node that has never met Alice verifies her Action long after it was made, from the proof string alone. Schnorr is Fiat–Shamir applied to the log relation: proof of knowledge plus Fiat–Shamir is a signature; a SNARK is the same recipe, kept short.

Do not say

Verification-time figures. The deck says $O(\log n)$ per proof plus one shared linear MSM, nothing in milliseconds.

Source: *Crypto Guide*, "SNARKs: succinct non-interactive arguments of knowledge"; "The Fiat–Shamir transform: from interactive to non-interactive"; *Halo 2 Guide*, "Removing the verifier: the Fiat–Shamir transform"

Board 95 — Arithmetisation: the claim becomes a table

all four checks

Draw

Rule eleven columns; brace a, b, c as *advice*, $q_M \ldots q_C$ as *fixed*, PI as *instance*. Write the rows top to bottom:

	advice			fixed					instance	
row	a	b	c	q_M	q_L	q_R	q_O	q_C	PI	gate
0	3	3	9	1	0	0	−1	0	0	$x \cdot x = x^2$
1	9	3	27	1	0	0	−1	0	0	$x^2 \cdot x = x^3$
2	27	3	30	0	1	1	−1	0	0	$x^3 + x = s$
3	30	0	0	0	1	0	0	5	−35	$s + 5 = 35$

Write

1. Toy claim: "I know x with $x^3 + x + 5 = 35$" over $\mathbb{F}_{97}$ ($x = 3$).
   $q_M\, a\, b + q_L\, a + q_R\, b + q_O\, c + q_C + \mathrm{PI} = 0$ on every row
2. Copy constraints: $a_0 = b_0 = b_1 = b_2$, $c_0 = a_1$, $c_1 = a_2$, $c_2 = a_3$.
3. Range check = lookup into a table, not algebra.

Say

The Action statement in miniature. "I know x with $x^3 + x + 5 = 35$" over $\mathbb{F}_{97}$; the answer is 3. One operation per row, one gate equation evaluated on every row.

Three kinds of column. Advice is the prover's secret trace. Fixed columns are the circuit: the selectors pick which operation a row performs. The instance column carries the public 35.

Wiring and lookups. Copy constraints wire the rows: the x in row 0 is the x in rows 1 and 2, and each row's output feeds the next. A range check is not algebra; it is a lookup into a table.

The contrast. Bitcoin Script is executed by every validator; a circuit is never executed, only checked.

Ask

Which column does the verifier never see? — Advice: the prover's trace. Fixed columns are the circuit, known to everyone; the instance column is the public 35.

Source: *Halo 2 Guide*, "From computation to a grid of constraints"; "Proving a value sits in a table: lookups"; "PLONKish arithmetisation"

Board 96 — Rows become polynomials

all four checks

Write

1. Roots of unity in $\mathbb{F}_{97}$: $\omega = 22$, $\omega^2 = 96 = -1$, $\omega^4 = 1$; row i lives at $\omega^i \in H = \{1, 22, 96, 75\}$.
   $a(1) = 3$, $a(22) = 9$, $a(96) = 27$, $a(75) = 30 \implies a(X) = 90 + 61X + 22X^2 + 24X^3$
   $G(X) := q_M ab + q_L a + q_R b + q_O c + q_C + \mathrm{PI}$, $\quad \deg G \le 9$
   every gate holds $\iff$ $G(\omega^i) = 0$ for all i $\iff$ G vanishes on H

Say

Name the rows by roots of unity. In $\mathbb{F}_{97}$ the number 22 is a fourth root of unity, so the four rows live at $1, 22, 96, 75$.

Interpolate each column. The advice column a becomes the unique polynomial of degree below 4 through its four values. Four numbers in, four coefficients out: the column *is* the polynomial, sampled at the rows. Likewise b, c and the public columns.

Substitute into the gate. The polynomial G has degree at most 9, and $G(\omega^i)$ is exactly row i's gate equation. So every gate holds if and only if G vanishes on H.

The contrast. A Merkle root also folds many rows into one object, but it opens one leaf at a time; a polynomial can be probed anywhere.

Source: *Halo 2 Guide*, "From the grid to polynomials"

Board 97 — All rows at once: one identity, or none

all four checks

Write

$Z_H(X) := \prod_{h \in H}(X - h) = X^4 - 1$
G vanishes on H $\iff$ $Z_H \mid G$ $\iff$ $G = t \cdot Z_H$

1. Honest: $\deg G = 9$, $\deg t = 5$, remainder 0.
2. Cheat, $c_0 = 10$: row 0 reads $3 \cdot 3 - 10 = -1 \neq 0$, so $Z_H \nmid G^*$.
   $D(X) := G^*(X) - t^*(X)\, Z_H(X) = 24\,(1 + X + X^2 + X^3) \neq 0$

Say

The vanishing polynomial. The polynomial that is zero exactly on the rows is $X^4 - 1$. By the factor theorem, applied four times, G vanishes on H if and only if Z_H divides G, that is, $G = t \cdot Z_H$ for some quotient t.

Honest and dishonest traces. The honest trace divides exactly: degree 9 over degree 4, quotient of degree 5, remainder zero. Let the prover claim $x^2 = 10$ and write $c_0 = 10$: row 0's gate no longer holds, Z_H no longer divides G^*, and the Euclidean division leaves the remainder $24(1+X+X^2+X^3)$.

The one thing to keep. A correct computation *is* a polynomial identity; a wrong one cannot be. The copy constraints get the same treatment (a running product that equals 1 when the wiring holds and, at random challenges, almost never otherwise), and so do lookups.

The contrast. Bitcoin's validator checks each input's script on its own; here every row collapses into one identity that holds everywhere or fails almost everywhere.

Source: *Halo 2 Guide*, "From vanishing to a single identity"; "Enforcing the wiring: the permutation argument"

Board 98 — One random point: the protocol

all four checks

Write

1. Prover commits to a, b, c and t; verifier knows $q_M, \ldots, q_C$ and PI.
2. Transcript hash draws $z \in \mathbb{F}$ (Fiat–Shamir; BLAKE2b in the deployed transcript).
3. Prover opens $a(z), b(z), c(z), t(z)$ with short proofs.
4. Verifier forms $G(z)$, computes $Z_H(z) = z^4 - 1$, checks
   $G(z) \stackrel{?}{=} t(z)\, Z_H(z)$
5. Honest, $z = 20$: $Z_H(20) = 46$, $t(20) = 67$, $G(20) = 75 = 67 \cdot 46$. ✓

Say

Four moves. The prover commits to the advice columns and the quotient; the fixed and instance columns the verifier already knows. The transcript hash draws a random point z. The prover opens the committed polynomials at z, with short proofs that these are the committed values. The verifier assembles $G(z)$ from the openings, computes $z^4 - 1$ itself, and checks one field equation.

Run the numbers. At $z = 20$: $Z_H(20) = 46$, $t(20) = 67$, and $G(20) = 75$, which is $67 \cdot 46$ in $\mathbb{F}_{97}$. Every honest prover passes at every z, because $G = t\, Z_H$ holds as polynomials.

The contrast. Bitcoin's validator re-executes the script; this one evaluates one equation at one point.

Source: *Halo 2 Guide*, "Why checking one random point is a proof"; "Removing the verifier: the Fiat–Shamir transform"; "The complete Halo 2 protocol"

Board 99 — One random point: why a lie is caught

all four checks

Write

1. Cheat: $D := G^* - t^* Z_H \neq 0$, frozen before z is drawn.
   $\Pr_z[\text{cheat accepted}] = \Pr_z[D(z) = 0] \leq \dfrac{\deg D}{|\mathbb{F}|}$ (Schwartz–Zippel)
2. Roots of $D = 24(1 + X + X^2 + X^3)$: $\{22, 75, 96\}$; 3/97 for that t^*, 11/97 for any admissible t^*.
3. At $z = 20$: $G^*(20) = 20$ but $t^*(20)\, Z_H(20) = 64$. Caught.
4. Over $\mathbb{F}_{p_{\text{Pallas}}}$, $\deg D \approx 2^{14}$: about 2^{-240}; the discrete-log layer gives about 126 bits.

Say

The lie is frozen first. The commitments pin G^* and t^* before z is drawn, so the difference D is a fixed nonzero polynomial. The check passes exactly when $D(z) = 0$, and a nonzero polynomial has at most $\deg D$ roots — Schwartz–Zippel.

On the toy. For the $c_0 = 10$ cheat the roots are 22, 75 and 96: the three rows where the tampered gate still held. That is $3/97$ for this quotient, $11/97$ for any admissible one. At $z = 20$ the two sides read 20 and 64: caught.

At scale. Over $\mathbb{F}_{p_{\text{Pallas}}}$ with $\deg D$ around 2^{14} the term is about 2^{-240}. That is one Schwartz–Zippel term, not the system's security level; the discrete-log layer gives about 126 bits.

The one thing to keep. A false statement is too rigid to hide: it disagrees with the truth almost everywhere, and one probe lands where it disagrees. Bitcoin never needs this; its validator sees the data.

Ask

Why $3/97$ for this cheat but $11/97$ in general? — This D has exactly three roots, the rows where the tampered gate still held; the general bound is $\deg D \leq 11$.

Do not say

Quoting 2^{-240} as the security level. It is one Schwartz–Zippel term; the system rests on discrete log at about 126 bits.

Source: *Halo 2 Guide*, "Why checking one random point is a proof"; *Math Guide*, "The Schwartz–Zippel lemma"

Board 100 — Committing to a polynomial: the problem

all four checks

Write

1. Needed: a short commitment P to f, and a short proof that $f(x) = v$ at a drawn x for the committed f.
   $P = \mathsf{Commit}(f; r) = \langle \vec{a}, \vec{G} \rangle + [r]\,H \in \mathbb{G}$
2. One Vesta point for 2^{11} coefficients. Binding under discrete log; hiding by $[r]H$; no secret behind $\vec{G}$.
3. Merkle: one entry in $\log_2 d$ hashes. IPA: a combination of all entries in $2\log_2 d$ points.

Say

The random point needs the polynomials fixed first. The argument on the last two boards assumed the polynomials were pinned before z was drawn. So we need a short commitment to f, and a short proof that $f(x) = v$ at a drawn point for the committed f.

Pedersen, vectorised. Commit to the coefficient vector with generators $\vec{G}$ and H hashed into existence: one Vesta point stands for 2^{11} coefficients. Binding under discrete log, hiding by the $[r]H$ term, and no secret behind $\vec{G}$.

The contrast. A Merkle root also commits to a vector, and opens one entry in $\log_2 d$ hashes; the next board opens a linear combination of *all* entries in $2\log_2 d$ points.

Source: *Halo 2 Guide*, "Why the prover cannot wriggle: binding commitments"; *Crypto Guide*, "Polynomial commitment schemes"; "Hashing to a curve point"

Board 101 — Committing to a polynomial: the opening claim

all four checks

Write

$f(x) = \langle \vec{a}, \vec{b}(x) \rangle, \qquad \vec{b}(x) := (1, x, x^2, \ldots, x^{d-1})$

$P + [v]\,U = \langle \vec{a}, \vec{G} \rangle + [r]\,H + [\langle \vec{a}, \vec{b} \rangle]\,U$

1. Sending $\vec{a}$: d elements. Inner-product argument: $2\log_2 d$ points, 22 for $d = 2^{11}$.

Say

Evaluation is an inner product. The value $f(x)$ is the coefficient vector dotted with the public vector of powers of x.

Fold the claimed value into the commitment. With a second hashed generator U, the opening claim becomes one group equation: $P + [v]U$ equals the commitment plus the inner product times U.

The cost. Sending $\vec{a}$ would cost d elements. The inner-product argument — the Bulletproofs family you know from range proofs — proves the inner product in $2\log_2 d$ points: 22 for $d = 2^{11}$.

The contrast. A Merkle proof names one leaf; an IPA opening is one number computed from every coefficient, at a point chosen after they were fixed.

Ask

How many points does the opening cost for $d = 2^{11}$? — $2\log_2 d = 22$.

Source: *Halo 2 Guide*, "The inner-product polynomial commitment"; *Crypto Guide*, "Polynomial commitment schemes"

Board 102 — The inner-product argument: one fold

all four checks

Write

1. Split: $\vec{a} = (\vec{a}_{\text{lo}}, \vec{a}_{\text{hi}})$, likewise $\vec{G}$ and $\vec{b}$.
   $L := \langle \vec{a}_{\text{lo}}, \vec{G}_{\text{hi}} \rangle + [\ell]\, H + [\langle \vec{a}_{\text{lo}}, \vec{b}_{\text{hi}} \rangle]\, U$
   $R := \langle \vec{a}_{\text{hi}}, \vec{G}_{\text{lo}} \rangle + [r_R]\, H + [\langle \vec{a}_{\text{hi}}, \vec{b}_{\text{lo}} \rangle]\, U$
2. Challenge u; fold:
   $\vec{a}' := u\,\vec{a}_{\text{lo}} + u^{-1}\vec{a}_{\text{hi}}, \quad \vec{G}' := u^{-1}\vec{G}_{\text{lo}} + u\,\vec{G}_{\text{hi}}, \quad \vec{b}' := u^{-1}\vec{b}_{\text{lo}} + u\,\vec{b}_{\text{hi}}$
   $\langle \vec{a}', \vec{G}' \rangle = \langle \vec{a}, \vec{G} \rangle + u^2 \langle \vec{a}_{\text{lo}}, \vec{G}_{\text{hi}} \rangle + u^{-2} \langle \vec{a}_{\text{hi}}, \vec{G}_{\text{lo}} \rangle$
   $P + [v]\, U + [u^2]\, L + [u^{-2}]\, R = \langle \vec{a}', \vec{G}' \rangle + [r + u^2\ell + u^{-2} r_R]\, H + [\langle \vec{a}', \vec{b}' \rangle]\, U$

Say

Halve everything. Split each vector into a low and a high half and publish the two cross terms L and R: low $\vec{a}$ against high $\vec{G}$ and $\vec{b}$, and the reverse, each with its own blinder.

Fold with a challenge. Draw u and fold, both parties in step: the prover folds $\vec{a}$ with weights (u, u^{-1}), the verifier folds $\vec{G}$ and $\vec{b}$ with the inverse weights.

Multiply out. The folded inner product is the old one plus u^2 times one cross term plus u^{-2} times the other, and the same for $\vec{b}$. So the old claim plus $[u^2]L + [u^{-2}]R$ is the same claim at half the length. The verifier updates $\vec{G}'$ and $\vec{b}'$ by public arithmetic and never sees $\vec{a}$.

The contrast. This is the textbook (Bulletproofs) form; a Bulletproofs range proof folds its vectors by exactly this move.

Do not say

Presenting $[u^2]L + [u^{-2}]R$ as the deployed accumulation; it is the textbook form. The deployed rescaling comes two boards on.

Source: *Halo 2 Guide*, "The inner-product polynomial commitment", Construction "IPA-based polynomial commitment"

Board 103 — The inner-product argument: the recursion

all four checks

Write

1. After $k = \log_2 d$ rounds every vector has length 1.
2. Proof: $2k$ points (L_j, R_j), final scalar a, blinder $r^* = r + \sum_j (u_j^2 \ell_j + u_j^{-2} r_j)$.
   $P + [v]\, U + \sum_{j=1}^{k} ([u_j^2]\, L_j + [u_j^{-2}]\, R_j) \stackrel{?}{=} [a]\, G^{(0)} + [r^*]\, H + [a \cdot b^{(0)}]\, U$
   $b^{(0)} = \prod_\ell (u_\ell^{-1} + u_\ell\, x^{2^{\ell-1}})$ $\quad O(\log d)$ field operations
   $G^{(0)} = \langle \vec{s}, \vec{G} \rangle$ $\quad$ an MSM of length d

Say

Fold until one element remains. After $\log_2 d$ rounds every vector has length one. The proof is the $2k$ cross terms, the final scalar a, and the aggregated blinder. The verifier accumulates the cross terms into the commitment and checks one group equation.

Two verifier costs. The scalar $b^{(0)}$ is a product of $\log d$ factors — cheap. The point $G^{(0)}$ is a multi-scalar multiplication of length d — the one linear step. The node shares it across a batch; recursion would defer it into an accumulator, which is not deployed.

Why it binds. Two openings of one commitment would exhibit a non-trivial discrete-log relation among $\vec{G}, H$ on Vesta; an extractor rewinds three challenges per round.

The contrast. A Bitcoin signature check costs a fixed handful of group operations; an IPA opening is logarithmic in size but ends in a circuit-sized wall of group arithmetic.

Do not say

Claiming recursion or accumulation is deployed. The node verifies each proof directly and shares the MSM across a batch.

Source: *Halo 2 Guide*, "The inner-product polynomial commitment"; "Accumulation and the Halo trick"; *Crypto Guide*, "Polynomial commitment schemes"

Board 104 — What the fold hides, and the deployed normalisation

all four checks

Write

1. Folding alone is not zero knowledge: the final a is a challenge-dependent linear form in the coefficients.
2. Deployed: mask f by a random polynomial vanishing at the opening point, commit as S, draw two more challenges, then fold.
3. Deployed weights: $(1, u_j^{-1})$ on coefficients, $(1, u_j)$ on $\vec{b}$ and $\vec{G}$; accumulate $[u_j^{-1}]\,L_j + [u_j]\,R_j$.
4. Same $2k$ points, plus S and two final scalars.

Say

Folding alone leaks. The final scalar a is a linear form in the coefficients whose weights the challenges fix; blinding P and the round points hides those points, not a.

The deployed fix. The opening first masks f by a random polynomial vanishing at the opening point, commits to the mask as S, draws two further challenges, and only then folds.

The deployed rescaling. The fold weights are $(1, u_j^{-1})$ on the coefficients and $(1, u_j)$ on $\vec{b}$ and $\vec{G}$, and the verifier accumulates $[u_j^{-1}]L_j + [u_j]R_j$. The two conventions are related by per-round rescaling; the previous board's form is the textbook one, and the deployed proof carries the same $2k$ points plus S and two final scalars.

The contrast. A Bitcoin signature exposes its public key by design; here even the last scalar of an opening must be masked.

Source: *Halo 2 Guide*, "The inner-product polynomial commitment", Remarks "Folding alone is not zero knowledge", "The deployed IPA normalisation"

Board 105 — The fold by hand: two rounds

all four checks

Draw

Write the setup line first, then the table, one round per row:

round	L_j	R_j	u_j	$\vec{a}'$	$\vec{b}'$	$\vec{G}'$	fold check
$j = 2$ $(\ell_2, r_2 = 44, 16)$	$(74, 6)$	$(38, 72)$	52	$(58, 61)$	$(70, 42)$	$(54, 40), (30, 71)$	$(72, 23)$
$j = 1$ $(\ell_1, r_1 = 12, 37)$	$(28, 25)$	$(39, 25)$	20	(33)	(19)	$(12, 25)$	$(54, 39)$

Write

1. Toy: $a(X) = 90 + 61X + 22X^2 + 24X^3$ opened at $z = 20$, so $v = 59$. Group: $y^2 = x^3 + 3$ over $\mathbb{F}_{79}$, 97 points.
   $\vec{a} = (90, 61, 22, 24)$, $\vec{b} = (1, 20, 12, 46)$, $r = 10$, $H = (78, 9)$, $U = (53, 26)$
   $\vec{G} = ((12, 25), (17, 27), (71, 26), (49, 10))$, $P = \langle \vec{a}, \vec{G} \rangle + [r]H = (39, 25)$
2. Fold check per row: $\langle \vec{a}', \vec{G}' \rangle + [r^*]H + [\langle \vec{a}', \vec{b}' \rangle]U = P + [v]U + [u^2]L + [u^{-2}]R$.

Say

The Halo 2 Guide's toy opening. The advice polynomial $a(X)$ of the running example is opened at the verifier's point $z = 20$, so the claimed value is 59. The group is the curve $y^2 = x^3 + 3$ over $\mathbb{F}_{79}$, with 97 points, so its scalars are in $\mathbb{F}_{97}$. Every discrete log on it is a short search: only the fold's algebra shows.

Round two, then round one. With four coefficients there are two rounds. Each row shows the cross terms, the challenge, the three folded vectors, and the fold check: the folded claim on the left equals the accumulated commitment on the right — $(72, 23)$, then $(54, 39)$.

The one thing to keep. Each row's check is exactly the one-fold identity from three boards back, instantiated.

Do not say

Opening the statement polynomial at the witness 3: the proof opens a column polynomial at the verifier's point. Claiming the toy is binding or hiding: on 97 points every discrete log is a short search.

Source: *Halo 2 Guide*, "The inner-product polynomial commitment"

Board 106 — The fold by hand: the final check

all four checks

Write

1. Prover sends $a = 33$ and $r^* = 40$.

$\vec{s} = (u_1^{-1}u_2^{-1},\ u_1u_2^{-1},\ u_1^{-1}u_2,\ u_1u_2) = (79, 75, 22, 70), \qquad G^{(0)} = \langle \vec{s}, \vec{G} \rangle = (12, 25)$

$b^{(0)} = g(z; u_1, u_2) = (u_1^{-1} + u_1 z)(u_2^{-1} + u_2 z^2) = 19$

$P + [v]U + [u_1^2]L_1 + [u_1^{-2}]R_1 + [u_2^2]L_2 + [u_2^{-2}]R_2 = (54, 39)$

$[a]\, G^{(0)} + [r^*]\, H + [a \cdot b^{(0)}]\, U = (54, 39)$

Say

The verifier works from the challenges alone. It builds the weight vector $\vec{s}$ from products of the challenges and their inverses, takes the inner product with $\vec{G}$ to get $G^{(0)} = (12, 25)$, and evaluates the folded b polynomial to get $b^{(0)} = 19$.

Compare the two sides. The accumulated commitment comes to $(54, 39)$; the prover's claimed side comes to $(54, 39)$. Equal: accepted.

What travelled. Four coefficients were never sent; four points and two scalars were.

The contrast, and the caveat. On 97 points every discrete log is a short search, so nothing here is hidden or binding; on Vesta the same equations are, and $G^{(0)}$ is a length-2^{11} multi-scalar multiplication.

Source: *Halo 2 Guide*, "The inner-product polynomial commitment"

Board 107 — Fields and curves in the deployed circuit

all four checks

Write

1. Native field: $\mathbb{F}_{p_{\mathsf{Pallas}}}$, the *base* field of Pallas.
2. Native points: cm, rk, cv, ak, $\mathsf{pk_d}$ — coordinates in $\mathbb{F}_{p_{\mathsf{Pallas}}}$, group law by native arithmetic.
3. Column polynomials committed on Vesta, whose *scalar* field is exactly $\mathbb{F}_{p_{\mathsf{Pallas}}}$.

Say

The circuit computes in Pallas's base field. The points the statement handles — the note commitment, the re-randomised key, the value commitment, the spend key, the diversified key — have coordinates there, so the group law is native arithmetic, not emulated.

The commitments live on the other curve. The column polynomials are committed on Vesta, whose scalar field is exactly $\mathbb{F}_{p_{\mathsf{Pallas}}}$: the coefficients are the circuit's own field elements.

The contrast. Bitcoin's secp256k1 is one curve doing one job: its validator verifies signatures, never circuits. And its base field, as the two-adicity board shows, cannot even host the prover's evaluation domain.

Source: *Ironwood Guide*, "The Action circuit and the Halo 2 proof"; *Math Guide*, "Base fields, scalar fields, and the Pasta cycle"

Board 108 — The Pasta cycle, once

all four checks

Write

1. Pallas over $\mathbb{F}_{p_{\mathsf{Pallas}}}$ has order p_{Vesta}; Vesta over $\mathbb{F}_{p_{\mathsf{Vesta}}}$ has order p_{Pallas}.
2. Vesta-committed proof: Vesta points, coordinates in $\mathbb{F}_{p_{\mathsf{Vesta}}}$ → verified natively by a circuit over $\mathbb{F}_{p_{\mathsf{Vesta}}}$, committed on Pallas.
3. Its proof: Pallas points, coordinates in $\mathbb{F}_{p_{\mathsf{Pallas}}}$ → verified natively by a circuit over $\mathbb{F}_{p_{\mathsf{Pallas}}}$, committed on Vesta.
4. Not deployed: the node verifies each bundle's proof directly and batches the deferred MSMs.

Say

Each curve's scalar field is the other's base field. That is the whole definition of the cycle.

Why that lets a verifier be a circuit. A Vesta-committed proof is Vesta points with coordinates in Vesta's base field, which is Pallas's scalar field; the circuit that verifies it natively is one over that field, committed on Pallas. That circuit's proof is Pallas points with coordinates in $\mathbb{F}_{p_{\mathsf{Pallas}}}$, verified natively by a circuit over $\mathbb{F}_{p_{\mathsf{Pallas}}}$, committed on Vesta. Proofs alternate between the two curves, each verifier a circuit, so one proof can attest to the previous one.

Not deployed. The node verifies each bundle's proof directly and batches the deferred multi-scalar multiplications.

The contrast. Bitcoin needs no such pair; a signature is verified by arithmetic, not by another proof.

Do not say

Presenting recursion as deployed. Say the cycle once, then say "batched".

Source: *Halo 2 Guide*, "A cycle of curves for efficient native recursion"; "Accumulation and the Halo trick"; *Math Guide*, "Base fields, scalar fields, and the Pasta cycle"

Board 109 — Why not Bitcoin's curve: two-adicity

all four checks

Draw

Write the table, one field per row:

field	bits	two-adicity of $p-1$	largest domain
$\mathbb{F}_{p_{\mathsf{Pallas}}}$ (Pallas base)	255	32	2^{32}
$\mathbb{F}_{p_{\mathsf{Vesta}}}$ (Vesta base)	255	32	2^{32}
secp256k1 base field	256	1	2
secp256k1 scalar field	256	6	64

Write

1. Interpolating over $n = 2^{11}$ rows needs a primitive 2^{11}-th root of unity: $2^{11} \mid p-1$.
2. Quotient FFTs run on $8n = 2^{14}$ points at the deployed gate degree.
3. Two-adicity: the largest k with $2^k \mid p-1$.
4. Bitcoin's prime $p = 2^{256} - 2^{32} - 977$: $p-1$ is twice an odd number, a domain of two points.

Say

Interpolation needs roots of unity. Naming 2^{11} rows by roots of unity needs a primitive 2^{11}-th root, so 2^{11} must divide $p-1$; the prover's quotient FFTs run on 2^{14} points. The largest power of two dividing $p-1$ is the field's two-adicity.

The Pasta primes were designed for this. Two-adicity exactly 32 in both, so domains of every size up to 2^{32}.

Bitcoin's prime was not. The secp256k1 curve buys its implementation speed from $a = 0$; its prime $2^{256} - 2^{32} - 977$ was not chosen for evaluation domains: $p-1$ is twice an odd number, so the largest domain has two points. The scalar field manages 64.

Attribution. The Pasta two-adicity is the Math Guide's, the prime values the protocol specification's ("Pallas and Vesta"); the secp256k1 rows are our own computation, not a volume figure.

Ask

Could secp256k1's base field host even the four-row toy? — No: two-adicity 1, largest domain two points.

Source: *Math Guide*, "Pallas and Vesta assembled", Remark "Two-adicity and FFT-friendly primes"; *Crypto Guide*, "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"; *Halo 2 Guide*, "The complete Halo 2 protocol"

Board 110 — Trust: what a ceremony is, and why there is none

conserved

Write

$pp = ([\tau^0]G, [\tau^1]G, \dots, [\tau^d]G, \dots)$, τ destroyed

1. Whoever keeps τ forges: for any C, z, y,
   $\pi := [(\tau - z)^{-1}]\,(C - [y]G)$ passes, no polynomial behind it — the toxic waste.
2. Groth16: secrets per circuit. MPC: safe if *one* participant destroyed their share. The Sapling pool still rests on that.
3. Bound (ZIP-209): each pool's chain value balance is public and may not go negative ⇒ counterfeit capped at the pool's exit.
4. Orchard, Ironwood: IPA generators hashed from a public domain tag; nothing to leak, nothing to destroy.

Say

Pairing SNARKs bake a secret into the parameters. The pairing-based commitment (KZG) shows the shape: the parameters are powers of a secret τ in the group, and τ must be destroyed. Whoever keeps it forges an opening for any commitment, point and value — with no polynomial behind it. That is the toxic waste. Groth16's secrets are per circuit.

The ceremony, and what still rests on it. A multiparty ceremony keeps the waste unknown provided one participant destroyed their share. The Sapling pool still rests on that assumption.

The bound. Under ZIP-209 each pool's chain value balance is public and may not go negative, so a forged Sapling proof creates value only inside the Sapling pool and can withdraw at most what that pool publicly holds — the turnstile, Part H.

Orchard and Ironwood. The IPA generators are hashed from a public domain tag: nothing to leak, nothing to destroy; binding rests on discrete log alone. Bitcoin never had a setup; Sapling has one; Orchard returns to none.

Do not say

"No ceremony" without the Sapling caveat: the Sapling pool still rests on its MPC. "The pool is auditable": the balance is public, but counterfeit inside the pool is undetectable; the bound acts at the exit.

Source: *Crypto Guide*, "Polynomial commitment schemes"; "SNARKs: succinct non-interactive arguments of knowledge"; *Halo 2 Guide*, "The "Halo" in Halo 2: deferring the expensive check"; *Ironwood Guide*, "Chain state and pool rules"

Board 111 — Three generations

all four checks

Draw

Three columns, Sprout, Sapling, Orchard and Ironwood; write the rows top to bottom:

	Sprout	Sapling	Orchard, Ironwood
SNARK	BCTV14 (original)	Groth16	Halo 2
curve	BN254	BLS12-381, Jubjub in-circuit	Pallas, Vesta
setup	six-party MPC	large MPC	none
hash in circuit	SHA-256	Pedersen	Sinsemilla
proof	296 B	192 B	2720 + 2272a B
verification	constant	three pairings	$O(\log n)$ + shared MSM

Write

1. Sprout: BCTV14 soundness flaw found after deployment; re-proved with Groth16, deprecated.
2. Sapling: Groth16 over BLS12-381, Jubjub inside the circuit.
3. Orchard: proof grows because transparency costs $2 \log_2 n$ points.

Say

Sprout. A soundness flaw in BCTV14, found after deployment, would have allowed undetected counterfeiting; Sprout was re-proved with Groth16 and deprecated.

Sapling. Groth16 over BLS12-381, Jubjub inside the circuit: a far faster prover, a per-circuit ceremony, no curve cycle.

Orchard and Ironwood. Halo 2 on Pallas and Vesta, no setup. The proof grew because transparency costs $2 \log_2 n$ points.

The contrast. Bitcoin changed its signature scheme once, by opt-in; Zcash has carried shielded payments through three proof systems, each with its own pool — the third with two, Orchard and Ironwood, on one circuit.

Do not say

Years. "A few hundred bytes": the sizes are 296 B, 192 B, and $2720 + 2272a$ B, with units. "No ceremony" as a blanket statement: Sapling's still stands.

Source: *Ironwood Guide*, "The Action circuit and the Halo 2 proof"; "The transaction format"; protocol specification, "Zero-Knowledge Proving Systems"

Board 112 — The Orchard circuit by the numbers

all four checks

Draw

Two columns, label then value; write the rows top to bottom:

rows	$2^{11} = 2048$; one fixed circuit, every Action an instance
columns	ten advice, fixed columns, a Sinsemilla lookup table
proof per bundle	$2720 + 2272\,a$ B: 4992 for one Action, 7264 for Alice's two
of which IPA	22 pair points and one blinding commitment, 32 B each
proving	a few seconds on commodity hardware
verifying	$O(\log n)$ per proof, plus one MSM shared across a batch

Write

1. One aggregate proof per bundle: per-Action term = that Action's column commitments and evaluations; constant term = the shared quotient, multipoint and IPA data.
2. Spend-auth signature: 64 B. One-Action proof: 78 signatures' worth.

Say

One circuit, one proof per bundle. The circuit has 2^{11} rows and ten advice columns; every Action is an instance of it. One aggregate proof covers all of a bundle's Actions: the per-Action term is that Action's column commitments and evaluations, the constant term the shared quotient, multipoint and IPA data — 4992 bytes for one Action, 7264 for Alice's two.

Where the bytes go. Of the constant term, the IPA is 22 pair points and one blinding commitment at 32 bytes each. A spend-auth signature is 64 bytes; a one-Action proof is 78 signatures' worth.

Costs. Proving takes a few seconds on commodity hardware; verifying is logarithmic per proof plus one MSM shared across a batch.

The contrast. Bitcoin's witness cost is per input and grows with the script it satisfies; here it is per bundle, logarithmic in the circuit, and independent of the anonymity set.

Do not say

"A few hundred bytes": it is 4992 B for one Action. Millisecond verification figures: the deck states only $O(\log n)$ plus the shared MSM. "A few seconds" is the only timing phrase.

Source: *Ironwood Guide*, "The Action circuit and the Halo 2 proof"; "The transaction format"; *Halo 2 Guide*, "The complete Halo 2 protocol"

Board 113 — Five gadget families, three circuit versions

all four checks

Write

1. ECC: $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$ (A6), $\mathsf{pk_d} = [\mathsf{ivk}]\, \mathsf{g_d}$ (A7), cv (A4), the nullifier's point (A5).
2. Sinsemilla: generator table as a lookup; per 10-bit chunk one lookup and two incomplete additions; NoteCommit (A1, A2), $\mathsf{Commit}^{\mathsf{ivk}}$ (A7).
3. Merkle path: 32 layer-tagged hashes, a conditional swap per layer by one bit of the position, root = anchor gated by v_{old} (A3).
4. Poseidon: the nullifier PRF; only non-linearity $x \mapsto x^5$ (A5).
5. Decomposition: $v \in [0, 2^{64})$ (typing) and the sign bit (A4); canonicity of witnessed encodings.
6. Three versions: original, a correction, the Ironwood circuit with the cross-address input. Verifying key selected by branch.

Say

Five families carry the ten clauses. The Pallas group law does the key re-randomisation, the diversified key, the value commitment and the nullifier's point. Sinsemilla, with its generator table as a lookup, carries both note commitments and the ivk commitment. The Merkle gadget walks 32 layer-tagged hashes with a conditional swap per layer, root equal to the anchor, gated by v_{old}. Poseidon is the nullifier PRF, its only non-linearity $x \mapsto x^5$. Decomposition does the 64-bit range check, the sign bit, and canonicity of witnessed encodings — distinct from the node's parsing of public encodings, Part E.

Three versions, one key per branch. The original circuit, a correction, and the Ironwood circuit with the cross-address input; consensus selects the verifying key by branch, so every Action in either pool now verifies against the Ironwood key.

The contrast. Bitcoin's validator runs whatever script an output declares; this one runs a verifier keyed to one fixed circuit.

Do not say

Multiplicative re-randomisation: it is $\mathsf{ak} + [\alpha]\, G$, never $[\alpha]\, \mathsf{ak}$. The position as ρ: the position is a private witness of the Merkle gadget and never enters the nullifier.

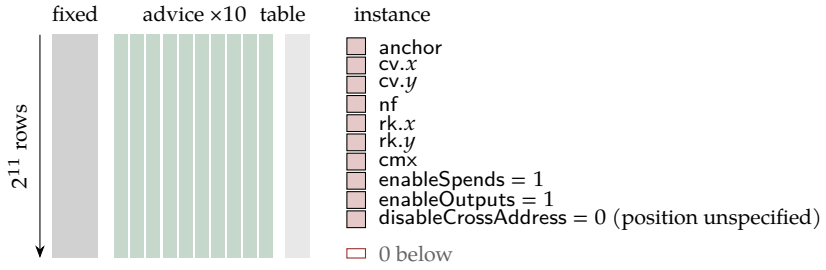
Source: *Ironwood Guide*, "The Action circuit and the Halo 2 proof"; *Halo 2 Guide*, "From statement to circuit: arithmetisation in practice"

Board 114 — Alice's Action as an instance

all four checks

Draw

1. A tall grid: one grey column at the left, labelled *fixed*; ten thin green columns, labelled *advice* ×10; a light grey column, labelled *table*.
2. A vertical arrow down the left edge, labelled 2^{11} rows.
3. To the right, one narrow column of ten red cells, labelled *instance*, with one small empty cell at the bottom marked "0 below".
4. Label the ten cells top to bottom: anchor, cv.x, cv.y, nf, rk.x, rk.y, cmx, enableSpends = 1, enableOutputs = 1, disableCrossAddress = 0.



Write

1. Instance (Action 1: 2 ZEC → Bob 1.5): eight public inputs, ten field elements; points by coordinates; flags as the Ironwood component sets them.
2. Advice: both notes; 32 siblings and the position; ak, nk, $\mathsf{r_{ivk}}$, ivk; α; rcv; (magnitude, sign) = (50 000 000, +1).
3. Copy constraints tie each instance cell to the advice cell holding the same value.

Say

The public column, for Alice's first Action. Eight named public inputs make ten field elements: the two points enter by their coordinates, and the three flags as the Ironwood component sets them. Below the tenth cell the column is zero.

The advice cells hold everything secret. Both notes, the 32 siblings and the position, the keys ak, nk, $\mathsf{r_{ivk}}$, ivk, the re-randomiser α, the value-commitment trapdoor rcv, and the value as magnitude and sign: 50 000 000 and +1.

How the public column binds. Copy constraints tie each instance cell to the advice cell holding the same value; the verifier computes the instance commitment itself, so a proof for other values fails the opening.

The contrast. Bitcoin's witness — scriptSig or witness field — is in the clear; here the witness fills advice cells nobody reads.

Ask

Checkpoint: what could a holder of the Sapling ceremony's toxic waste do, what stops the analogue in Orchard, and what bounds the damage?

- Forge accepting Sapling proofs of false statements: counterfeit inside the Sapling pool, invisible to every verifier. Not decrypt notes — soundness and zero knowledge are separate guarantees.
- In Orchard there is no trapdoor to hold: the generators are hashed, and binding reduces to discrete log.
- Under ZIP-209 each pool's chain value balance is public, and a block that would drive it negative is invalid — counterfeit is capped at the pool's exit.

Do not say

"The proof hides sender and receiver": the instance tuple is public and unlinkable for reasons outside π. Answering the checkpoint with "decrypt notes": toxic waste breaks soundness, not zero knowledge. "The pool is auditable": the bound acts at the exit, not inside.

Source: *Ironwood Guide*, "The Action statement"; *Halo 2 Guide*, "Instance columns and binding the public statement"; checkpoint: *Crypto Guide*, "SNARKs: succinct non-interactive arguments of knowledge"; *Ironwood Guide*, "Chain state and pool rules"

8 Part F — Delivery and light clients

Board 115 — The note plaintext: what travels to Bob

exists

Write

1. np := (0x03, d, v, rseed, memo)
2. $1 + 11 + 8 + 32 + 512 = 564$ bytes
3. rseed → esk $\quad (\mathsf{PRFexpand}_{\mathsf{rseed}}(\mathtt{0x04} \,\|\, \rho))$
4. rseed → ψ $\quad (\mathsf{PRFexpand}_{\mathsf{rseed}}(\mathtt{0x09} \,\|\, \rho))$
5. rseed → rcm $\quad$ (next board)
6. $\rho := \mathsf{nf}_{\mathrm{old}}$

Say

The chain stores only cmx**; the note itself must still reach Bob.** In Bitcoin the output is the note: amount and script sit on chain and the recipient's wallet reads them there. Here the chain holds a commitment, so Alice has to deliver the opening herself. What she sends is the note plaintext.

The lead byte names the format. Byte 0x03 is the Ironwood note, the only value the Ironwood pool admits.

The 32-byte seed is the note's whole randomness. Three PRFexpand calls keyed by rseed, each under its own domain byte, yield esk, ψ and rcm (next board); with $\rho := \mathsf{nf}_{\text{old}}$ the seed is bound to its Action.

The memo never appears in the clear. It is 512 bytes of sender-chosen data, encrypted with the rest.

Bitcoin contrast. An output shows its amount and script to everyone, and an OP_RETURN note is public.

Do not say

- Calling ρ a tree position. It is the per-note uniqueness seed, here the nullifier of the note spent in the same Action; the leaf index never enters it.

Source: *Ironwood Guide*, "The note plaintext"; "The note seed: Ironwood derivations (lead byte 0x03)"; protocol specification, "Note Plaintexts and Memo Fields"

Board 116 — The Ironwood trapdoor: one hash over every note field

exists

Write

1. Ironwood pool (0x03): $\mathsf{rcm} := \mathrm{ToScalar}(\mathsf{PRFexpand}_{\mathsf{rseed}}(\mathsf{pre}_{\mathsf{rcm}}))$
2. $\mathsf{pre}_{\mathsf{rcm}} := \mathtt{0x0B} \,\|\, \mathsf{g_d}^{\star} \,\|\, \mathsf{pk_d}^{\star} \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{256}(\rho) \,\|\, \mathrm{LE}_{256}(\psi)$
3. $1 + 32 + 32 + 8 + 32 + 32 = 137$ bytes
4. Orchard pool (0x02): $\mathsf{rcm} := \mathrm{ToScalar}(\mathsf{PRFexpand}_{\mathsf{rseed}}(\mathtt{0x05} \,\|\, \mathrm{LE}_{256}(\rho)))$
5. $1 + 32 = 33$ bytes

Say

This is the one note-level difference between the pools: how rcm **leaves the seed.** Everything else about the note is shared.

Every note field enters the Ironwood trapdoor. The preimage is 137 bytes against 33, so rcm is derived last, after ψ. The commitment itself is unchanged: same NoteCommit, same tree construction, each pool its own instance (ZIP-229).

Only the trapdoor differs. Both pools' notes carry esk and ψ byte-identically, and the lead byte says which derivation applies.

Bitcoin contrast. A Bitcoin output has no trapdoor at all: there is nothing hidden to bind.

Source: *Ironwood Guide*, "The note seed: Ironwood derivations (lead byte 0x03)"; protocol specification, "Sending Notes (Orchard)"

Board 117 — Why the detour through BLAKE2b: post-quantum binding

exists

Write

1. Sinsemilla binding = discrete log → falls to a quantum computer
2. PRFexpand as a random oracle; every field enters $\mathsf{pre}_{\mathsf{rcm}}$
3. derivation + NoteCommit = post-quantum binding
4. *provided a verifier checks the derivation* — consensus does for coinbase outputs only
5. Ironwood note: recoverable; Orchard-/Sapling-pool note: not

Say

The commitment's binding is a discrete-log assumption. Sinsemilla collision resistance is what binds NoteCommit, and a large quantum computer breaks it (Part I). Once it falls, NoteCommit alone binds nothing.

The detour puts every field through a random oracle. Model PRFexpand as one. Every field enters $\mathsf{pre}_{\mathsf{rcm}}$, so any change to a committed note re-randomises rcm; landing two honestly derived note–trapdoor pairs on one commitment is a collision event in the oracle, with no group structure for Shor.

The composite is post-quantum binding — provided a verifier checks the derivation. No deployed Action clause does; the claim is conditional on that future check. Say the condition every time.

This is what makes an Ironwood note recoverable, and no Orchard- or Sapling-pool note. Parts H and I return to it.

Bitcoin contrast. Nothing in a Bitcoin output is hidden, so nothing must stay bound after a break; here the hidden note must.

Source: *Ironwood Guide*, "The note seed: Ironwood derivations (lead byte `0x03`)", Remark "Purpose of the hashed trapdoor"; "The Action statement", Remark "What the statement does not check"

Board 118 — Sealing the note: one-shot Diffie–Hellman, then an AEAD

exists

Write

1. Bob's address $(d, \mathsf{pk_d})$, $\quad \mathsf{g_d} = \mathsf{GroupHash}(d)$, $\quad \mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$
2. $\mathsf{epk} := [\mathsf{esk}]\,\mathsf{g_d}$
3. $\mathsf{s} := [\mathsf{esk}]\,\mathsf{pk_d}$
4. $K_{\text{enc}} := \mathsf{KDF}(\mathsf{s},\ \mathsf{epk})$
5. $C_{\text{enc}} := \mathsf{Enc}_{K_{\text{enc}}}(\mathsf{np})$ $\quad 564 + 16$-byte tag $= 580$ bytes
6. $[\mathsf{ivk}]\,\mathsf{epk} = [\mathsf{ivk}][\mathsf{esk}]\,\mathsf{g_d} = [\mathsf{esk}]\,\mathsf{pk_d} = \mathsf{s}$

Say

Alice holds Bob's address. The address is $(d, \mathsf{pk_d})$ with $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$ and $\mathsf{g_d} = \mathsf{GroupHash}(d)$. With esk drawn from rseed she runs a key agreement to the diversified base.

The primitives. The KDF is BLAKE2b-256 over the encodings of s and epk; the AEAD is ChaCha20-Poly1305 under a fixed all-zero nonce and empty associated data: a fresh key for every note.

Why Bob can open it. The holder of ivk computes $[\mathsf{ivk}]\,\mathsf{epk} = [\mathsf{ivk}][\mathsf{esk}]\,\mathsf{g_d} = [\mathsf{esk}]\,\mathsf{pk_d} = \mathsf{s}$, and so does the sender, who knows esk; either recovers K_{enc} from the published pair $(\mathsf{epk}, C_{\text{enc}})$.

Bitcoin contrast. BIP-352 runs the same ECDH with the input keys standing in for esk — no epk is published — to derive an output key the chain then shows in the clear; here the shared secret encrypts the note, amount and memo included.

Source: *Ironwood Guide*, "Encryption to the recipient"; protocol specification, "In-band secret distribution (Sapling and Orchard)"

Board 119 — The second ciphertext: the sender's own copy

exists

Write

1. $\mathsf{ock} := \text{BLAKE2b-256}(\mathsf{ovk} \,\|\, \mathsf{cv} \,\|\, \mathsf{cmx} \,\|\, \mathsf{epk})$
2. $C_{\text{out}} := \mathsf{Enc}_{\mathsf{ock}}(\mathsf{pk_d},\ \mathsf{esk})$ $\quad 64 + 16 = 80$ bytes
3. ovk holder: $\mathsf{s} = [\mathsf{esk}]\,\mathsf{pk_d} \rightarrow$ reads the note as Bob does
4. $\mathsf{ovk} = \perp \rightarrow$ random bytes under a random key
5. Action publishes $(\mathsf{epk}, C_{\text{enc}}, C_{\text{out}})$ beside cv, nf, rk, cmx

Say

One ciphertext serves Bob; a second can serve Alice's books. Given an outgoing viewing key, the pair $(\mathsf{pk_d}, \mathsf{esk})$ is sealed under the outgoing cipher key.

Eighty bytes buy a sent history. From $(\mathsf{pk_d}, \mathsf{esk})$ the holder of ovk re-derives $\mathsf{s} = [\mathsf{esk}]\,\mathsf{pk_d}$ and reads the note as Bob does.

The key pins the copy to its Action. Binding cv, cmx, epk into ock ties C_{out} to its own Action; with $\mathsf{ovk} = \perp$ the sender seals random bytes under a random key instead, and no outgoing key ever opens it.

The node never opens any of the three. The Action publishes $(\mathsf{epk}, C_{\text{enc}}, C_{\text{out}})$ beside cv, nf, rk, cmx; no clause of the Action statement mentions them.

Bitcoin contrast. A Bitcoin sender needs no copy: the chain is the copy. Here a wallet's sent history exists only if it kept a key that reads it.

Do not say

- "The proof hides sender and receiver." The proof says nothing about the three ciphertexts; no clause mentions them.
- Deriving ovk from ivk. The outgoing key derives from fvk beside ivk (Part D), not from the incoming key.

Source: *Ironwood Guide*, "The outgoing ciphertext"; "The note plaintext"; protocol specification, "Decryption using an Outgoing Viewing Key (Sapling and Orchard)"

Board 120 — Finding your money: detection is successful decryption

exists

Write

1. for *every* Orchard-protocol output:
2. $\mathsf{s} := [\mathsf{ivk}]\,\mathsf{epk}, \quad K_{\text{enc}} := \mathsf{KDF}(\mathsf{s}, \mathsf{epk}), \quad \mathsf{np} := \mathsf{Dec}_{K_{\text{enc}}}(C_{\text{enc}})$
3. tag rejects a wrong key → an open is a *candidate*
4. check 1: esk from rseed, $[\mathsf{esk}]\,\mathsf{g_d} = \mathsf{epk}$
5. check 2: note from (d, v, rseed) and $\rho := \mathsf{nf}_{\text{old}}$, $\mathsf{Extract}(\mathsf{NoteCommit}_{\mathsf{rcm}}(\ldots)) = \mathsf{cmx}$
6. cost: one key agreement per output per ivk + every output's bandwidth

Say

Nothing on chain names a recipient. A Bitcoin wallet asks for its addresses' outputs, a lookup over public scripts. Here no field names Bob, so there is nothing to ask for: his wallet trial-decrypts every Orchard-protocol output with ivk.

An open is a candidate, not yet a note. The AEAD tag rejects a wrong key; two re-derivations close the obvious forgeries. The esk from rseed must give $[\mathsf{esk}]\,\mathsf{g_d} = \mathsf{epk}$, and the note rebuilt from (d, v, rseed) and the Action's own $\rho := \mathsf{nf}_{\text{old}}$ must give $\mathsf{Extract}(\mathsf{NoteCommit}_{\mathsf{rcm}}(\ldots)) = \mathsf{cmx}$.

The cost is a count, not a clock. One key agreement per output for each ivk, plus the bandwidth of every output. One ivk scans both pools.

Bitcoin contrast. The Bitcoin wallet's lookup is over public scripts; here the "lookup" is a decryption attempt on every output.

Do not say

- Timing claims. The slide gives the cost as a count of key agreements and bytes; put no seconds on the board.

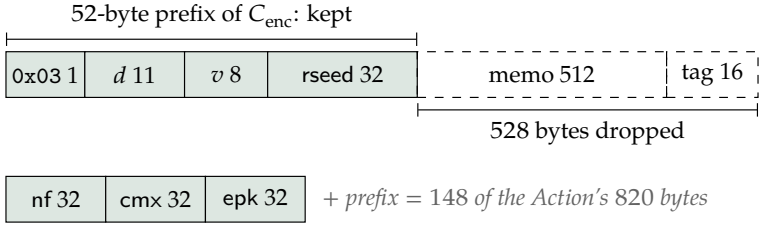
Source: *Ironwood Guide*, "Trial decryption and note acceptance"; *Wallet Guide*, "Compact blocks (ZIP-307)"; protocol specification, "Decryption using an Incoming Viewing Key (Sapling and Orchard)"

Board 121 — Compact blocks (ZIP-307): what a light client downloads

exists, not twice

Draw

1. Top row, left to right, four adjoining solid boxes: "0x03 1", "d 11", "v 8", "rseed 32".
2. Continue the row with two dashed boxes: "memo 512", "tag 16".
3. Bracket the four solid boxes from above: "52-byte prefix of C_{enc}: kept".
4. Bracket the two dashed boxes from below: "528 bytes dropped".
5. Second row, three solid boxes: "nf 32", "cmx 32", "epk 32"; to their right write "+ prefix = 148 of the Action's 820 bytes".



Write

1. compact Action = $\mathsf{nf} \,\|\, \mathsf{cmx} \,\|\, \mathsf{epk} \,\|\, C_{\text{enc}}[0..52)$
2. $3 \cdot 32 + 52 = 148$ of 820 bytes
3. dropped: memo, tag, C_{out}, cv, rk, anchor, proof, signatures
4. per block, per pool: end-of-block tree size

Say

What does a light client need from each block? A compact block keeps only what three tasks need: detect a payment, detect a spend of one's own notes, update witnesses.

Per Action: nf, cmx, epk, and the 52-byte prefix. Dropped: memo, tag, C_{out}, cv, rk, anchor, proof, signatures — 528 bytes of C_{enc} alone. The compact Action is 148 of the Action's 820 bytes. Each block also states its end-of-block tree size per pool.

Bitcoin contrast. An SPV client downloads headers and asks for what matches; this client downloads every compact block in range and asks for no particular output.

Do not say

- "A few hundred bytes" for the Action. The Action is 820 bytes; its compact form is 148.

Source: *Wallet Guide*, "Compact blocks (ZIP-307)"; "The compact block and the compact transaction"; "Compact outputs, spends, and Actions"; *Ironwood Guide*, "Action descriptions and bundles"; "Transaction digests and signatures"

Board 122 — Accepting a note without its tag

exists

Write

1. $\mathsf{np} := \mathsf{ciphertext}[0..52) \oplus \mathsf{ChaCha20}_{K_{\text{enc}}}(\text{nonce} = 0)[64..116)$
2. bare keystream, past block 0 (the tag key's block)
3. accept iff: 1. the 52 bytes parse as a compact note plaintext; 2. recomputed commitment = served cmx; 3. esk from rseed reproduces epk
4. commitment binding stands in for the tag

Say

Truncation at 52 bytes discards the Poly1305 tag. A compact decryption cannot be authenticated the AEAD way, so the wallet applies the bare keystream, past block 0, which the full AEAD reserves for the tag key.

The commitment stands in for the tag. The wallet accepts iff the 52 bytes parse as a compact note plaintext, the recomputed commitment equals the served cmx, and the esk re-derived from rseed reproduces epk. Bob holds neither cv nor the binding signature, so commitment binding is what he relies on: under binding, no sender can make Bob accept a note the chain did not commit to.

Bitcoin contrast. BIP-37 hands each peer a Bloom filter of the wallet's scripts; BIP-157 matches a per-block filter locally, then fetches the matching blocks. Here the "filter" is ivk, applied to every block, and no match is ever requested.

Source: *Wallet Guide*, "Compact trial decryption";
Ironwood Guide, "Trial decryption and note acceptance"

Board 123 — Leaks, in both directions

none — privacy of the scan

Write

1. scan stage: **less** — every compact block in range fetched; server sees range bounds + timing, never a per-block match
2. after detection: **more** — full-transaction fetch names an exact txid (BIP-157: a whole block)
3. ZIP-307 on that fetch: decoys (SHOULD); every transaction of any block touched (SHOULD); tell the user (MUST)
4. deployed libraries: no decoy policy

Say

At the scan stage, less. Every compact block in a range is fetched whether or not anything inside opens; the server sees range bounds and timing, never a per-block match. A BIP-157 client reveals each block it fetches after a filter hit.

After detection, more. The memo and C_{out} live in the full transaction, which compact blocks omit; fetching it names an exact txid, where a BIP-157 client names a whole block.

ZIP-307 attaches three requirements to that fetch. Obscure interest with uninteresting transactions (SHOULD), fetch every transaction of any block touched (SHOULD), tell the user (MUST). The deployed client and server libraries ship no decoy policy.

Structural, not accidental. A client that asks for ranges reveals where it stopped; one that fetches what it detected reveals what it detected.

Source: *Wallet Guide*, "Leakage of range queries"; "Leakage of transaction queries"

Board 124 — Placing the note: a position from public data

exists, not twice

Write

1. pos(output i of tx) = start-of-block size + offset(tx) + i
2. start size = stated end-of-block size − the block's own outputs
3. offset(tx) = outputs of the block's earlier transactions
4. append *every* cmx; mark own
5. spend: every compact nf vs own unspent nullifiers, locally

Say

Trial decryption tells Bob that an output is his; the tree needs to know where. The position is arithmetic on public data, never a replay: the start size is the stated end-of-block size minus the block's own outputs, and the offset counts the outputs of the block's earlier transactions.

The size is load-bearing, not a convenience. The wallet appends every cmx — Bob's and everyone else's — and marks its own; a note without a position has no witness.

A spend of Bob's note is detected the same way. Every compact nullifier is compared against the wallet's own unspent nullifiers, locally.

Bitcoin contrast. An SPV client's branch arrives because its filter (BIP-37), its block fetch (BIP-157), or — on Electrum — the txid itself named its interest. Bob names nothing; his position is his own arithmetic.

Do not say

- Calling ρ a tree position. The position is this arithmetic on public sizes; ρ never enters it, and the leaf index never enters the nullifier.

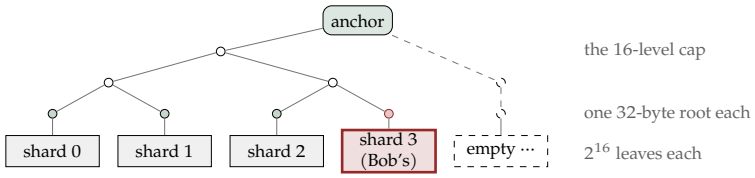
Source: *Wallet Guide*, "Note positions and tree sizes"; *Ironwood Guide*, "Authentication paths"

Board 125 — Shards and the cap: a witness without the whole tree

exists

Draw

1. Bottom row, five boxes left to right: "shard 0", "shard 1", "shard 2", "shard 3 (Bob's)" (thick, red, two lines), "empty ⋯" (dashed).
2. Above each box a small circle, its root; Bob's red, the empty one dashed; one stroke from each box up to its circle.
3. Pair the roots: one circle above shards 0–1, one above 2–3, a dashed one above the empty root; join each pair.
4. One circle above the two pairs; join; a rounded box "anchor" at the top, joined to that circle and dashed to the empty branch.
5. Right margin, level with the rows: "2^{16} leaves each", "one 32-byte root each", "the 16-level cap".



Write

1. depth 32, cut at 16: shards of 2^{16} leaves, at most 2^{16} of them, 16-level cap of shard roots
2. own shard + unfinished tip shard: hashed from leaves
3. every completed shard: one root from the server
4. path: low 16 siblings from leaves, high 16 from roots
5. unbegun shard = per-level constant

Say

Cut the depth-32 tree at half depth. Shards of 2^{16} leaves, at most 2^{16} of them, under a 16-level cap of shard roots.

Bob hashes his own shard and takes the rest as roots. He hashes his own shard from its leaves and takes every completed shard as one root from the server; the unfinished tip shard, like his own, is scanned from leaves. Low 16 siblings come from leaves, high 16 from roots; an unbegun shard is a per-level constant.

Bitcoin contrast. An SPV client is handed one branch by its peer. Bob folds his own from public roots and asks for nothing — asking for one note's path would say which note is his. Only the anchor is ever public.

Source: *Ironwood Guide*, "Authentication paths"; *Wallet Guide*, "The information a wallet retains"

Board 126 — Checkpoints: the tree as of a block

exists

Write

1. checkpoint at height h = last-leaf position as of block h + marks dropped since the previous one
2. uses: fold a path against block h's root; truncate to it on a reorganisation
3. *shared* = present in every pool's tree at that height
4. bounded recent set + durable ones exempt from pruning
5. a block appending nothing need not create one

Say

A checkpoint is the wallet's record of its tree at a block boundary. It holds the position of the last leaf as of that block, plus the marks dropped since the previous one, and is identified by the block's height.

Two uses. Fold a path against that block's root, so a witness can target any checkpointed block; and truncate the tree back to it when a reorganisation strikes.

Shared means present in every pool's tree. When one tree gains a checkpoint at a height, the others gain one too, so a single height names a state of every tree. A bounded recent set is kept, plus durable ones exempt from pruning; a block that appends nothing need not create one.

Bitcoin contrast. Bitcoin Core's checkpoints are hard-coded block hashes inside the node; these are wallet-local tree snapshots that consensus never sees.

Source: *Ironwood Guide*, "Anchors"; *Wallet Guide*, "The information a wallet retains"; "Reorganisation detection and recovery"

Board 127 — The frontier as the scan seed

exists

Write

1. frontier of a tree of size n = rightmost path: last leaf + one sibling hash per filled subtree up to the root
2. wallet birth: frontier of the block *before* the birthday

3. batch seed: chain state of the block before it, checked against the first block's own end-of-block size
4. contradiction → batch mis-sequenced; size contradicted by the block's contents = continuity error → reorganisation path
5. note tree-ready: own shard's block extent + tip range scanned

Say

A frontier is the rightmost path of a tree. The last leaf plus one sibling hash per filled subtree on its way to the root: enough to keep appending, and enough to seed a shard tree.

A wallet is born from the frontier of the block before its birthday. That frontier summarises every earlier commitment, so no block below the birthday is ever replayed.

Every scanned batch is seeded and checked before any tree is touched. The seed is the chain state of the block before the batch, checked against the first block's own end-of-block size; a contradiction rejects the batch as mis-sequenced. A size the block's own contents contradict is the continuity error — the reorganisation path.

A found note is tree-ready before the whole history is. Once its own shard's block extent and the tip range are scanned, it is ready.

Bitcoin contrast. A restored Bitcoin wallet also rescans from its birthday, but needs no tree state: the UTXO set is public, and a peer answers a filter, or an Electrum server answers by address.

Source: *Wallet Guide*, "Frontiers as scan seeds"; "The account birthday"; "Witness readiness"

Board 128 — What the server learns: the schedule

none — privacy of the scan

Draw

Three columns: observable, reveals, mitigation status. Rows in order:

1. peer IP, connection timing — endpoint, activity windows — Tor capability; deployment unknown
2. lowest range start — approximate account birthday — imported seed starts at Sapling activation (ZIP-307)
3. range boundaries — scan state and progress — bucketing proposed, unimplemented

Write

1. keys never travel; requests do
2. ZIP-307's goal: payment-detection privacy against an honest-but-curious proxy; spending out of scope

Say

Keys never travel; requests do. ZIP-307's stated goal is payment-detection privacy against an honest-but-curious proxy; spending is outside its scope.

The schedule is what leaks. The peer address and connection timing give the endpoint and its activity windows; the lowest range start gives an approximate account birthday; the range boundaries give scan state and progress. A Tor capability exists, with deployment unknown; bucketing is proposed and unimplemented; for the birthday, ZIP-307 starts an imported seed's scan at Sapling activation.

Bitcoin contrast. A BIP-157 client shows a peer the same kind of schedule — which filter ranges it pulls, and when — and then every block it fetches on top. Here the content holds and the schedule leaks.

Source: *Wallet Guide*, "Payment-detection privacy"; "Key independence of queries"; "Leakage of range queries"; "Summary of server observables"

Board 129 — What the server learns: the content channels

none — privacy of the scan

Draw

Same three columns. Rows in order:

1. fetch after detection — **transactions of interest** — decoys a SHOULD; no library policy
2. submission via the server — the whole transaction, linked to origin — circuit isolation; deployment unknown
3. transparent-address calls — addresses outright — jitter (one-use addresses only)

Write

1. detection stays on the device
2. three requests carry content, each sent only after the wallet decided to

Say

Detection stays on the device; three kinds of request carry content anyway. Each is sent only after the wallet itself decided to.

The rows. A fetch after detection names the transactions of interest, and decoys are a SHOULD with no library policy. A submission through the server hands it the whole transaction, linked to origin; circuit isolation exists, deployment unknown. Transparent-address calls give addresses outright; jitter covers one-use addresses only.

Bitcoin contrast. A BIP-37 client leaks its addresses up to the false-positive rate, a BIP-157 client its matched blocks; here content leaks only through what the wallet sends after detecting.

Source: *Wallet Guide*, "Key independence of queries"; "Leakage of transaction queries"; "The submission channel"; "Summary of server observables"

Board 130 — Mitigations that ship, and the structural reading

none — privacy of the scan

Write

1. no request carries key material: heights, hashes, txids, transparent addresses, raw transactions only
2. Tor client with circuit isolation: in the wallet library; default routing not determinable
3. jitter: exponential intervals, shuffled order — ephemeral (ZIP-320) address checks only; none for fetch-on-detect
4. bucketing: range start rounded down to a multiple of N — ZIP-307's proposal, unimplemented

Say

The baseline holds. Across the whole service surface no request carries key material — only heights, hashes, txids, transparent addresses, and raw transactions. Trial decryption and nullifier matching run on the device.

Transport. A Tor client with circuit isolation exists in the wallet library, so submission need not share a circuit with scanning; whether any deployed wallet routes through it by default is not determinable from the sources.

Timing. Exponentially distributed intervals in a shuffled order — only for the checks on ephemeral addresses, the one-use transparent addresses of ZIP-320 payments (Part H). No jitter exists for the shielded fetch-on-detect.

Bucketing — rounding a range start down to a multiple of N — is ZIP-307's own proposal, unimplemented.

The reading. None of this is a property of compact-block contents, and none of it is inevitable; Tor, bucketing, jitter, and cover traffic can each change it.

Source: *Wallet Guide*, "Key independence of queries"; "Leakage of range queries"; "Summary of server observables"

Board 131 — Spending from a light client: the witness and the anchor

exists

Write

1. witness: 32 siblings, folded on the device from own shard's leaves + other shards' roots, against a checkpointed block
2. the path = private input of the membership clause
3. anchor: any main-chain root since the pool's activation, no window

4. proving decoupled from broadcasting; appending alters no old root
5. older anchor: no soundness hole; a second spend is the nullifier's problem

Say

The witness is folded on the device. Bob's 32 siblings come from his shard's leaves and the other shards' roots, against a checkpointed block; the path is the private input of the membership clause.

The anchor, recalled from Part C. Any main-chain block's root since the pool's activation is admissible, no window, so proving is decoupled from broadcasting; appending alters no old root; an older anchor is no soundness hole, and a second spend is the nullifier's problem.

Bitcoin contrast. A Bitcoin input names one txid and index, and the validator looks it up. The anchor names a tree; the path into it never leaves the device.

Source: *Ironwood Guide*, "Anchors"; "Authentication paths"

Board 132 — Which anchor: the wallet's policy, and whose reasons

exists

Write

1. H = target height = tip +1
2. anchor = root at height $A = H - c_{\text{tr}} = H - 3$ (wallet policy, not consensus)
3. spendable: 3 confirmations trusted, 10 untrusted, from the mined block; shard fully scanned; note's block at or below the anchor

Say

The anchor is the root at height $A = H - 3$. Here H is the target height, one above the tip. This is wallet policy, not consensus; ZIP-315's stated reasons and the Guide's anonymity-set argument are Part C's.

Spendable under the same policy. Three confirmations for trusted notes, ten for untrusted, counted from the mined block; the note's shard fully scanned; and its block at or below the anchor.

Bitcoin contrast. A Bitcoin wallet also waits for confirmations, but which block it "anchors" to is nobody's business, because the input is named either way.

Do not say

- Presenting $H - 3$ as a consensus rule. Consensus admits any main-chain anchor since the pool's activation; $H - 3$ is ZIP-315 wallet policy.
- The anonymity-set argument as ZIP-315's. It is the Guide's; ZIP-315 states its own reasons (Part C).

Source: *Ironwood Guide*, "Anchors"; *Wallet Guide*, "The confirmation policy"; "The transaction plan"

Board 133 — Expiry and broadcast

not twice

Write

1. expiry height (ZIP-203): 0 = none; included above it → block invalid
2. ZIP-218 node default: target +120; ZIP-315 wallet: 40; open
3. store → lock inputs → broadcast, one call
4. expired unmined → inputs released, notes spendable again

Say

Every transaction carries an expiry height. Zero means none; a block that includes it above that height is invalid. ZIP-218 sets target +120 for nodes; ZIP-315 still says 40 for wallets, so the wallet default is open. A uniform delta, like a uniform fee, is not a fingerprint.

Store, then broadcast. The wallet persists the transaction and locks its inputs before the network sees it, then submits it in one call. Once expired unmined, it releases its inputs: the notes are spendable again.

Bitcoin contrast. A stuck transaction has no consensus expiry — mempools drop it by policy after a timeout, or RBF or CPFP bumps it. Here it expires by rule, and its inputs come free.

Source: *Wallet Guide*, "Expiry (ZIP-203)"; "Submission and the transaction identifier"; "The transaction record and locked notes"

Board 134 — The reorganisation case

exists, not twice

Write

1. reorganisation past the anchoring block → anchor invalid
2. → truncate to a checkpoint, rescan, rebuild
3. rebuilt transaction re-reveals the *same* nullifiers (deterministic derivation)
4. re-anchoring, txid unchanged: specified (ZIP-374 v2, anchors in authorizing data); wallet support pending

Say

A reorganisation past the anchoring block invalidates the anchor. The wallet truncates to a checkpoint, rescans, and rebuilds.

The rebuilt transaction re-reveals the same nullifiers. Their derivation is deterministic: the notes are the same, so the serial numbers are.

Re-anchoring with the txid unchanged is specified, in ZIP-374 v2 with anchors in authorizing data; wallet support is pending.

Bitcoin contrast. A reorganised Bitcoin transaction is simply mined again, since it names its inputs and no anchor. Here a struck anchor means a rebuild.

Source: *Wallet Guide*, "Reorganisation detection and recovery"; "PCZT v2: deferred anchors and the Ironwood bundle"

Board 135 — Three openings on Alice's transaction

exists

Draw

Four columns: ciphertext, opened by, with, yields. Rows in order:

1. Action 1, C_{enc} — Bob — ivk — (d, 1.5 ZEC, rseed); the memo, once fetched
2. Action 2, C_{enc} — Alice — ivk^{int} — change 0.4999 ZEC, at her internal address
3. Action 1, C_{out} — Alice — ovk — (pk_{d}, esk), hence Bob's note as he reads it
4. Action 2, C_{out} — nobody — (no key) — random: the wallet default seals change under no ovk

Write

1. Bob: cmx and epk check, position recorded; with nk a nullifier absent from the set; from a stranger: untrusted, 10 confirmations
2. Alice: change by trial decryption under ivk^{int}, never through ovk; internal notes trusted: 3 confirmations
3. Alice: C_{out} → what she sent Bob (sent history; an auditor's, given ovk)

Say

Bob's opening. His cmx and epk checks pass and the position is recorded; with nk he derives a nullifier absent from the set. From a stranger the note is untrusted: ten confirmations.

Alice's change. She finds it by trial decryption under the internal scope, never through an outgoing key; internal notes are trusted: three confirmations.

Alice's sent history. She re-reads what she sent Bob through C_{out} — her sent history, and an auditor's, given ovk. The change Action's C_{out} opens for nobody: the wallet default seals change under no ovk.

Bitcoin contrast. Bob's payment and Alice's change are the two outputs everyone reads, and a memo would be an OP_RETURN.

Ask

Checkpoint: Bob's light client holds ivk and the 52-byte prefixes: what can the server still infer, and what would a Bitcoin SPV client have given away instead?

- Still inferred: the endpoint and when it is active; the earliest range start, about his birthday; the range bounds, his progress; the exact txid, if he fetches the full transaction for the memo; the whole transaction before broadcast, if he submits through the same server.

- Never: which outputs opened, their values, or ivk itself.
- A BIP-37 client hands each peer a Bloom filter of its scripts: addresses, balance, and graph, up to the false-positive rate. A BIP-157 client narrows that to the blocks it fetches after a match — still by asking.

Do not say

- Change recovered via ovk. Alice's change is found by trial decryption under $\mathsf{ivk}^{\mathsf{int}}$; the wallet default seals change under no ovk at all.
- A minimum-Actions constant as the reason for the dummy. Action 2 exists because two outputs need two Actions (Part E); the dummy pads the spend list.
- Deriving ovk from ivk. Both derive from fvk (Part D), neither from the other.

Source: *Ironwood Guide*, "Trial decryption and note acceptance"; "The outgoing ciphertext"; *Wallet Guide*, "Internal keys: change and auto-shielding"; "The transaction plan"; "The confirmation policy". Checkpoint: *Wallet Guide*, "Key independence of queries"; "Leakage of range queries"; "Leakage of transaction queries"; "The submission channel"

9 Part H — The transaction and the turnstile

Board 136 — A v6 transaction beside a Bitcoin transaction

all four

Draw

Two columns headed *Bitcoin* and *Zcash v6*; write the rows top to bottom:

1. header — version, lock time | + version group, branch id, expiry height: 20 B in all
2. inputs — outpoint, scriptSig, sequence | the same bytes; ECDSA over secp256k1, no Taproot; the scriptSig outside the txid
3. outputs — value, scriptPubKey | the same bytes
4. Sapling bundle — — | spends, outputs, Groth16 proofs
5. Orchard bundle — — | Actions of the sealed pool
6. Ironwood component — — | a Actions at 820 B; flags, $v^{\mathsf{Ironwood}}_{\mathsf{balance}}$, anchor; one proof of $2720+2272a$ B; $a+1$ signatures at 64 B

Write

- Proof: $2720 + 2272a$ B; for $a = 2$: 7264 B
- Alice: $a = 2$ Actions, 7264 proof bytes, 9166 bytes in all
- "One ledger, one transaction type, one asset"

Say

In Bitcoin the node reads every field of the transaction: the header, the inputs, the outputs. A Zcash v6 transaction keeps those three parts. The header gains a version group, a branch id and an expiry height, twenty bytes in all; the inputs are the same bytes, still signed with ECDSA over secp256k1, no Taproot, the scriptSig outside the txid; the outputs are the same bytes.

The shielded parts are components beside the transparent ones. The Sapling bundle carries spends, outputs and Groth16 proofs; the Orchard bundle carries Actions of the sealed pool; the Ironwood component carries a Actions at 820 bytes each, the flags, the balancing field $v^{\mathsf{Ironwood}}_{\mathsf{balance}}$, the anchor, one proof of $2720 + 2272a$ bytes and $a + 1$ signatures of 64 bytes.

Alice's payment fills the last component alone. Two Actions, 7264 proof bytes, 9166 bytes in all; the transparent, Sapling and Orchard parts are empty.

Emphasise: one ledger, one transaction type, one asset. The shielded parts are components of the one transaction, not a second ledger.

Do not say

Never "a few hundred bytes": the numbers are 820 B per Action, one proof of 7264 B for Alice's two Actions, 9166 B in all.

Source: *Ironwood Guide*, "The transaction format"; *Crypto Guide*, "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"

Board 137 — Effecting data, authorizing data: the SegWit shape

authorised

Write

- Effecting data = everything under the txid: header; transparent outpoints, sequences, outputs; every Action's public fields; flags; balancing fields
- Authorizing data = what attests to it: proofs, signatures; from v6 the three anchors; each scriptSig
- Bitcoin: txid = SHA-256d(bytes); SegWit moves the witness out, re-commits it through the coinbase
- v6: txid = BLAKE2b-256 tree over effecting data; hashAuthDataRoot → hashBlockCommitments (ZIP-244)
- Re-anchoring, txid unchanged: specified (ZIP-374 v2), wallet support pending
- NU7 permits v5 and v6, not v4; the Orchard-pool seal applies

Say

Bitcoin's txid is SHA-256d of the bytes, and SegWit moved the witness out of them, re-committed through the coinbase. Zcash applies the same move to everything that attests rather than effects.

Effecting data is everything under the txid. The header; the transparent outpoints, sequences and outputs; every Action's public fields; the flags; the balancing fields. Each scriptSig is authorizing data — SegWit's move, applied to the legacy input. Authorizing data is what attests to the effecting data: proofs and signatures, and from v6 the three anchors.

A v6 txid is a BLAKE2b-256 tree over the effecting data alone. ZIP-244 re-commits the authorizing data through hashAuthDataRoot, folded into the header's hashBlockCommitments. Recomputing a proof or a signature changes nothing the identifier names.

Because the anchor is authorizing data, a v6 transaction can be re-anchored with its txid unchanged: its membership proof rebased onto a later root. Say both halves: specified (ZIP-374 v2), wallet support pending.

NU7 permits versions v5 and v6, not v4 (draft ZIP-259). The Orchard-pool seal applies independently of the version.

Do not say

Do not present same-txid re-anchoring as deployed: it is specified (ZIP-374 v2), wallet support pending.

Source: *Ironwood Guide*, "Transaction digests and signatures"; "The transaction format"; *Crypto Guide*, "The transparent layer's hashes"; *FlyClient Guide*, "The authorising-data root"; *Wallet Guide*, "PCZT v2: deferred anchors and the Ironwood bundle"

Board 138 — The sighash (ZIP-244): a tree of digests

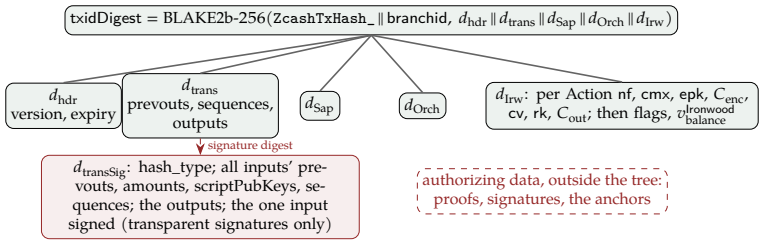
authorised

Draw

1. Top centre, one box:

$$\mathsf{txidDigest} = \text{BLAKE2b-256}(\texttt{ZcashTxHash_} \,\|\, \mathsf{branchid},\; d_{\text{hdr}} \,\|\, d_{\text{trans}} \,\|\, d_{\text{Sap}} \,\|\, d_{\text{Orch}} \,\|\, d_{\text{Irw}})$$

2. Below it, five boxes left to right, a line from the root to each: d_{hdr} (version, expiry); d_{trans} (prevouts, sequences, outputs); d_{Sap}; d_{Orch}; d_{Irw} (per Action nf, cmx, epk, C_{enc}, cv, rk, C_{out}; then flags, $v^{\text{Ironwood}}_{\text{balance}}$).
3. Under d_{trans}, in red, a box d_{transSig}: hash_type; all inputs' prevouts, amounts, scriptPubKeys, sequences; the outputs; the one input signed (transparent signatures only). A dashed red arrow from d_{trans} down to it, labelled "signature digest".
4. Bottom right, a dashed red box outside the tree: "authorizing data, outside the tree: proofs, signatures, the anchors".



Write

- Every node = one BLAKE2b-256 under its own personalisation; the root's carries the branch id
- Signature digest = the txid tree with $d_{trans} \to d_{transSig}$
- No transparent inputs ⇒ signature digest = txid

Say

Every node of the tree is one BLAKE2b-256 under its own personalisation. The root's personalisation carries the branch id, so the same bytes have a different identity on a parallel chain.

The five children are the header digest, the transparent digest, and one per shielded component. The Ironwood child commits, per Action, to nf, cmx, epk, C_{enc}, cv, rk and C_{out}, then to the flags and $v_{balance}^{Ironwood}$.

The signature digest is this tree with d_{trans} swapped for $d_{transSig}$. That child carries the hash_type, all inputs' prevouts, amounts, scriptPubKeys and sequences, the outputs, and the one input being signed — transparent signatures only. A transaction with no transparent inputs has nothing to swap: its signature digest *is* its txid.

Emphasise what sits outside the tree: proofs, signatures, the anchors — authorizing data, committed through hashAuthDataRoot instead.

Source: *Ironwood Guide*, "Transaction digests and signatures"; *Wallet Guide*, "Submission and the transaction identifier"

Board 139 — What the shielded signatures sign

authorised

Write

- Per Action: spend-authorisation signature under rk; per bundle: binding signature under bvk — both sign the signature digest: every Action's nf, cmx, cv, rk, ciphertexts; the flags; $v_{balance}$
- Bitcoin hash_type: ALL, NONE, SINGLE, each with or without ANYONECANPAY: six; a transparent Zcash input: exactly those six
- Taproot's SIGHASH_DEFAULT (a seventh spelling of ALL): not in ZIP-244
- Shielded: SIGHASH_ALL, always; no shielded ANYONECANPAY
- Anchor: not under the signature from v6; the proof carries it as a public input (A3)
- Alice: no transparent inputs ⇒ 2 spend-auth + 1 binding signature all sign her txid

Say

In Bitcoin each signer picks a hash_type. ALL, NONE, SINGLE, each with or without ANYONECANPAY: six meaningful values. Taproot adds SIGHASH_DEFAULT as a seventh spelling of ALL, which ZIP-244 does not define. A transparent Zcash input is restricted by consensus to exactly those six.

A shielded signature has one: SIGHASH_ALL, always. Each Action's signature under rk (spend authorisation) and the bundle's signature under bvk (binding) sign the signature digest: every Action's nf, cmx, cv, rk and ciphertexts, the flags, $v_{balance}$. No Action can be transplanted into another transaction, no field altered. There is no shielded ANYONECANPAY.

The anchor is not under the signature from v6, and does not need to be. The proof carries it as a public input, clause A3: the signature attests to the spend, the proof pins the tree.

Alice has no transparent inputs. Her two spend-authorisation signatures and her binding signature all sign her txid.

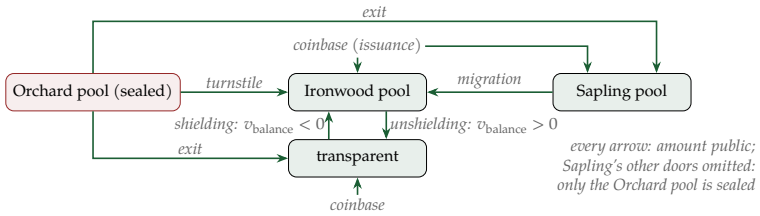
Source: *Ironwood Guide*, "Transaction digests and signatures"; "Verification of a transaction"; *Wallet Guide*, "Signing"; "Proving"

Board 140 — Pools and the doors

conserved

Draw

1. Three boxes in a row: *Orchard pool* (*sealed*) on the left, in red; *Ironwood pool* in the centre; *Sapling pool* on the right.
2. Below the Ironwood pool a fourth box, *transparent*.
3. Above the Ironwood pool the words *coinbase* (*issuance*), with an arrow down into it.
4. An arrow Orchard → Ironwood labelled *turnstile*; an arrow Sapling → Ironwood labelled *migration*.
5. Two vertical arrows between transparent and Ironwood: up, labelled *shielding:* $v_{\text{balance}} < 0$; down, labelled *unshielding:* $v_{\text{balance}} > 0$.
6. Orchard's other exits, labelled *exit*: to transparent, and over the top to Sapling. Coinbase also into Sapling, and from below into transparent.
7. Bottom right, a note: "every arrow: amount public; Sapling's other doors omitted: only the Orchard pool is sealed".



Write

- Shielding: transparent inputs fund $v^{\mathsf{Ironwood}}_{\text{balance}} < 0$; the Actions spend dummies, create real notes. Alice shields 10 ZEC: $v^{\mathsf{Ironwood}}_{\text{balance}} = -9.99985$ ZEC printed; her new note not
- Coinbase: issuance enters a shielded pool directly; every shielded coinbase output decrypts under the all-zero ovk
- Migration: Sapling → Ironwood; Orchard → Ironwood through the turnstile; both balancing fields cleartext

Say

Bitcoin has one pool and needs no doors. Zcash has several, and value crosses between them only through doors that print the amount.

Shielding. Transparent inputs fund a negative $v^{\mathsf{Ironwood}}_{\text{balance}}$; the Actions spend dummies and create real notes. Alice shields 10 ZEC: $v^{\mathsf{Ironwood}}_{\text{balance}} = -9.99985$ ZEC is printed, her new note is not. Wallets shield automatically, to an internal address.

Coinbase. Issuance enters a shielded pool directly, but every shielded coinbase output must decrypt under the all-zero ovk: anyone can read what issuance entered which pool.

Migration. Sapling or the sealed Orchard pool to Ironwood, the latter through the turnstile; both balancing fields are cleartext.

Emphasise: every arrow on the board prints its amount. Only the Orchard pool is sealed; Sapling's other doors are left off the drawing.

Source: *Ironwood Guide*, "The balancing value"; "Chain state and pool rules"

Board 141 — The turnstile (ZIP-209): audited at the boundary

conserved

Write

- $\mathsf{balance}_P := -\sum_{\text{tx}} v^P_{\text{balance}}(\text{tx}) \geq 0$ in every block, for $P \in \{\mathsf{Sap}, \mathsf{Orch}, \mathsf{Irw}\}$
- Sprout: $\sum$ `vpub_old` $-\sum$ `vpub_new`; transparent: the UTXO sum; lockbox: its accrued subsidy
- A block driving any balance negative, or the total past MAX_MONEY: invalid
- Inside a pool: counterfeit undetectable. At the exit: capped at what the pool publicly holds
- Example: pool holds only Alice's 9.99985 ZEC; Bob's +9.99985 ZEC exit empties it exactly; one zatoshi more ⇒ block invalid

Say

Bitcoin audits by summing every coin. Inside a shielded pool no check sums the notes, so a counterfeit is undetectable there. Sprout's original proof system had exactly such a flaw (Part G): a forged proof mints silently.

The mechanism is a public balance per pool, checked at the door. The balance of pool P is minus the sum of its balancing fields over every transaction, and it must stay non-negative in every block, for the Sapling, Orchard and Ironwood pools. Every pool has a public balance under this rule: Sprout's is $\sum$ `vpub_old` $-\sum$ `vpub_new`, the transparent pool's the UTXO sum, the lockbox's its accrued subsidy. A block driving any negative, or the total past MAX_MONEY, is invalid.

At the exit the damage is capped. A forger can withdraw at most what the pool publicly holds. The Sapling pool still rests on its Groth16 ceremony; the turnstile bounds it.

On the running example: had the Ironwood pool held only Alice's deposit of 9.99985 ZEC, Bob's +9.99985 ZEC exit would empty it exactly; one zatoshi more and his block is invalid.

Emphasise: Zcash audits each pool at its door, not inside.

Do not say

Never "the pool is auditable": nothing sums the notes inside; only the balance at the door is public, and a counterfeit inside is undetectable until it tries to leave. Never "no ceremony" without the caveat: the Sapling pool still rests on its Groth16 ceremony, bounded by the turnstile.

Source: *Ironwood Guide*, "Chain state and pool rules", Remark "Outflow bound"; ZIP-2005, "Effects of discrete-logarithm-breaking attacks before the switch to the Recovery Protocol"; ZIP-209; protocol specification, "Chain Value Pool Balances"; "BCTV14"

Board 142 — Why the Orchard pool is sealed

conserved

Write

- Three rules: no issuance; no circulation ($\mathsf{enableCrossAddress} = 0$); no entry ($v^{\mathsf{Orchard}}_{\mathsf{balance}} \geq 0$) — whatever the transaction version
- Legacy note, lead byte `0x02`: not quantum-recoverable. Ironwood note, lead byte `0x03`, hashed rcm: recoverable
- Orchard pool balance can only fall; every exit metered in cleartext by $v^{\mathsf{Orchard}}_{\mathsf{balance}}$
- Bitcoin never retires an output type; a P2PK coin spends forever

Say

Bitcoin never retires an output type; a P2PK coin spends forever. Here a pool can be sealed by three rules and drained through a metered gate.

The seal is the three rules of Parts C and E. They bind whatever the transaction version: no issuance; no circulation, $\mathsf{enableCrossAddress} = 0$; no entry, $v^{\mathsf{Orchard}}_{\mathsf{balance}} \geq 0$.

The main reason is supply integrity after the Orchard soundness bug (ZIP-257); the other is Part F's one note-level difference. A legacy note, lead byte `0x02`, is not quantum-recoverable; an Ironwood note, lead byte `0x03` with the hashed rcm, is. So consensus stops the old notes circulating and funnels their value out through the turnstile, while owners can still spend what they hold.

The turnstile is the seal's accounting shape. The Orchard pool's balance can only fall, every exit metered in cleartext by $v^{\mathsf{Orchard}}_{\mathsf{balance}}$, so the legacy pool drains through a public gate.

Source: *Ironwood Guide*, "Chain state and pool rules"; "The Action circuit and the Halo 2 proof"; "The note seed: Ironwood derivations (lead byte `0x03`)", Remark "Purpose of the hashed trapdoor"

Board 143 — Fees (ZIP-317): a function of the public shape

conserved

Write

$$fee = 5000 \cdot \max(2,\ logical_actions) \quad \text{zatoshi}$$

$$logical_actions = \max\Big(\Big\lceil \tfrac{\text{in bytes}}{150} \Big\rceil, \Big\lceil \tfrac{\text{out bytes}}{34} \Big\rceil\Big) + 2\, nJoinSplit_{\text{Sprout}} + \max(spends, outputs)_{\text{Sap}} + nActions_{\text{Orch}} + nActions_{\text{Irw}}$$

- Divisors: standard P2PKH input 150 B, output 34 B
- Alice: two Actions, $5000 \cdot \max(2,2) = 10\,000$ zatoshi
- One Action padded to two: 10 000, the same
- Shielding with one P2PKH input: $1 + 2 = 3$ actions, 15 000

Say

In Bitcoin the node reads the fee by subtraction, inputs minus outputs, both in plaintext. Here the amounts are hidden, so the fee must be a function of what is public: the shape of the transaction.

The fee is 5000 **zatoshi per logical action, with a floor of two.** Logical actions count, per pool: transparent, the larger of input bytes over 150 and output bytes over 34, each rounded up; two per Sprout JoinSplit; Sapling, the larger of spends and outputs; Orchard and Ironwood, the number of Actions. Sum them.

The divisors are the standard P2PKH input and output. One typical transparent input or output weighs one action, and a fatter script pays in proportion. An Action spends one note *and* creates one, so it is counted once — charging inputs and outputs separately would tax the padding.

Alice pays 10 000 **zatoshi:** two Actions, $5000 \cdot \max(2,2)$. A one-Action payment padded to two pays the same. A shielding transaction with one P2PKH input counts $1 + 2 = 3$ actions and pays 15 000.

Do not say

Do not give the fee's floor of two as the reason for Alice's dummy: she has two Actions because she has two outputs (Bob and her change); the dummy pads her single spend to match. The floor is the fee's grace window, not the padding rule.

Source: *Wallet Guide*, "The conventional fee"; "The fee on padded counts"

Board 144 — Fees (ZIP-317): why a convention, and who enforces it

conserved

Write

- Computable from public fields alone
- Uniform across wallets: not a fingerprint; padding to two is free (grace window = 2)
- A convention, not consensus: relay at least the fee; template by fee ratio, capped at 4×; at most 50 unpaid actions per block
- Bitcoin: a market — bid per virtual byte and coin selection vary; both fingerprints

Say

Bitcoin runs a fee market. The bid per virtual byte, and the coin selection behind it, vary by wallet and moment — and both are read as fingerprints. Zcash flattens the market on purpose.

Computable from public fields alone. A fee that depended on which notes were spent or what change came back would print that data in the amount paid.

Uniform across wallets. Every wallet is expected to pay exactly this fee, so the fee is not a fingerprint. Padding to two Actions is free by design: the grace window is two.

A convention, not consensus. Nodes relay transactions paying at least it; the recommended block template picks among those by fee ratio, capped at 4×, and admits at most 50 unpaid actions per block. Overpaying beyond 4× buys no further priority.

Source: *Wallet Guide*, "The conventional fee"; "Relay, mempool, and block template"

Board 145 — The public face of Alice's transaction

all four

Draw

Two columns, *public field* and *Alice pays Bob* 1.5 *ZEC from a* 2 *ZEC note*; write the rows top to bottom:

1. version — v6; transparent, Sapling and Orchard parts empty
2. Ironwood Actions — 2 (one real spend, one dummy; shuffled)
3. nullifiers, commitments — 2 and 2
4. $v_{\text{balance}}^{\text{Ironwood}}$ — +10 000 zatoshi = the fee
5. anchor — the Ironwood root at height $A = H - c_{\text{tr}} = H - 3$
6. expiry — target height + 40 or 120 blocks; wallet default open
7. signatures — 2 spend-authorisation + 1 binding
8. size — 9166 bytes, 7264 of them proof

Write

- Bitcoin prints: input coin, both amounts, both scripts, fee by subtraction
- Here printed: the fee alone, as $v_{\text{balance}}^{\text{Ironwood}}$
- Inside the commitments: 2 ZEC, 1.5 ZEC, 0.4999 ZEC change, both addresses

Say

Bitcoin prints the same payment as the input coin, both amounts, both scripts and, by subtraction, the fee. Here the fee alone is printed, as $v_{\text{balance}}^{\text{Ironwood}}$.

Walk the table. Version v6, with the transparent, Sapling and Orchard parts empty. Two Ironwood Actions, one real spend and one dummy, shuffled. Two nullifiers and two commitments. The balancing field +10 000 zatoshi, which is the fee. The anchor is the Ironwood root at height $H - 3$. The expiry is the target height plus a delta of 40 or 120 blocks; the wallet default is open. Two spend-authorisation signatures and one binding signature. Then 9166 bytes, 7264 of them proof.

Emphasise what stays inside the commitments: the 2 ZEC, the 1.5 ZEC, the 0.4999 ZEC change and both addresses.

Do not say

Never "the proof hides sender and receiver": the amounts and addresses stay inside the commitments; the proof only attests to the statement. Never "a few hundred bytes": 9166 bytes, 7264 of them proof. Do not give a two-Action minimum as the reason for the dummy: two outputs need two Actions, and the dummy pads the single spend.

Source: *Ironwood Guide*, "The transaction format"; "Anchors"; "The balancing value"; *Wallet Guide*, "Padding, dummies, and shuffling"; "Expiry (ZIP-203)"

Board 146 — The privacy sliders: pool, address type, timing

all four

Write

- Pool: transparent leg prints its amount as v_{balance}; fully shielded prints the fee and nothing else
- Address type: unified address = bundle of receivers; wallet MUST use the most preferred it supports: Orchard-protocol (Ironwood pool) > Sapling > transparent. TEX address ⇒ two transactions via an ephemeral transparent output, both public
- Timing: one shared expiry delta (40 or 120 blocks, open). Migration: denominations $m \cdot 10^k$ ZEC, $m \in \{1, 2, 5\}$; anchors and expiries from shared grids
- "Uniformity is the strategy"

Say

Bitcoin has the address-type slider and the timing slider; it has no pool slider, because it has one plaintext ledger.

Pool. A transparent leg prints its amount as v_{balance}; fully shielded prints the fee and nothing else. The slider is the sender's, set by which funds and which receiver she uses.

Address type. A unified address bundles receivers; the sender's wallet MUST use the most preferred one it supports: Orchard-protocol, into the Ironwood pool, over Sapling over transparent. A TEX address demands transparent funds, so the wallet builds two transactions through an ephemeral transparent output — both public.

Timing. Wallets share one expiry delta, 40 or 120 blocks while the default is open: an idiosyncratic delta fingerprints a transaction as an idiosyncratic fee would. Migration wallets split value into canonical denominations $m \cdot 10^k$ ZEC, $m \in \{1, 2, 5\}$, and draw anchors and expiries from shared grids for the same reason.

Emphasise: uniformity is the strategy. The defaults protect only while everyone keeps them.

Source: *Wallet Guide*, "Unified addresses"; "Input selection"; "Expiry (ZIP-203)"; ZIP-318, "Amount selection, canonical quantization"; "Anchor-height bucketing and cohorts"

Board 147 — PSBT → PCZT (ZIP-374): the unfinished transaction

authorised

Write

- PSBT (BIP-174 + BIP-370): Creator, Constructor, Updater, Signer, Combiner, Input Finalizer, Transaction Extractor
- PSBT cannot carry shielded: no shielded spends/outputs; no proof distinct from a signature; tolerates unknown fields
- PCZT = PSBT roles + IO Finalizer, Redactor, Prover; ten roles
- "Proving is delegated, signing is not"

Say

Bitcoin passes an unfinished transaction between roles: PSBT. BIP-174, plus BIP-370's Constructor: Creator, Constructor, Updater, Signer, Combiner, Input Finalizer, Transaction Extractor. Hardware wallets and multisig live on it.

A PSBT cannot carry a shielded transaction. No representation for shielded spends and outputs; no notion of a proof distinct from a signature; and it tolerates unknown fields — unacceptable for a signer that must understand everything it authorises.

A PCZT keeps the PSBT role structure and adds Zcash's own. Among them an IO Finalizer, which declares the inputs and outputs complete and derives the binding-signature key; a Redactor, which strips what later parties need not see; and a Prover. Ten roles in all.

Emphasise: proving is delegated, signing is not. The Prover has no PSBT counterpart because a hardware signer cannot compute a Halo 2 proof, and a proof needs no spending key. Signers can be fully offline; the format carries everything they need.

Do not say

No timing claims: say that a hardware signer cannot compute a Halo 2 proof, not how long a proof takes anywhere.

Source: *Wallet Guide*, "Partially created transactions (PCZT)"; "Roles and the modification state"

Board 148 — Signer versus Prover

authorised

Draw

Two columns headed *Signer* and *Prover*; write the rows top to bottom:

1. holds — ask, spend authority | fvk, no spending key
2. reads from the PCZT — α, rk; the spend's public fields and, unless redacted, its note fields | the note ($\mathsf{g_d}$, $\mathsf{pk_d}$, v, ρ, rseed), the witness, α, rcv, the anchor
3. computes — the sighash from the effecting data; $\mathsf{rsk} = \mathsf{ask} + \alpha$, checks it matches rk, signs | one Halo 2 proof over all Actions; anchor and flags as public inputs
4. verifies first — nf, cv, cmx against the note fields: exactly what it authorises | fvk owns each note; $\rho_{\text{new}} = \mathsf{nf}_{\text{old}}$
5. learns — what it signs | the notes' private data, not authority

Write

- $\mathsf{rsk} = \mathsf{ask} + \alpha$; check against rk before signing
- $\rho_{\text{new}} = \mathsf{nf}_{\text{old}}$
- Either role runs without the other; the Prover never sees ask; the Signer may see the note data, or have it redacted

Say

In Bitcoin there is no proof distinct from the signature, so the signer is the only role that attests. Here proof and signature are separate artefacts with separate secrets, and the table shows how little the two roles share.

The Signer holds ask, the spend authority. It reads α and rk from the PCZT, plus the spend's public fields and, unless redacted, its note fields. It computes the sighash from the effecting data and $\mathsf{rsk} = \mathsf{ask} + \alpha$, checks that this matches rk, and signs. Before signing it verifies nf, cv and cmx against the note fields: exactly what it authorises. It learns what it signs.

The Prover holds fvk and no spending key. It reads the note $(\mathsf{g_d}, \mathsf{pk_d}, v, \rho, \mathsf{rseed})$, the witness, α, rcv and the anchor. It computes one Halo 2 proof over all Actions, with the anchor and flags as public inputs. First it checks that fvk owns each note and that $\rho_{\text{new}} = \mathsf{nf}_{\text{old}}$. It learns the notes' private data, not authority.

Emphasise: either role can run without the other; the Prover never sees ask; the Signer may see the note data, or have it redacted.

Do not say

Re-randomisation is additive: $\mathsf{rsk} = \mathsf{ask} + \alpha$, never $[\alpha] \cdot \mathsf{ak}$. The seed ρ is a per-note uniqueness value, $\rho_{\text{new}} = \mathsf{nf}_{\text{old}}$, not the note's position in the tree.

Source: *Wallet Guide*, "Signing"; "Proving"

Board 149 — PCZT: where it stands

authorised

Write

- From v6: signatures commit to no anchor ⇒ sign today, anchor, witness and prove at broadcast
- Delegating the Prover reveals note data, never spend authority; a Redactor strips what a later party need not see
- Bins: ZIP Draft; format released as a library; deployed wallet pipeline drives the roles end to end (Ironwood sent-output records not yet stored: backend gap)
- + FROST (Part I): "no script, so no multisig and no hardware wallets" — closed

Say

From v6 the Signer and the Prover are independent even of the anchor. Signatures commit to no anchor, so a wallet can sign today and anchor, witness and prove at broadcast.

Delegating the Prover reveals the notes' private data to the delegate, never spend authority. A Redactor strips what a later party need not see.

Bins. The ZIP is Draft; the format is released as a library; the deployed wallet pipeline drives the roles end to end — it builds the PCZT, hands it out for proving and signing, and extracts and stores the result. Ironwood sent-output records are not yet stored: a backend gap, not a format limit.

Emphasise: with FROST threshold signing (Part I) this closes the objection "no script, so no multisig and no hardware wallets". The roles carry both.

Ask

Alice shields 10 ZEC; the next day Bob unshields 9.9997 ZEC. What can a chain analyst conclude, and what can they not?

Conclude. Two cleartext balancing fields, exact to the zatoshi: −9.99985 ZEC entered, +9.99985 ZEC left — 10 ZEC less two conventional fees of 15 000 zatoshi; the heights; the transparent addresses on each side; that the amounts match exactly and the times are close.

Not conclude. That Bob's coins were Alice's. Her commitment hides her note; his nullifier is keyed by his nk; his candidate set is every leaf under his anchor. Her note may still be resting.

Bitcoin would have handed over the path; Zcash hands over a correlation. Canonical denominations and shared timing grids exist because that correlation is evidence.

Do not say

Never "the pool is auditable": the analyst reads two door amounts and a correlation, nothing inside the pool.

Source: *Wallet Guide*, "Partially created transactions (PCZT)"; "Combining, redaction, and privacy"; "PCZT v2: deferred anchors and the Ironwood bundle"; *Ironwood Guide*, "The balancing value"; "Anchors"; "Nullifier uniqueness and unlinkability"; ZIP-318, "Amount selection, canonical quantization"; "Anchor-height bucketing and cohorts"

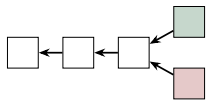
10 Part K — Consensus: why one fork wins, and what the chain must remember

Board 150 — Bitcoin's chain rule, weighed in work

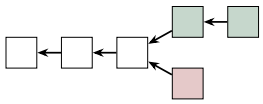
not twice

Draw

1. Panel (a): three white boxes in a row; an arrow from the second box back to the first and from the third back to the second (a block points at its parent).
2. Right of the third box, two boxes one above the other, the upper shaded green, the lower shaded red; an arrow from each back to the third box.
3. Caption under the panel: "(a) equal work: nodes disagree".
4. Panel (b), to the right: the same five boxes, plus a sixth green box right of the upper green one, with an arrow back to it.
5. Caption: "(b) one more block: the upper branch is heavier".



(a) equal work: nodes disagree



(b) one more block: the upper branch is heavier

Write

- work of a block = $\lfloor 2^{256} / (\texttt{ToTarget(nBits)} + 1) \rfloor$
- best valid chain = greatest total work; ties → the block received first
- "longest" = "most work" only at constant difficulty
- nothing marks a chain as final

Say

In Bitcoin every block names its parent, so the node holds a tree of blocks, and Zcash keeps the same rule. Each branch is one consistent history, and sibling branches may conflict.

A branch is weighed in work, not counted in blocks. The work of a block is the first line on the board; the best valid chain has the greatest total work; ties go to the block received first.

Bitcoin: the same rule. "Longest" equals "most work" only at constant difficulty.

Emphasise: nothing marks a chain as final. A heavier branch wins however deep the fork; the rest of this part measures how likely that is.

Do not say

"Ties go to the block received first" as every node's behaviour: Zebra applies it from 6.4.0; earlier versions took the larger tip hash (the departure board, later in this part).

Source: *Consensus Guide*, "Work, and the best chain"; protocol specification, "The Block Chain"; "Definition of Work"

Board 151 — Two double-spends; consensus answers one

not twice

Write

- same chain: two spends of one note ⇒ the same nf twice ⇒ invalid by rule; no race, no probability

- chain level: pay on the chain everyone sees, then replace that chain; no proof rules it out, only the proof-of-work race bounds it
- Bitcoin: the same two attacks; a UTXO lookup there, a nullifier lookup here

Say

In Bitcoin the node stops a double spend in two places: a UTXO lookup on the chain it holds, and the work rule between chains. Both attacks exist here; the first changes shape and the second does not.

On the same chain, two spends of one note reveal the same nullifier, and a repeated nullifier is invalid by rule (Part C, and the nullifier-set board later in this part). No race, no probability.

At the chain level the attacker pays the merchant on the chain everyone sees, then replaces that chain with one in which the payment never happened. No proof rules this out; only the proof-of-work race bounds it.

Emphasise: the same two attacks as Bitcoin. The first is a UTXO lookup there and a nullifier lookup here; the second is what this part measures.

Source: *Consensus Guide*, "Scope, sources, and the two double-spends"

Board 152 — The attack

not twice

Write

1. broadcast the payment to the merchant
2. mine a secret branch from the last pre-payment block, carrying a conflicting payment to yourself
3. wait until the merchant ships at n confirmations
4. extend the secret branch until it carries more work, then release it

Under the list: "step 4 breaks no validity rule — does the secret branch ever get ahead?"

Say

Bitcoin's whitepaper analyses this attack in its §11, approximately. Meni Rosenfeld's *Analysis of hashrate-based double-spending* is the exact counterpart, and this part follows it throughout.

Four steps. Broadcast the payment; mine a secret branch from the last pre-payment block carrying a conflicting payment to yourself; wait until the merchant ships at n confirmations; extend the secret branch until it carries more work, then release it — a well-formed chain, merely a different one.

Emphasise: nothing in step 4 breaks a validity rule. The only question is probabilistic: does the secret branch ever get ahead? From here on its odds are computed exactly.

Source: *Consensus Guide*, "The attack"

Board 153 — The model: memoryless clocks

not twice

Write

- $\Pr[X > t] = e^{-\lambda t}$; $\Pr[X > s + t \mid X > s] = \Pr[X > t]$
- two clocks: $\Pr[X < Y] = \lambda/(\lambda + \mu)$; the winner is independent of the winning time
- honest rate p/T_0, attacker rate q/T_0, $p + q = 1$; both restart at every block
- next block is the attacker's with probability q, independently of everything before
- block attribution = an i.i.d. p/q coin sequence; T_0 drops out of every answer

Say

Bitcoin's whitepaper races an honest and an attacking hashrate; this is Rosenfeld's version of that race, with the two assumptions he makes explicit — constant hashrates and constant difficulty.

An exponential clock of rate λ is memoryless: having waited s says nothing about the wait still to come. Two clocks race: the rate-λ clock rings first with probability $\lambda/(\lambda + \mu)$, and which clock rang is independent of when it rang.

Put the honest network at rate p/T_0 and the attacker at q/T_0, with T_0 the network's mean block time and $p+q=1$; both restart at every block. Then the next block is the attacker's with probability q, independently of everything before.

Emphasise: block attribution is an i.i.d. coin sequence, and T_0 drops out of every answer. How well Equihash fits the model is a deployment question (the rails, later in this part).

Source: *Consensus Guide*, "Memoryless clocks and the attribution sequence"; "The catch-up probability"

Board 154 — The negative binomial law

not twice

Write

- m = attacker blocks found strictly before the honest network's n-th block
- $P(m) = \binom{m+n-1}{m} p^n q^m, \quad m = 0, 1, 2, \ldots$
- first-to-n race: H_n or A_n; no tie; $\Pr[H_n] + \Pr[A_n] = 1$
- $q = 1/5$, $n = 6$: $P(0) = 0.262144$, $P(1) = 0.314573$, $P(2) = 0.220201$, ..., $P(5) = 0.021139$; $\Pr[m \geq 6] = 0.011654$

Say

Bitcoin's whitepaper approximates the attacker's progress by a Poisson; here is the exact law. Let m be the number of attacker blocks found strictly before the honest network's n-th block.

The $(m+n)$-th trial is the n-th honest success, and the m attacker blocks sit among the first $m+n-1$ trials in any order: $P(m) = \binom{m+n-1}{m} p^n q^m$.

The same coin sequence is a first-to-n race. Either the honest side reaches n first (H_n) or the attacker does (A_n); counts advance one at a time, so there is no tie and the two probabilities sum to one.

Write the running numbers: at $q = 1/5$, $n = 6$, the six probabilities and the tail $\Pr[m \geq 6] = 0.011654$; they return in the worked race. The Poisson stand-in gets its own board after that race.

Source: *Consensus Guide*, "The negative binomial law"

Board 155 — The catch-up theorem (Rosenfeld §3)

not twice

Write

- z = the honest lead in blocks; up 1 with probability p, down 1 with probability q; success = z reaches -1 (strictly ahead)

$$a_z \;=\; \Pr[\text{walk from } z \text{ ever reaches } -1] \;=\; \begin{cases} 1 & z < 0 \text{ or } q \geq p, \\ (q/p)^{z+1} & z \geq 0 \text{ and } q < p. \end{cases}$$

- each block of deficit multiplies the chance by q/p
- $q \geq p$: overtake with probability 1 from any finite deficit
- whitepaper: $(q/p)^z$ — breakeven scoring, no pre-mined block

Say

Bitcoin's whitepaper asks how an attacker z blocks behind ever catches up; Rosenfeld answers exactly. Let z be the honest lead in blocks. Each block moves z up by 1 with probability p, down by 1 with probability q. The attack succeeds the moment z reaches -1: strictly ahead.

For $q < p$ the chance is $(q/p)^{z+1}$; otherwise it is 1. Each block of deficit multiplies the chance by q/p: two forks cannot compete for long, and one quickly wins.

At $q \geq p$ the attacker overtakes with probability 1 from any finite deficit; no confirmation count defends.

Bitcoin: the whitepaper writes $(q/p)^z$. It scores success at breakeven and banks no pre-mined block; the two conventions cancel term for term, shown on the confirmations boards.

Do not say

The whitepaper's $(q/p)^z$ as this board's theorem: the theorem is Rosenfeld's — success at -1, strictly ahead, exponent $z+1$.

Source: *Consensus Guide*, "The catch-up probability"

Board 156 — Proof idea: a walk between two barriers

not twice

Write

$$h_N(z) = p\,h_N(z+1) + q\,h_N(z-1), \qquad h_N(-1) = 1, \quad h_N(N) = 0$$

$$p\,t^2 - t + q \;=\; p\,(t-1)\,(t-q/p)$$

$$q \neq p: \quad h_N(z) \;=\; \frac{(q/p)^z - (q/p)^N}{(q/p)^{-1} - (q/p)^N} \xrightarrow{N\to\infty} \begin{cases} (q/p)^{z+1} & q < p, \\ 1 & q > p, \end{cases}$$

$$q = p: \quad h_N(z) = (N-z)/(N+1) \to 1$$

Say

Absorb the walk at -1 and at a far barrier $N > z$, and let $h_N(z)$ be the chance of reaching -1 first. Condition on the first step: up with probability p, down with probability q; the boundary values are $h_N(-1) = 1$ and $h_N(N) = 0$.

Try t^z. It solves the recurrence exactly when t is a root of $pt^2 - t + q$, which factors as $p(t-1)(t-q/p)$; the two boundary values then fix $h_N(z)$ for $q \neq p$.

Send N to infinity. For $q < p$ the limit is $(q/p)^{z+1}$; for $q > p$ it is 1; for $q = p$ the double root gives $(N-z)/(N+1) \to 1$.

Emphasise why the limit is legitimate: the events "reach -1 before N" increase to "ever reach -1".

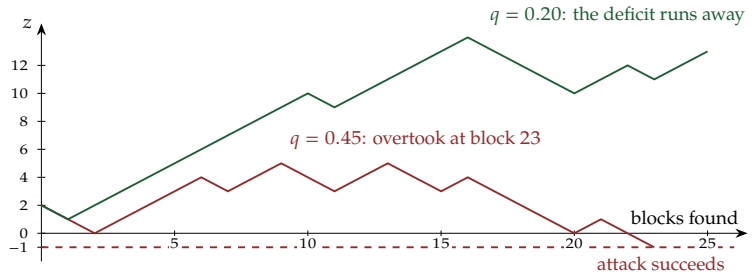
Source: *Consensus Guide*, "The catch-up probability"

Board 157 — The deficit walk

not twice

Draw

1. Axes: horizontal "blocks found" with ticks at $5, 10, 15, 20, 25$; vertical z from -2 to 15 with ticks at $-1, 0, 2, 4, \ldots, 12$.
2. A dashed red line across at $z = -1$, labelled "attack succeeds".
3. The red run, $q = 0.45$: start at $z = 2$; down to 0 by block 2; up to 5 by block 9; wander between 3 and 5 until block 16; fall to 0 at block 20; cross to -1 at block 23. Label it "$q = 0.45$: overtook at block 23".
4. The green run, $q = 0.20$: start at $z = 2$; dip to 1 at block 1; climb one per block to 10 at block 10; wobble up to 14 at block 16; drift back to 10 at block 20; end at 13 at block 25. Label it "$q = 0.20$: the deficit runs away".



Write

- from deficit 13 at $q = 0.20$: $(1/4)^{14} \approx 3.7 \times 10^{-9}$
- $q = 10\%$: $a_0 = 1/9 \approx 0.1111$, $\quad a_5 \approx 1.88 \times 10^{-6}$
- $q = 30\%$: $a_0 = 3/7 \approx 0.4286$, $\quad a_5 \approx 6.20 \times 10^{-3}$

Say

Two seeded runs from $z = 2$ show the theorem in motion. At $q = 0.45$ this run reaches -1 after 23 blocks; at $q = 0.20$ the deficit runs to 13, from which the chance of ever returning is $(1/4)^{14} \approx 3.7 \times 10^{-9}$.

Write the decay anchors. At $q = 10\%$ the ratio q/p is $1/9$: $a_0 \approx 0.1111$ and $a_5 \approx 1.88 \times 10^{-6}$. At $q = 30\%$ the ratio is $3/7$: $a_0 \approx 0.4286$ and $a_5 \approx 6.20 \times 10^{-3}$.

Emphasise: every block of deficit multiplies the chance by q/p; at $q = 0.20$ the deficit runs away.

Source: *Consensus Guide*, "The catch-up probability"

Board 158 — Waiting for confirmations: the pre-mined block

not twice

Write

- the merchant ships after n confirmations; one attacker block banked before scoring; m further attacker blocks
- $z = n - m - 1$

$$r(q,n) \;=\; \underbrace{\sum_{m=0}^{n-1} P(m)\,(q/p)^{\,n-m}}_{\text{behind: must catch up}} \;+\; \underbrace{\sum_{m\geq n} P(m)}_{\text{already ahead}}$$

- $r = 1$ when $q \geq p$ (every catch-up factor is 1)
- no banked block: walk starts at $n - m$, every probability shrinks

Say

In Bitcoin the merchant ships after n confirmations, and the whitepaper computes the odds approximately; this is the same question, answered exactly over the next boards.

Rosenfeld banks one attacker block before the race is scored, so with m further attacker blocks the walk starts at $z = n - m - 1$.

Average the catch-up probability over the negative binomial m. With $m \leq n - 1$ the attacker is behind and must catch up from z; with $m \geq n$ he is already ahead. And $r = 1$ when $q \geq p$, since every catch-up factor is then 1.

Emphasise: the convention is a modelling choice. With no banked block the walk starts at $n - m$ and every probability shrinks.

Source: *Consensus Guide*, "Waiting for confirmations"

Board 159 — Rosenfeld eq. (1): the closed forms

not twice

Write

$$\binom{m+n-1}{m} p^n q^m \left(\frac{q}{p}\right)^{n-m} \;=\; \binom{m+n-1}{m} p^m q^n$$

$$\begin{aligned} r(q,n) \;&=\; 1 - \sum_{m=0}^{n-1} \binom{m+n-1}{m} (p^n q^m - p^m q^n) \\ &=\; 2 \sum_{m=0}^{n-1} \binom{m+n-1}{m} p^m q^n \;=\; 2\Pr[A_n] \end{aligned}$$

Under it: "race doubling: catch-up half = already-ahead half".

Say

Each catch-up term transforms exactly (the first line on the board) into the probability that the attacker's n-th block arrives while the honest side holds m. Summed over $m \leq n - 1$ this is $\Pr[A_n]$, and so is the already-ahead tail.

Hence the two closed forms: one minus a sum of differences, or twice $\Pr[A_n]$.

Emphasise the race doubling. Under the one-pre-mined-block convention the catch-up half equals the already-ahead half exactly; the double-spend probability is twice the chance of winning a first-to-n race, and at $q = p$ each half is $\frac{1}{2}$, giving $r = 1$ continuously.

Source: *Consensus Guide*, "Waiting for confirmations"

Board 160 — A full race, exactly: $q = 1/5$, $n = 6$

not twice

Draw

Four columns; write the rows top to bottom:

1. m $P(m)$ $a_{5-m} = 4^{-(6-m)}$ product
2. 0 0.262144 0.000244 0.000064
3. 1 0.314573 0.000977 0.000307
4. 2 0.220201 0.003906 0.000860
5. 3 0.117441 0.015625 0.001835
6. 4 0.052848 0.062500 0.003303
7. 5 0.021139 0.250000 0.005285
8. catch-up half 0.011654
9. already-ahead half, $\Pr[m \geq 6]$ 0.011654
10. $r(1/5, 6)$ 0.023308

Write

- deficit starts at $z = 5 - m$; catch-up factor $4^{-(6-m)}$
- the halves agree to every digit; closed form gives the same 0.023308
- six confirmations, 20% adversary: 2.33% strict-overtake risk

Say

Six confirmations is Bitcoin's folk number; here is what it buys, exactly. Every entry is an exact rational, printed to six decimals; the deficit starts at $z = 5 - m$ and the catch-up factor is $4^{-(6-m)}$.

Fill the table row by row, multiplying each $P(m)$ from the negative binomial board by its catch-up factor. The products sum to 0.011654; the already-ahead tail $\Pr[m \geq 6]$ is 0.011654 too.

The two halves agree to every digit, as the race doubling says they must; the closed form gives the same 0.023308.

Emphasise: a merchant who ships after six confirmations against a possible twenty-percent adversary carries a 2.33% strict-overtake risk.

Source: *Consensus Guide*, "Waiting for confirmations"

Board 161 — Satoshi's approximation

not twice

Draw

Four columns; write the rows top to bottom:

1. q n whitepaper exact
2. 10% 6 0.0243% 0.0591%
3. 30% 6 13.211% 15.645%
4. 10% 10 0.0001% 0.0008%

Write

- Poisson, mean $\lambda = nq/p$: $\Pr[k] = e^{-\lambda}\lambda^k/k!$
- no pre-mined block, success at breakeven: catch-up factor again $(q/p)^{n-m}$, term for term
- the conventions cancel; the Poisson step is the entire gap

Say

The whitepaper computes the same quantity with one approximation and two offsetting conventions. The attacker's progress is taken as Poisson with mean $\lambda = nq/p$.

No block is pre-mined, but success is scored at breakeven rather than strictly ahead, so the catch-up factor is again $(q/p)^{n-m}$, term for term. The conventions cancel and the Poisson step is the entire gap.

In each pair on the board the approximation understates the risk; the direction is not uniform over all (q, n).

Emphasise: the reason to prefer the negative binomial is that it is exact under the stated assumptions.

Do not say

The whitepaper's $(q/p)^z$ as the theorem: it is the breakeven, no-banked-block convention; the catch-up board's theorem is Rosenfeld's $(q/p)^{z+1}$.

Source: *Consensus Guide*, "Waiting for confirmations"

Board 162 — Blocks, not time

not twice

Write

- $r(q, n)$ contains no T_0: security per confirmation depends on block counts alone
- NU7 25-second blocks; Bitcoin 600
- thirty minutes = 3 Bitcoin or 72 Zcash confirmations (nominal); under a 10% attack either wait $\approx$ 33.3 min (public blocks arrive at rate p)
- $r(0.1, 3) = 1.712\%$ versus $r(0.1, 72) \approx 9.35 \times 10^{-34}$
- caveat: no network in the model; faster blocks raise honest self-orphaning, which effectively raises q

Say

The catch-up formula contains neither T_0 nor any other time scale: a deficit's security is a function of block counts alone. Two chains with the same q offer identical security per confirmation — and therefore different security per minute.

NU7 targets 25-second blocks; Bitcoin 600. Thirty minutes buys 3 Bitcoin confirmations or 72 Zcash ones in nominal expected time; under a 10% withholding attack either wait takes about 33.3 minutes, since public blocks arrive at rate p.

Conditional on those counts, at $q = 10\%$: $r(0.1, 3) = 1.712\%$ against $r(0.1, 72) \approx 9.35 \times 10^{-34}$ under the model's fixed-hash-share assumption.

Emphasise the caveat: the model has no network. Faster blocks make honest blocks race one another and self-orphan, which effectively raises q; the comparison holds q fixed.

Do not say

A per-minute security claim beyond the board: the comparison holds q fixed, and the model has no network.

Source: *Consensus Guide*, "The catch-up probability"; "Confirmation times at 25-second spacing"

Board 163 — What the numbers say: $r(q, n)$ in percent

not twice

Draw

Five columns; write the rows top to bottom:

q	n=1	n=3	n=6	n=10
2%	4.000	0.016	$5.4 \cdot 10^{-6}$	$1.6 \cdot 10^{-10}$
5%	10.000	0.232	0.0012	$1.2 \cdot 10^{-6}$
10%	20.000	1.712	0.059	0.0008
20%	40.000	11.584	2.331	0.316
30%	60.000	32.616	15.645	6.511
40%	80.000	63.488	49.300	37.218
45%	90.000	81.375	73.375	65.793
50%	100 for every n			

Write

- box the columns $n = 3$ and $n = 10$: deployed wallet defaults, 3 trusted / 10 untrusted
- 10% adversary: 1.712% at three, 0.0008% at ten; 20% adversary: 0.316% at ten
- every fixed count = a (q, ε) policy

Say

Bitcoin says "nothing is special about six"; here is the whole table. The columns $n = 3$ and $n = 10$ are the deployed wallet defaults: three confirmations for trusted outputs, ten for untrusted ones.

Read the wallet rows. Three confirmations assign a 10% adversary 1.712%; ten assign the same adversary 0.0008%, and a 20% adversary 0.316%.

Emphasise: every fixed count is a (q, ε) policy — an assumed adversary and an accepted risk — not a law of nature.

Bitcoin's six assumes an attacker below 10% and a tolerable risk below 0.1%; both figures are arbitrary.

Source: *Consensus Guide*, "Waiting for confirmations"; "Confirmation requirements"; "Wallet confirmation policy"

Board 164 — Pick the adversary and the risk; read off n

not twice

Draw

Four columns; write the rows top to bottom:

q	$r < 10\%$	$r < 1\%$	$r < 0.1\%$
5%	2	3	4
10%	2	4	6
20%	4	8	13
30%	9	20	32
40%	34	82	133
45%	135	331	539

Write

- least n with $r(q, n)$ below target: logarithmic in the inverse risk, explosive in q
- 10% attacker → 6 confirmations; 45% attacker → 539 (3.74 h nominal; ≈ 6.81 h while he withholds)
- no finite count is final: $r(q, n) \geq 2q^n > 0$
- $r(1/2, n) = 1$ for every n: required $n \to \infty$ as $q \uparrow 1/2$

Say

Turn the table round: pick the adversary and the risk, and read off n. The least n pushing $r(q, n)$ below each target is logarithmic in the inverse risk and explosive in q. A 10% attacker is answered by 6 confirmations, a 45% attacker by 539 — 3.74 hours of 25-second blocks nominally, about 6.81 hours while he withholds his blocks.

No finite count is final: $r(q, n) \geq 2q^n > 0$, since the attacker may simply find the first n blocks.

Divergence at the majority line: the required n tends to infinity as $q \uparrow 1/2$, because $r(1/2, n) = 1$ for every n.

Bitcoin: the same table, the same shape; only the wall-clock price of a row differs.

Source: *Consensus Guide*, "Confirmation requirements"

Board 165 — The economics: when the attack pays

not twice

Write

- k merchants at v each; goods worth αv to the attacker; give up after o vain blocks; block income B

$$k\alpha v - (1 - r)(kv + oB) \;=\; kv(\alpha + r - 1) - (1 - r)\, oB$$

$$v \;\leq\; \frac{o\,(1 - r)\,B}{k\,(\alpha + r - 1)} \;\overset{\alpha=1}{=}\; \frac{o}{k} \cdot \frac{B\,(1 - r)}{r}$$

- B is the miner's block income after subsidy and fee allocation; measure exposure in units of B
- Bitcoin (Rosenfeld): $B = 25$ BTC, $o = 20$, $k = 5$, $\alpha = 1$: $v \leq 100\,(1/r - 1)$ BTC; here $\lfloor 4B(1-r)/r \rfloor$

Say

Rosenfeld's profitability question is Bitcoin's; only the cost unit changes. The attacker targets k merchants at v each; the goods are worth αv to him; he gives up after o vain blocks; each of his blocks would have earned the block income B.

Expected profit over not attacking is $k\alpha v - (1-r)(kv + oB)$, non-positive whenever v is at most $o(1-r)B/k(\alpha + r - 1)$; at $\alpha = 1$ that is $(o/k) \cdot B(1-r)/r$.

The cost unit is the miner's block income after subsidy and fee allocation. Exposure scales linearly with B. Expressing the payment in units of B separates the security calculation from the issuance schedule.

Bitcoin: Rosenfeld's $B = 25$ BTC with $o = 20$, $k = 5$, $\alpha = 1$ gives $v \leq 100\,(1/r - 1)$ BTC; the same choices here give $\lfloor 4B(1-r)/r \rfloor$.

Source: *Consensus Guide*, "The economics of the attack"; "Reward-normalised exposure"; protocol specification, "Calculating Block Subsidy, Funding Streams, Lockbox Disbursement, and Founders' Reward"; "ZIP 214 Funding Streams"; ZIP-1016

Board 166 — The largest payment the modelled attack cannot profit from

not twice

Draw

Seven columns; write the rows top to bottom:

q	n=1	2	4	6	8	10
5%	36	271	10,327	344,743	10,931,506	336,737,801
10%	16	67	729	6,759	59,475	508,917
20%	6	15	55	167	467	1,262
30%	2	5	11	21	35	57
40%	1	1	2	4	5	6

Write

- whole block-income units: $\lfloor 4(1-r)/r \rfloor$
- heuristic: unlimited-horizon $r(q, n)$ stands in for the $o = 20$ stopping rule
- 30% attacker, six confirmations: 21 whole units of B

Say

Measure the payment in units of block income. The entries are whole units $\lfloor 4(1-r)/r \rfloor$ — heuristic, as in Rosenfeld's table: the unlimited-horizon $r(q, n)$ stands in for the $o = 20$ stopping rule.

Read one cell. Against a 30% attacker, six confirmations admit 21 whole units of B under this heuristic.

Emphasise: patience buys safety exponentially; the required count grows only logarithmically in the value at stake.

Bitcoin: the same table with $B = 25$ BTC is Rosenfeld's Table 2.

Source: *Consensus Guide*, "Reward-normalised exposure"

Board 167 — Majority is a different problem

not twice

Write

- $q = p$: strict overtake certain, zero drift, no lasting monopoly
- $q > p$: double-spend at no cost beyond normal mining; reject every other miner's blocks; exclude transactions at will
- confirmation policy is not the defence; the cost of assembling the majority is, and it lies outside this model

Say

At $q = p$ strict overtake is certain, but the walk has zero drift; that alone gives no lasting monopoly.

Past $q = p$ the attacker double-spends at no cost beyond normal mining, rejects every other miner's blocks to take the whole issuance, and excludes transactions at will — an attempt to destroy the currency, motivated from outside it.

Emphasise: confirmation policy is not the defence there. The defence is the cost of assembling the majority, which lies outside this model.

Bitcoin: Rosenfeld's remark is about Bitcoin; nothing in it depends on the chain.

Source: *Consensus Guide*, "Reward-normalised exposure"

Board 168 — NU7 rails

not twice

Draw

Three columns, *Rail*, *Value*, *Standing*; write the rows top to bottom:

1. reorg window — 600 blocks (≈ 4.17 h) — ZIP-218 node-policy recommendation
2. coinbase maturity — 100 blocks (≈ 41.7 min), transparent — consensus
3. difficulty — every block; window 102, damping 4, clamp 16%/32% — consensus
4. proof of work — Equihash $(200, 9)$, SHA-256d filter — consensus
5. wallet waits — 3 trusted / 10 untrusted; anchor $\leq H - 3$ — wallet policy

Write

- transparent coinbase outputs, $n = 100$: $r(0.3, 100) \approx 3.7 \times 10^{-9}$, $r(0.4, 100) \approx 0.43\%$, $r(0.45, 100) \approx 15.7\%$
- the window caps automatic rollback, not the theory

Say

Coinbase maturity and the memoryless race are inherited from Bitcoin; per-block retargeting and the block spacing are not.

Walk the table. ZIP-218 recommends a 600-block automatic rollback window, about 4.17 hours; this is node policy, not consensus. The consensus maturity rule for transparent coinbase outputs is 100 blocks, about 41.7 minutes. Difficulty moves every block over a 102-block window with damping 4 and clamps of 16% and 32%. Proof of work is Equihash $(200, 9)$ behind the SHA-256d filter. Wallets wait 3 confirmations for trusted outputs and 10 for untrusted ones, anchoring at or below $H - 3$.

For transparent coinbase outputs, $n = 100$: $r(0.3, 100) \approx 3.7 \times 10^{-9}$, $r(0.4, 100) \approx 0.43\%$, yet $r(0.45, 100) \approx 15.7\%$ — porous near the majority line, as every fixed count must be.

Emphasise: the window caps automatic rollback, not the theory. A fork rooted below it needs operator intervention.

Do not say

Do not present the recommended 600-block window as a consensus rule or an established implementation default.

Source: *Consensus Guide*, "Difficulty adjustment"; "Rollback limits, coinbase maturity, and finality"; "Wallet confirmation policy"; protocol specification, "Transaction Consensus Rules"; "Difficulty adjustment"; "Proof of Work"; draft ZIP-315

Board 169 — NU7: time and state

draft, 23 September 2026

Write

$$\text{target spacing}: \quad 25\,\text{s}$$

$$\text{difficulty window}: \quad 102 \cdot 25 = 2550\,\text{s}$$

$$\text{node expiry default}: \quad 120 \cdot 25 = 3000\,\text{s}$$

Say

The NU7 deployment draft retains transaction formats v5 and v6, rejects v4, and selects the halving-preserving Network Sustainability Mechanism. That mechanism keeps the scheduled subsidy and adds delayed reissuance of funds removed from circulation. The timing and reserve derivations are in the *Consensus Guide*, "NU7: 25-second blocks and bounded shielded work".

Do not say

Do not call the draft active: activation heights remain unassigned. Expiry is a node default (ZIP-218), not consensus; the wallet default is open.

Source: ZIP-259; ZIP-218; ZIP-237

Board 170 — Where the deployed rule departs from the model

not twice

Write

- tie-break: specification, Rosenfeld and Zebra from 6.4.0 = first received; tip hash only when receipt times cannot be ordered
- $r(q, n)$: exact for the strict-overtake model; a lower bound under pre-6.4.0 Zebra's larger-hash tie-break; not a bound on the full deployed process
- Equihash: each candidate solution = a fresh SHA-256d lottery; headers are the model's Bernoulli trials; granularity per solver run against the NU7 target spacing of 25 seconds

Say

Bitcoin's nodes keep the first-seen rule; so does Zebra from 6.4.0. The specification and Rosenfeld break equal-work ties first-seen. Zebra from version 6.4.0 keeps the tip it received first and compares tip hashes only when it cannot order receipt times; earlier versions preferred the larger tip hash.

What the old rule did to r. At equal work an attacker whose private tip hash won could reveal at once; the strict-overtake boundary $z = -1$ does not count that success. So $r(q, n)$ is exact for the strict-overtake model and was a lower bound under the old tie-break — not a bound on the full deployed process, where difficulty moves and rollback is capped.

Equihash and memorylessness. Each candidate solution faces a fresh SHA-256d lottery, so headers are the Bernoulli trials of the model; what Equihash changes is granularity — candidates arrive per solver run, and the exponential clock is good only while a run is short against the NU7 target spacing of 25 seconds.

Emphasise the Bitcoin contrast: its hash trials are the model's ideal.

Do not say

The larger-tip-hash tie-break as current: Zebra dropped it in 6.4.0; the hash now orders only tips whose receipt times it cannot order.

Source: *Consensus Guide*, "Chain selection and the tie-break"; "Equihash and the memoryless model"

Board 171 — What consensus adds for shielded state: the nullifier set

not twice

Write

- "A nullifier MUST NOT repeat either within a transaction, or across transactions in a valid block chain."
- pools' nullifiers "considered disjoint, even if they have the same bit pattern"
- in block order, per transaction: check $\mathsf{nf}_{\text{old}} \notin \text{set} \rightarrow$ record $\mathsf{nf}_{\text{old}} \rightarrow$ append $\mathsf{cmx}_{\text{new}}$
- the one check that cannot sit inside a proof
- sets per pool (ZIP-229): $\rho_{\text{new}} := \mathsf{nf}_{\text{old}}$ unique pool-locally, not globally

Say

In Bitcoin the node looks the outpoint up in the UTXO set, an ordered, stateful check against the whole history. Here the note is never named, so the node checks a serial number instead.

The specification's rule, verbatim: "A nullifier MUST NOT repeat either within a transaction, or across transactions in a valid block chain." Each pool's nullifiers are "considered disjoint, even if they have the same bit pattern".

The check is stateful and ordered. Transactions apply in block order; each nf_{old} is tested against the set as it stands, then recorded, then each $\mathsf{cmx}_{\text{new}}$ is appended. The next spender proves membership against the new root; a second spend of the same note fails.

Emphasise: this is the one check that cannot sit inside a proof, because it concerns the global history, not one transaction. Sets are per pool (ZIP-229), so uniqueness of $\rho_{\text{new}} := \mathsf{nf}_{\text{old}}$ is pool-local, not global.

Bitcoin: the UTXO set is the same kind of ordered stateful check; here the set records serial numbers that name no output.

Do not say

Calling ρ a tree position: it is a per-note uniqueness seed, set at creation as $\rho_{\text{new}} := \mathsf{nf}_{\text{old}}$.

Source: protocol specification, "Nullifier Sets"; *Ironwood Guide*, "Verification of a transaction"; "Nullifier sets"

Board 172 — Anchors, balances, and no mempool chains

exists, conserved

Write

- anchor "MUST refer to some earlier block's final" treestate of its pool; never mid-block; any main-chain block since the pool's activation; $H - 3$ is wallet policy
- no spend of an unmined note: not an output of its own transaction; not under any earlier block's final root
- Bitcoin: mempool chains spend unconfirmed outputs; here only the transparent layer keeps that
- specified route: ZIP-374 deferrable v6 anchors — sign now, fill the anchor after mining; signatures commit to neither anchor nor witness
- balances: each pool's chain value balance public; a block that would drive one negative is invalid (ZIP-209); every crossing = a public row

Say

Anchors. The bundle's anchor "MUST refer to some earlier block's final" treestate for its pool — never a mid-block state, and any main-chain block since the pool's activation qualifies; the depth $H - 3$ is wallet policy.

No spend of an unmined note. An Action cannot spend an output of its own transaction, and a note mined in this block is under no earlier block's final root. Bitcoin lets a mempool chain spend unconfirmed outputs; a shielded note cannot be spent until it is mined (the transparent layer keeps Bitcoin's rule).

The specified route. ZIP-374 makes v6 anchors deferrable: sign a spend of a not-yet-mined note now, fill the anchor after the note is mined; signatures commit to neither anchor nor witness.

Balances. Each pool's chain value balance is public and a block that would drive one negative is invalid (ZIP-209); every crossing writes a public row.

Do not say

"The pool is auditable": what is public is each pool's chain value balance and the crossings, and a block that would drive a balance negative is invalid; nothing inside the pool is audited. Deferred anchors as deployed: ZIP-374 is the specified route.

Source: protocol specification, "Transactions and Treestates"; "Action Transfers and their Descriptions"; *Ironwood Guide*, "Anchors"; "Chain state and pool rules"; *Wallet Guide*, "PCZT v2: deferred anchors and the Ironwood bundle"

Board 173 — Reorganisation of shielded state

exists, not twice

Write

- trees, nullifier sets, pool balances = chain state; a reorg rolls all three back with the blocks
- reorg reaches the anchoring block ⇒ anchor struck, transaction with it

- rebuild spends the same notes; nullifier derivation is deterministic ⇒ the same nullifiers again
- Bitcoin: re-broadcast unchanged; here: rebuild the proof against a surviving root

Say

In Bitcoin a transaction struck by a reorganisation is simply re-broadcast unchanged. Here, once the fork reaches the anchoring block, the proof must be rebuilt against a surviving root.

Trees, nullifier sets, and pool balances are chain state; a reorganisation rolls all three back together with the blocks.

A reorganisation reaching the anchoring block strikes the anchor and the transaction with it. The rebuild spends the same notes, and nullifier derivation is deterministic, so it re-reveals the same nullifiers (the rebuild, truncate-and-rescan, and ZIP-374's re-anchoring: Part C).

Emphasise: the rebuild spends the same notes, so the same nullifiers reappear — say that, and no more.

Ask

A merchant sees Alice's shielded payment at one confirmation. Which of the two double-spends is she exposed to, and what does the catch-up theorem say about waiting?

Answer. Not the same-chain one: a second spend of Alice's note reveals the same nullifier, and the set rejects it by rule, at any depth. The chain-level one: a secret branch may replace the block she saw. At $n = 1$ a 20% adversary succeeds in 40% of runs; each added confirmation drops the risk by a roughly constant factor, and $r(0.2, 10) = 0.316\%$. Waiting works in blocks, not minutes; it never reaches zero, and it stops working at $q = 1/2$.

Do not say

"The attempts are linked" as a volume fact; and same-txid re-anchoring as deployed — it is specified (ZIP-374 v2), wallet support pending.

Source: *Ironwood Guide*, "Anchors"; *Wallet Guide*, "Reorganisation detection and recovery"; protocol specification, "Transactions and Treestates"; "Nullifier Sets"; *Consensus Guide*, "The catch-up probability"; "Confirmation requirements"

11 Part I — Limits, frontier, takeaways

Board 174 — Limits: what the theorem leaves public

none — privacy of the graph

Write

1. Heading: *hidden* — note contents; the spend-to-note link.
2. Heading: *public* — public balance; component and Action counts; any transparent leg; timing; address reuse; voluntary disclosures; wallet and network metadata.
3. Alice's bundle, in the clear: 2 Actions, 1 anchor, 2 nullifiers, 2 commitments, valueBalance = +10 000 zatoshi = the fee.
4. Below: "candidate set = the tree under the anchor; effective set ≤ that".

Say

The end-to-end theorem says what is hidden, and nothing more. It hides the note contents and the link from a spend to the note it consumes. Everything else on the transaction is an envelope the theorem never claimed to seal.

Read the envelope off Alice's bundle. Two Actions, one anchor, two nullifiers, two commitments, and a valueBalance of +10 000 zatoshi. That last number is her fee, in the clear: a fully shielded transaction still publishes what it paid the miner.

The anonymity set is smaller than the tree. The candidate set is every leaf under the anchor, but leaves publicly known to be dummy or spent, and other transaction metadata, shrink the effective set; tree size alone does not measure it. And an idiosyncratically old anchor fingerprints the spender.

The contrast. In Bitcoin the graph itself is published. Here the graph is gone and the envelope stays: counts, fee, anchor, timing.

Do not say

"The proof hides the sender and receiver": the proof adds nothing beyond the statement; the hiding comes from commitment hiding, the keyed nullifier PRF and key re-randomisation. "The anonymity set is the tree": the tree is only the candidate set.

Source: *Ironwood Guide*, "The end-to-end theorem"; "Privacy"; "Anchors"

Board 175 — Limits: the light client's server

none — privacy of the scan

Write

1. Goal (ZIP-307): "an honest-but-curious server must not learn which transactions are a wallet's".
2. "Keys never travel; requests do."
3. Leak 1: fetch-on-detect names a txid.
4. Leak 2: the submission channel links the transaction to its origin.
5. Bitcoin column: scan < BIP-157; fetch after a hit > BIP-157 (one txid, not a block).

Say

The stated goal is one sentence. ZIP-307 asks that an honest-but-curious server not learn which transactions are a wallet's. The keys never leave the device; the requests do, and the requests are the leak. The full inventory is in the delivery part; two leaks survive the shipped mitigations.

Leak one: the fetch after a hit. A scan is issued with exact range bounds and no per-block match, so the scan itself reveals only its bounds. But the memo is not in the compact block, so after a hit the wallet fetches the full transaction — and that request names exactly one txid. ZIP-307 asks for decoy fetches as a SHOULD; the reference library does not supply them.

Leak two: the submission channel. A wallet that broadcasts through the same server hands it the whole transaction before the network sees it, tied to the session that scanned. Circuit isolation exists in the client library; whether any deployment uses it is unknown.

The contrast runs both ways. The scan leaks less than a BIP-157 client, which names each block it fetches after a filter hit. The fetch after a hit leaks more: one txid, not a block.

Source: *Wallet Guide*, "Payment-detection privacy"; "Leakage of range queries"; "Leakage of transaction queries"; "The submission channel"; "Summary of server observables"

Board 176 — Limits: custody is a key

authorised

Write

1. The note: $(\mathsf{addr}, v, \rho, \psi, \mathsf{rcm})$ — "no script".
2. Spending condition: knowledge of ask, proved through rk (A6, A7).
3. Seed S: 32 to 252 bytes, $\geq$ 256 bits of entropy (ZIP-32).
4. Wallet layer: FROST = t-of-n over ask; PCZT = signer holds ask, prover computes the proof.

Say

A note carries no script. Its five fields are an address, a value, a per-note uniqueness seed and two pieces of randomness. The only spending condition is knowledge of ask, proved through rk by clauses A6 and A7. Whoever holds ask spends, and nothing below it on the ladder recovers it.

Every key descends from one seed. ZIP-32 requires 32 to 252 bytes with at least 256 bits of entropy. A weak seed is a weak link for every key at once, and a brute-force search over weak seeds runs against many wallets simultaneously.

What script gave Bitcoin moves into the wallet layer. Threshold authority is FROST: t-of-n over ask, and the verifier is untouched. Hardware and offline signers use PCZT: a signer holds ask but cannot compute a Halo 2 proof, so proving is a separate role.

The contrast. Bitcoin's spending conditions live in script and every validator runs them — multisig, timelocks, hash locks. Here the condition is fixed by the circuit and discharged by one signature.

Do not say

"No script means no multisig and no hardware wallets": FROST and PCZT answer both, in the wallet layer. The re-randomisation behind rk is additive, never $[\alpha] \cdot \mathsf{ak}$.

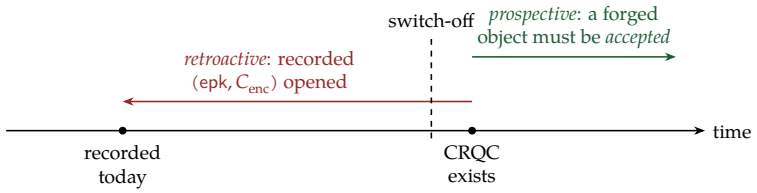
Source: *Ironwood Guide*, "Notes"; "Key capabilities and scope"; *Wallet Guide*, "Wallet seeds"; "Partially created transactions (PCZT)"; *FROST Guide*, "Scope"

Board 177 — Quantum: two kinds of break

privacy, then all four

Draw

1. A long horizontal arrow, left to right; label the head "time".
2. Two dots on it: one a sixth of the way along, labelled "recorded today"; one two thirds along, labelled "CRQC exists".
3. Just above the axis, a red arrow from the CRQC dot pointing *left* back to the first dot; label it "retroactive: recorded $(\mathsf{epk}, C_{\mathsf{enc}})$ opened".
4. Higher up, a green arrow from the CRQC dot pointing *right* towards the end of the axis; label it "prospective: a forged object must be *accepted*".
5. A dashed vertical line just left of the CRQC dot, labelled "switch-off" at its top.



Write

1. "CRQC = a quantum computer running Shor at scale: discrete logs in polynomial time; every group assumption falls."
2. Under the red arrow: "exploitable the day the adversary exists".
3. Under the green arrow: "a validator must *accept* the forgery; switch off first".
4. Bottom line: "Orchard, both pools: one retroactive break".

Say

Start from the adversary, not the calendar. A CRQC runs Shor's algorithm at scale and takes discrete logarithms in polynomial time. Every group assumption in this workshop falls together, so asking *whether* is unproductive; the useful axis is reversibility.

Retroactive breaks act on data already recorded. The red arrow runs backwards: what is on chain today is exploitable the day the adversary exists, and no later consensus change can help. Prospective breaks need a validator to *accept* a forged object; switching the mechanism off before that day removes acceptance. That is the dashed line.

Orchard, in both pools, has one retroactive break — the note encryption, next board. Everything else on the integrity stack is prospective.

The contrast. Bitcoin has no ciphertext to open, but a silent-payment scan key has the same retroactive shape, and an exposed public key is the prospective case.

Do not say

Any date for the CRQC or for switch-off: the axis is an ordering, not a calendar.

Source: *Crypto Guide*, "Quantum adversaries: Shor's algorithm"; ZIP-2005, "Effects of discrete-logarithm-breaking attacks before the switch to the Recovery Protocol"; "Effects of discrete-logarithm-breaking attacks after the switch to the Recovery Protocol"

Board 178 — Retroactive: harvest now, decrypt later

none — privacy of the note

Write

1. Published per Action: $(\mathsf{epk}, C_{\text{enc}})$.
2. The one discrete-log link: Diffie–Hellman between esk and $\mathsf{pk_d}$; "KDF and AEAD above it are symmetric".
3. The attack, as one line:

$$\mathsf{ivk} = \log_{\mathsf{g_d}} \mathsf{pk_d} \quad \Longrightarrow \quad K = \mathrm{KDF}([\mathsf{ivk}]\,\mathsf{epk}, \ldots)$$

4. "One logarithm per ivk scope, then the recipient's own trial-decryption loop over the recorded chain."
5. Recovered per note: $(d, v, \mathsf{rseed}, \mathsf{memo})$ — past and future.
6. Example: Bob's 1.5 ZEC and the memo.

Say

Each Action publishes an ephemeral key and a ciphertext. Their confidentiality rests on exactly one discrete-log link, the Diffie–Hellman between esk and $\mathsf{pk_d}$. The KDF and the AEAD above it are symmetric and are not the weak layer.

The attack is one logarithm and then honest work. An adversary holding a recipient's address computes $\mathsf{ivk} = \log_{\mathsf{g_d}} \mathsf{pk_d}$ once — one logarithm for the lifetime of that ivk scope — and then runs the recipient's own trial-decryption loop over the recorded chain. For every Action, $[\mathsf{ivk}]\,\mathsf{epk}$ feeds the KDF. Out comes every $(d, v, \mathsf{rseed}, \mathsf{memo})$ ever sent to any diversified address sharing that ivk, past and future alike.

Bring it back to the example. Bob's address went to Alice. Whoever else holds that address can one day read the 1.5 ZEC and the memo. Storage today, the logarithm later: harvest now, decrypt later.

Do not say

"The solved ivk yields dk and ovk": those are derived from fvk, not from ivk. Any claim that the KDF or the AEAD is the weak layer.

Source: *Ironwood Guide*, "Encryption to the recipient"; "Trial decryption and note acceptance"; ZIP-2005, "Effects of discrete-logarithm-breaking attacks after the switch to the Recovery Protocol"

Board 179 — Retroactive: two limits, and the bin

none — privacy of the note

Write

1. Limit 1: "ivk views, never spends; ask not exposed".
2. Limit 2: "keyed by the address: no $(d, \mathsf{pk_d})$, no base $\mathsf{g_d}$, no target $\mathsf{pk_d}$".
3. Bin: *open problem* — "nothing deployed or specified protects even future ciphertexts (ZIP-2005 non-requirement); recorded ones unrepairable".
4. Bitcoin column: silent payments — one logarithm of the scan key replays the scan; amount and memo never hidden.

Say

Two limits bound the damage. First, ivk is a viewing capability: no spend authority is conferred, and ask is not exposed — its exposure is the prospective case. Second, the attack is keyed by the address. Without $(d, \mathsf{pk_d})$ the adversary has neither the base $\mathsf{g_d}$ nor the target $\mathsf{pk_d}$, and no known efficient generic route recovers the plaintext from the ciphertext alone.

The bin is open problem. Nothing deployed and nothing specified protects even future ciphertexts; ZIP-2005 excludes privacy by an explicit non-requirement. The recorded ciphertexts are unrepairable by construction: any upgrade alters only ciphertexts not yet created.

The contrast. Bitcoin has no ciphertext to open, but silent payments carry the same shape: one logarithm of the published scan key replays the recipient's scan over recorded history and links everything that address ever received. There is no analogue for the amount and the memo, because Bitcoin never hid them.

Do not say

"ZIP-2005 fixes this": it repairs binding, not privacy. Deriving $(\mathsf{dk}, \mathsf{ovk})$ from ivk.

Source: *Ironwood Guide*, "Key capabilities and scope"; ZIP-2005, "Motivation"; "Non-requirements"

Board 180 — Prospective: the discrete-log integrity stack

all four

Draw

Two columns, "construction" and "with the generators' logarithms known"; write the rows in this order:

1. Sinsemilla binding ($\mathsf{NoteCommit}$, $\mathsf{Commit}^{\mathsf{ivk}}$): a second note under an on-chain cmx — balance breaks silently, turnstile-bounded; a second $(\mathsf{ak}, \mathsf{nk})$ for the same ivk — a spendability forgery.
2. Value commitment, binding signature: any cv reopens to any value; a solved bvk forges the signature — inflation, turnstile-bounded.
3. Spend authorisation (RedPallas): one logarithm of ak or of any published rk forges spend authority; theft still needs a witness or a proof forgery.
4. Proof soundness (IPA on Vesta): commitment equivocation; the knowledge-soundness reduction no longer holds.

Write

Under the table: "recorded cv perfectly hiding; α masks rk; transcripts simulatable; without the address, no generic route to these secrets".

Say

Shor invalidates the reductions; it does not turn them into attacks by itself. Each construction gets its own concrete consequence, so take the four rows one at a time.

Sinsemilla loses binding first. With the generators' logarithms known, a second note opens under an on-chain cmx: balance breaks silently, bounded only by the turnstile. A second $(\mathsf{ak}, \mathsf{nk})$ pair for the same ivk is a spendability forgery.

Value commitments and the binding signature follow. Any cv reopens to any value, and a solved bvk forges the binding signature: inflation, again turnstile-bounded. For spend authorisation, one logarithm of ak or of any published rk forges authority — but theft still needs a witness or a proof forgery. For proof soundness, the IPA commitment can be equivocated and the knowledge-soundness reduction no longer holds.

What the recorded fields still do not give away. Recorded cv values stay perfectly hiding, α masks rk, and transcripts are simulatable. Without the address, no generic route pulls these secrets from the recorded fields.

Do not say

"Turnstile-bounded means detected": a counterfeit inside a pool is undetectable; the turnstile caps what can leave. "The pool is auditable."

Source: *Ironwood Guide*, "The end-to-end theorem"; ZIP-2005, "Effects of discrete-logarithm-breaking attacks before the switch to the Recovery Protocol"

Board 181 — Why the integrity breaks are prospective

all four

Write

1. "A forged object — proof, signature, commitment opening — must be *accepted*, at current height under current rules."
2. "Switch off before the CRQC ⇒ no acceptance; history anchored by chain work and upgrade policy."
3. Residue 1: switch-off must precede the *first* forgery.
4. Residue 2: honest funds inside a disabled pool need a way out ⇒ ZIP-2005.

Say

Every integrity attack has to be accepted by a validator. A forged proof, signature or commitment opening is worthless until a node takes it, and acceptance happens at current height under current rules. That is what makes the four rows prospective.

So switching off first works. Disabling the discrete-log pools before a CRQC exists removes acceptance of new transactions under those rules; settled history stays anchored by chain work and upgrade policy.

Two residues. The switch-off must precede the *first* forgery — an attack landed inside the window is not repaired afterwards. And honest funds inside a disabled pool need another way out. That second residue is the problem ZIP-2005 exists to solve.

The contrast. Bitcoin has the same shape: ECDSA and Schnorr fall to one logarithm of an exposed public key, and the forged spend must still be mined.

Do not say

A schedule for the switch-off: none is specified.

Source: *Crypto Guide*, "Quantum adversaries: Shor's algorithm"; ZIP-2005, "Abstract"; "Effects of discrete-logarithm-breaking attacks before the switch to the Recovery Protocol"

Board 182 — The symmetric backbone survives

authorised

Write

1. BLAKE2b: PRFexpand (every key and note-randomness derivation), KDF, outgoing cipher key, RedPallas challenge, ZIP-244 digests; ZIP-32 Orchard tree hardened-only.
2. Poseidon: the nullifier PRF, keyed by nk — a heuristic, classically and quantumly alike.
3. Grover:
$$2^{256} \text{ classical evaluations } \longrightarrow \approx 2^{128} \text{ quantum ones}$$
4. "Best implementable generic collision attack: still the classical parallel one."
5. Bottom line: "ownership stays *provable* after it stops being *enforceable*".

Say

Two primitive families carry no discrete-log structure. BLAKE2b serves PRFexpand, hence every key and note-randomness derivation, the KDF, the outgoing cipher key, the RedPallas challenge and the ZIP-244 digests; the ZIP-32 Orchard tree is hardened-only and built entirely from it. Poseidon serves the nullifier PRF keyed by nk, a cryptanalytic heuristic classically and quantumly alike.

Only generic speedups apply. Grover halves preimage security: 2^{256} classical evaluations become about 2^{128} quantum ones. The best implementable generic collision attack remains the classical parallel one.

The load-bearing consequence. A seed holder can still prove, by re-executing the hardened derivations, that their keys descend from their seed. Ownership stays provable after the assumption that made it enforceable has fallen. That is the fact ZIP-2005 builds on.

The contrast. SHA-256 and hardened BIP-32 derivation stand on the same ground.

Source: *Crypto Guide*, "Quantum adversaries: Shor's algorithm"; ZIP-2005, "Adaptation to the quantum setting"; "Attacks addressed by the Recovery Protocol"

Board 183 — ZIP-2005: the hashed trapdoor

exists

Write

1. The Ironwood derivation, written out in full:
$$\mathsf{rcm} := \mathrm{ToScalar}(\mathsf{PRFexpand}_{\mathsf{rseed}}(\mathtt{0x0B} \,\|\, \mathsf{g_d}^{\star} \,\|\, \mathsf{pk_d}^{\star} \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{256}(\rho) \,\|\, \mathrm{LE}_{256}(\psi)))$$
2. Under it: "137-byte preimage — every note field traverses BLAKE2b into the trapdoor".
3. "Under a CRQC, NoteCommit alone is not binding."
4. "PRFexpand as a random oracle: two honest notes on one commitment = a collision event; bound $O(q^3/N)$, about 2^{-74} at $q = 2^{60}$."
5. "Post-quantum binding *provided a verifier checks the derivation* — consensus does for coinbase outputs only."
6. Legacy 0x02 notes: $\mathtt{0x05} \,\|\, \rho$ only ⇒ unrecoverable ⇒ one reason for the seal.

Say

Recall the one note-level difference between the pools. An Ironwood note derives its trapdoor rcm by hashing every note field: a 137-byte preimage through BLAKE2b, tagged 0x0B.

Why that matters under a CRQC. The commitment NoteCommit alone is not binding once Sinsemilla's collision resistance falls, and it is the first thing to fall. Model PRFexpand as a random oracle: opening one commitment to two honestly derived notes becomes a collision-type event with no group structure for Shor to exploit. The bound is $O(q^3/N)$ in the QROM, about 2^{-74} at $q = 2^{60}$ queries.

The binding is conditional. The composite is post-quantum binding *provided a verifier checks the derivation*. No Action clause does — the circuit checks the commitment, not the derivation behind its trapdoor. That check is the Recovery Statement's job.

Legacy notes hash only $0\text{x}05 \| \rho$. They are unrecoverable, and that is one reason the Orchard pool is sealed.

The contrast. Bitcoin ownership is a public key exposed at spend. The hardened derivation behind it is symmetric too, but no rule reads it; the exposed key alone decides a spend.

Do not say

Treating ρ in the preimage as a tree position: it is the per-note uniqueness seed. "ZIP-2005 makes Orchard quantum-secure."

Source: *Ironwood Guide*, "The note seed: Ironwood derivations (lead byte 0x03)", Remark "Purpose of the hashed trapdoor"; "The Action statement", Remark "What the statement does not check"; "Chain state and pool rules"; ZIP-2005, "QROM lift of the security arguments"

Board 184 — Recoverability, not resistance

exists; authorised

Write

1. Heading: "Recovery Statement — proved one day, post-quantum proof system, re-hashed tree".
2. The chain of claims, top to bottom: key derivations from sk; $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}$; $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$; ψ and rcm derivations; the commitment; a membership path; the nullifier; the α re-randomisation.
3. Cost: 7 BLAKE2b-512 compressions, 1 BLAKE3 compression, 2 Sinsemilla commitments, 2 Pallas scalar multiplications, a Merkle path.
4. Box: "binding only; privacy unchanged; opens no ciphertext".

Say

The Recovery Statement is what a holder would one day prove. In a post-quantum proof system, over a re-hashed tree, the holder shows the whole chain: the key derivations from sk, $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}$, $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$, the ψ and rcm derivations, the commitment, a membership path, the nullifier, and the α re-randomisation.

The cost is mostly hashing. Seven BLAKE2b-512 compressions, one BLAKE3 compression, two Sinsemilla commitments, two Pallas scalar multiplications and a Merkle path.

Binding only, privacy unchanged. The statement proves, with no group assumption, that one seed derives a committed note, its nullifier and its rk. It opens no ciphertext; the harvested corpus is untouched by it.

Do not say

"Quantum resistance": the strategy is recoverability. The statement does not exist as a deployed proof; it is what would be proved.

Source: *Ironwood Guide*, "The note seed: Ironwood derivations (lead byte 0x03)", Remark "Purpose of the hashed trapdoor"; ZIP-2005, "Abstract"; "Proposed Recovery Statement"; "Cost"

Board 185 — ZIP-2005: deployed, specified, designed, open

exists; authorised

Draw

Four rows, one bin each, written top to bottom:

1. *specified and deployed*: the 0x03 derivation and the three seal rules — every Ironwood output is recoverable.
2. *specified, undeployed*: the quantum-key path (qsk, qk); no consensus rule.
3. *designed, unspecified*: the Recovery Protocol — proof system, collapsing tree hash, linking commitments all unfixed.
4. *open*: post-quantum proofs and signatures for ongoing spends; the switch-off schedule.

Write

Under the rows: "Sprout, Sapling, Orchard-pool notes: unrecoverable ⇒ sweep into Ironwood, ongoing; change already routed there". Then: "the harvested corpus accrues now".

Say

Sort ZIP-2005 into the bins. Specified and deployed: the 0x03 derivation and the three seal rules, so every Ironwood output is recoverable. Specified but undeployed: the quantum-key path, qsk and qk, with no consensus rule behind it. Designed but unspecified: the Recovery Protocol — proof system, collapsing tree hash and linking commitments all unfixed. Open: post-quantum proofs and signatures for ongoing spends, and the switch-off schedule.

The consequence is forced migration. Sprout, Sapling and Orchard-pool notes are unrecoverable, so ZIP-2005 tells wallets to sweep everything into Ironwood as an ongoing process. Deployed today, change is already routed there.

The one cost the strategy cannot answer. The harvested corpus accrues now.

Do not say

A horizon for the switch-off: the schedule is an open problem. "Fully post-quantum": binding is repaired, privacy is not.

Source: *Ironwood Guide*, "The note seed: Ironwood derivations (lead byte 0x03)", Remark "Purpose of the hashed trapdoor"; "The spending key and the spend-side secrets"; "Chain state and pool rules"; ZIP-2005, "Proactive movement of funds to recoverable notes"; "Proposed Recovery Protocol"

Board 186 — Beyond the core: what each one adds

all four unchanged

Draw

Three columns — name, what it adds, Bitcoin — written row by row:

1. FROST: *t*-of-*n* custody of ask; the aggregate is an ordinary spend-auth signature under rk — a Taproot FROST suite.
2. FlyClient: logarithmic chain proofs, difficulty-weighted sampling over a Merkle mountain range — SPV fetches every header.
3. ZSA: issued assets, per-asset notes and value commitments, issuance, burn — Elements' issued assets.
4. Crosslink: proof of work kept, plus a trailing BFT finality layer cross-linked with it — confirmations, never finality.

Write

Under the table: "none of the four adds privacy: custody, chain verification, assets, settlement — hiding unchanged".

Say

Four extensions, one row each. FROST gives *t*-of-*n* custody of ask; the aggregate is an ordinary spend-auth signature under rk, and the verifier is untouched. FlyClient gives logarithmic chain proofs by difficulty-weighted sampling over a Merkle mountain range. ZSA gives issued assets: per-asset notes and value commitments, issuance and burn. Crosslink keeps proof of work and adds a trailing BFT finality layer cross-linked with it.

Each has a Bitcoin neighbour. A Taproot FROST suite; SPV, which fetches every header; Elements' issued assets; and confirmations, which are never finality.

The one thing to emphasise. None of the four adds privacy. They add custody, chain verification, assets and settlement to a core whose hiding is unchanged.

Do not say

A multiplicative re-randomisation in the FROST row: the aggregate sits under the same additive rk.

Source: *FROST Guide*, "Scope"; *FlyClient Guide*, "Classification of the security hypotheses"; "Header-only verification"; *ZSA Guide*, "Epistemic classes"; *Crosslink Guide*, "Scope and status"

Board 187 — Beyond the core: the bins

all four unchanged

Draw

Two columns — name, bin as the volume states it — row by row:

1. FROST: *specified* — RFC 9591 and ZIP-312 (Draft); shipped as a library; no consensus rule names it; key generation in no ZIP.
2. FlyClient: commitment *deployed and consensus-critical* (ZIP-221, its root committed in every header after Heartwood's activation block); proofs *unbuilt* — no server serves one, no client asks; the Zcash instantiation designed, not specified.
3. ZSA: core *specified* in Draft ZIPs 226 and 227; the carrier — v6 wire format, digests, lead byte, fees — *unspecified* since the ZIPs that held it were withdrawn; outside ZIP-259's NU7 scope.
4. Crosslink: *design stage* — no ZIP, nothing activated on mainnet or testnet; a feature net runs prototype code; construction rules *designed but unspecified*.

Write

Under the table, the three bins: *specified* / *designed but unspecified* / *open problem*; "deployed behaviour cites the implementation and the specification instead".

Say

Same four rows, now sorted by what actually exists. FROST is specified — RFC 9591 and ZIP-312, a Draft — and shipped as a library; no consensus rule names it, and key generation is in no ZIP. FlyClient's commitment is deployed and consensus-critical: ZIP-221's root sits in every header after Heartwood's activation block. Its proofs are unbuilt — no server serves one, no client asks — and the Zcash instantiation is designed, not specified.

ZSA and Crosslink are further out. The ZSA core is specified in Draft ZIPs 226 and 227, but the carrier — v6 wire format, digests, lead byte, fees — is unspecified since the ZIPs that held it were withdrawn, and ZIP-259 does not include it in NU7. Crosslink is design stage: no ZIP, nothing activated on mainnet or testnet, a feature net running prototype code, construction rules designed but unspecified.

The bins are the volumes' method. Specified, designed but unspecified, open problem — and deployed behaviour cites the implementation and the specification instead.

Do not say

"FlyClient is deployed": the commitment is, the proofs are unbuilt. Any activation height.

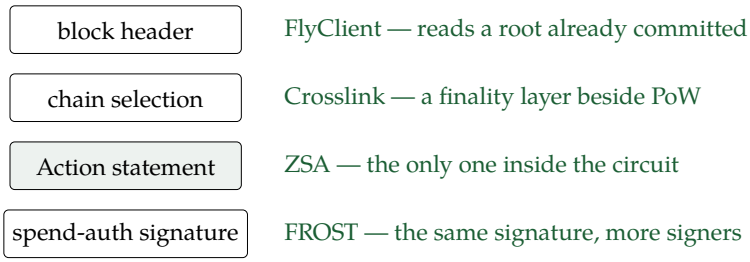
Source: *FROST Guide*, "Sources and their standing"; *FlyClient Guide*, "Classification of the security hypotheses"; "The specified commitment and its consumers"; *ZSA Guide*, "Scope and status"; "Epistemic classes"; *Crosslink Guide*, "Epistemic classes"; "Scope and status"

Board 188 — Where each one sits

all four unchanged

Draw

1. Four boxes stacked vertically, top to bottom: "block header", "chain selection", "Action statement", "spend-auth signature". Shade the third box.
2. To the right of each box, a tag: FlyClient — reads a root already committed; Crosslink — a finality layer beside PoW; ZSA — the only one inside the circuit; FROST — the same signature, more signers.



Write

Beside the shaded box: "ZSA only: Asset Base witnessed once; A1, A2 on a per-asset commitment; A4 by variable-base multiplication; A3 dummy skip kept for the native asset alone; split notes". Under the stack: "the other three leave the ten clauses as they are".

Say

Place each extension on the stack you already know. FlyClient reads a root the header already commits. Crosslink adds a finality layer beside proof of work, at the chain-selection layer. FROST produces the same spend-auth signature with more signers. Only ZSA sits inside the circuit.

What ZSA changes in the statement. An Asset Base witnessed once; A1 and A2 on a per-asset commitment; A4 by variable-base multiplication; the A3 dummy skip kept for the native asset alone; and split notes.

The one thing to emphasise. The other three leave the ten clauses exactly as they are.

Source: *ZSA Guide*, "The OrchardZSA Action statement"; *FROST Guide*, "Scope"; *FlyClient Guide*, "Classification of the security hypotheses"; *Crosslink Guide*, "Scope and status"

Board 189 — Takeaways: four checks, four mechanisms

all four

Draw

Three columns — check, Bitcoin, Zcash — one row per check, in the order of the spine:

1. coin exists: UTXO lookup — a commitment in an append-only tree; membership proven under an anchor, the position a private witness (A3).
2. authorised: script and signature — the circuit ties rk to ak (A6) and ak, nk to the note's address (A7); the node's signature check under rk proves $\mathsf{rsk} = \mathsf{ask} + \alpha$.
3. not spent twice: the UTXO is deleted — the nullifier, keyed by nk and seeded by ρ: revealed once, rejected on repeat, unlinkable to cm (A5).
4. conserved: $\sum$ in $\geq \sum$ out in the clear — homomorphic value commitments (A4); the binding signature proves knowledge of $\sum$ rcv; fully shielded, valueBalance is the fee.

Write

Under the table: "a stranger still performs all four; the note is never seen".

Say

Close on the spine. Four checks a Bitcoin node makes by looking at the coin; four mechanisms that let a stranger make the same checks without ever seeing the note.

Existence and authorisation. The coin is a commitment in an append-only tree, membership proven under an anchor with the position a private witness. Authorisation is the circuit tying rk to ak and ak, nk to the note's address, plus the node's signature check under rk, which proves $\mathsf{rsk} = \mathsf{ask} + \alpha$.

Double spend and conservation. The nullifier, keyed by nk and seeded by ρ, is revealed once, rejected on repeat and unlinkable to cm. Conservation is homomorphic value commitments and a binding signature proving knowledge of $\sum$ rcv; fully shielded, valueBalance is the fee.

The one sentence to leave on the board. A stranger still performs all four; the note is never seen.

Do not say

A multiplicative form, $\mathsf{rsk} = \alpha \cdot \mathsf{ask}$ or $\mathsf{rk} = [\alpha]\,\mathsf{ak}$: the re-randomisation is additive. Describing ρ as the leaf position: the position is a private witness of A3 and never enters the nullifier.

Source: *Ironwood Guide*, "Requirements on a shielded payment"; "Verification of a transaction"; "The Action statement"

Board 190 — Where to read more: the Zcash Arboretum

all four

Write

1. Foundations: *Math Guide* (fields, curves, hardness); *Crypto Guide* (the toolbox's primitives); *Halo 2 Guide* (the proof system, in full).
2. Deployed protocol: *Consensus Guide* (the consensus part); *Ironwood Guide* (the spine of this workshop); *Wallet Guide* (keys, fees, the pipeline, and what the server learns).
3. Frontier: *FlyClient*, *ZSA*, *Crosslink* and *FROST Guides*, each with its bins visible.
4. Bottom line: "all non-normative — the protocol specification and the ZIPs are authoritative".

Say

Three shelves. Foundations for the mathematics, the primitives and the proof system in full; deployed protocol for consensus, the Ironwood spine of this workshop, wallets and the light-client server; frontier for the four extensions, each with its bins visible.

None of it is normative. The protocol specification and the ZIPs are authoritative, and every volume cites the section it draws on.

12 Backup — optional boards

Every board in this section is optional: reach for one when a question calls for it, and skip the section otherwise. No board here carries a figure; three carry a table, given as rows in writing order.

Board 191 — Sources — the volumes, the specification, the ZIPs

optional

Write

- Volumes (non-normative): *Ironwood* (the shielded critical path); *Halo 2*, *Wallet*, *Crypto*, *Consensus*, *Math*; one row each from *FROST*, *FlyClient*, *ZSA*, *Crosslink*.
- Normative: the protocol specification; the ZIPs.
- Final or Active: 32 keys, 203 expiry, 209 pool balances, 213 shielded coinbase, 221 history tree, 244 digests, 316 unified addresses, 317 fees.
- Draft: 229 v6 format (the two pools), 258 the Ironwood upgrade, 307 compact blocks, 312 FROST, 315 wallet practice, 318 migration, 326 wallets after the seal, 374 PCZT.
- Proposed: 2005 quantum recoverability. Reserved stub: 2006 the cross-address restriction.
- Bins: *specified* / *designed but unspecified* / *open problem*.

Say

Everything in this workshop traces to a volume, and every deployed claim to the specification or a ZIP. The volumes are non-normative: the Ironwood Guide carries the shielded critical path; the Halo 2, Wallet, Crypto, Consensus and Math Guides carry the rest; the FROST, FlyClient, ZSA and Crosslink Guides each supplied one row.

Normative means the protocol specification and the ZIPs. Read the three ZIP lists off the board. One footnote on 318, migration: it was a Reserved stub at the pinned zips checkout, `69610984`, and a Draft at the recheck, `dd1667ff`.

The rule for every slide you saw: a deployed claim cites a specification section or a ZIP; a design-stage claim carries its bin — specified, designed but unspecified, open problem.

The Bitcoin contrast: BIPs, and no specification. In Bitcoin the reference client's behaviour is the definition. Here the specification is a document, and a node implements it.

Ask

What is normative here? The specification and the ZIPs; the volumes are not.

Source: *Ironwood Guide*, "Scope and status"; *Consensus Guide*, "Sources and their standing"

Board 192 — Implementation snapshots, as the volumes cite them

optional

Draw

1. Two columns, headed *Volume* and *Snapshot*.
2. Ironwood: orchard 0.15.0 (the librustzcash pin); orchard `bef8a27` (circuit, cross-address); librustzcash `e30517e4`; zips `69610984`, rechecked at `dd1667ff`.
3. Halo 2: halo2_proofs 0.3.2; halo2_gadgets 0.5.0; orchard 0.15.3.
4. Wallet: librustzcash `e30517e4`, later snapshot `91f448b5`; zips `c6b7035`; synchronisation: librustzcash `e30517e4`; lightwalletd `61fee32e`; Zaino `0e057e22`; zips `6961098`.
5. Consensus: zips `69610984`; zebra `fa7657f1c`; librustzcash `e30517e43`; rechecked at zips `e753a6a3`.

Write

Only the heading: *snapshots, as the volumes cite them*. Write a row when a question asks for it; otherwise read the rows out.

Say

Each volume pins the implementation it cites. This board is the answer to "which version?" for four of the volumes: the Ironwood Guide's orchard and librustzcash pins and its zips checkout with the recheck; the Halo 2 Guide's halo2_proofs, halo2_gadgets and orchard pins; the Wallet Guide's later librustzcash snapshot and its lightwalletd and Zaino pins; the Consensus Guide's zebra pin and its own zips recheck.

A Bitcoin Core release number does the same job. A claim about a default is a claim about one version, and this table is where that version lives.

Source: *Halo 2 Guide*, "From statement to circuit: arithmetisation in practice"; *Wallet Guide*, "Addresses and viewing keys (ZIP 316)"; "The compact block and the compact transaction"; *Consensus Guide*, "Sources and their standing"

Board 193 — Implementation snapshots, continued

optional

Draw

1. Same two columns, *Volume* and *Snapshot*.
2. FROST: frost `0966bd1`; reddsa `63c6ac0`; zips `e753a6a3`.
3. FlyClient: zcash_history 0.5.0; zips `e753a6a3`.
4. ZSA: zips `69610984`, rechecked at `e753a6a3`.
5. Crosslink: tfl-book `fe6e1d6`; zips `e753a6a`.
6. Crypto: no baseline pin; one design-stage cite, the orchard fork `cf801a5` (ZSA issuance).
7. Math: no implementation pin; instances cite the specification.

Write

Only the heading: *snapshots, continued*. Rows on request.

Say

The frontier volumes pin the same way. FROST, FlyClient and Crosslink each pin an artefact and a zips checkout; ZSA pins a zips checkout and its recheck.

Two volumes pin no deployed implementation. The Crypto Guide has no baseline pin — one design-stage cite only, the orchard fork for ZSA issuance. The Math Guide has no implementation pin at all: its instances cite the specification.

Source: *FROST Guide*, "Sources and their standing"; *FlyClient Guide*, "Sources and conventions"; *ZSA Guide*, "Scope and status"; *Crosslink Guide*, "Sources and conventions"; *Crypto Guide*, "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"

Board 194 — Scope

optional

Write

- A wallet default = the snapshot the volume cites, not every wallet.
- A node rail = Zebra, named only for the reorg window and the tie-break.

- Three toys: a prime-order group (balance); the $\mathbb{F}_{97}$ circuit (the proof system); the 97-point curve (the IPA fold, discrete logs a short search). One mechanism each; no soundness estimated.
- Bins: *specified* (a published artefact pins it down); *designed but unspecified* (an informal source describes it); *open problem* (no artefact exists).
- A deployed rule may still carry a Draft ZIP: bin and deployment tracked separately.

Say

Every default you heard today was one version's default. A wallet default describes the snapshot the volume cites, not every wallet; a node rail describes Zebra, and Zebra was named only for the reorg window and the tie-break. A Bitcoin Core release number does the same job: a claim about a default is a claim about one version.

The three toys isolated one mechanism each. The prime-order group stood in for balance, the $\mathbb{F}_{97}$ circuit for the proof system, the 97-point curve for the IPA fold — its discrete logs a short search. None of them estimates soundness.

The bins are the volumes' bins. Specified: a published artefact pins it down. Designed but unspecified: an informal source describes it. Open problem: no artefact exists.

The Bitcoin contrast is a BIP's status field. The difference is that here a deployed rule may still carry a Draft ZIP, so the bin and the deployment are tracked separately.

Ask

Which toys had no discrete-log hardness? The balance group, and the 97-point curve of the IPA fold.

Source: *Ironwood Guide*, "Scope and status"; *Consensus Guide*, "Sources and their standing"

Board 195 — The Pasta field discipline — where each object lives

optional

Draw

1. Three columns, headed *lives in*, *elements*, *role*.
2. $\mathbb{F}_{p_{\mathsf{Vesta}}}$ (Pallas *scalar* field): $\mathsf{ask}, \mathsf{r_{ivk}}, \mathsf{rcm}, \mathsf{rcv}, \alpha, \mathsf{rsk}, \mathsf{esk}, \mathsf{bsk}$ — multiply points.
3. $\mathbb{F}_{p_{\mathsf{Pallas}}}$ (Pallas *base* field): $\mathsf{ak}, \mathsf{nk}, \rho, \psi, \mathsf{ivk}, \mathsf{cmx}, \mathsf{nf}$ — the circuit's native field.
4. $\mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ (Pallas points): $\mathsf{ak}^{\mathbb{P}}, \mathsf{rk}, \mathsf{g_d}, \mathsf{pk_d}, \mathsf{epk}, \mathsf{cm}, \mathsf{cv}$ — group elements.
5. bit strings: $\mathsf{sk}, \mathsf{rseed}, \mathsf{dk}, \mathsf{ovk}$ (256); d (88) — keys, seeds, the diversifier.

Write

- One rule: multiplies a point ⇒ scalar modulo the group order p_{Vesta}; hashed or compared by the circuit ⇒ base-field element; consumed by BLAKE2b ⇒ byte string.
- Two base-field elements act as scalars: $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$ and the nullifier's $F_{\mathsf{nk}}(\rho)+\psi$ — valid because $p_{\mathsf{Pallas}} < p_{\mathsf{Vesta}}$ (both 255 bits).
- Bitcoin's secp256k1: coordinates modulo p, scalars modulo n; ECDSA $r := x(R) \bmod n$.

Say

One rule sorts every symbol of the workshop. What multiplies a point is a scalar modulo the group order p_{Vesta}; what the circuit hashes or compares is a base-field element; what BLAKE2b consumes is a byte string. Fill the table row by row and the rule does the sorting.

Two elements cross the line, and one inequality makes that legal. The incoming viewing key ivk is a base-field element yet multiplies $\mathsf{g_d}$; the nullifier's $F_{\mathsf{nk}}(\rho) + \psi$ is base-field arithmetic yet scales a point. Both are valid only because $p_{\mathsf{Pallas}} < p_{\mathsf{Vesta}}$ — every base-field element is below the group order — with both primes 255 bits.

Bitcoin makes the same move once. Its secp256k1 has the same two fields, coordinates modulo p and scalars modulo n, and ECDSA sets $r := x(R) \bmod n$. Nobody notices, because nothing recomputes it inside a proof.

Ask

Why may ivk, a base-field element, multiply a point? Because $p_{\mathsf{Pallas}} < p_{\mathsf{Vesta}}$: every base-field element is a valid scalar.

Do not say

Do not gloss ρ, in the base-field row, as the note's position in the tree: on this board it is the input to the nullifier's $F_{\mathsf{nk}}(\rho) + \psi$, a per-note field element.

Source: *Ironwood Guide*, "The spending key and the spend-side secrets"; "Viewing keys"; "Notes"; "The nullifier"; "Randomised validating keys"; *Crypto Guide*, "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"

Board 196 — The Pasta field discipline — the two primes

optional

Write

$$p_{\mathsf{Pallas}} = 2^{254} + \texttt{0x224698fc094cf91b992d30ed00000001},$$
$$p_{\mathsf{Vesta}} = 2^{254} + \texttt{0x224698fc0994a8dd8c46eb2100000001}$$

- The cycle: $\#\mathcal{E}_{\mathsf{Pallas}}(\mathbb{F}_{p_{\mathsf{Pallas}}}) = p_{\mathsf{Vesta}}$ and $\#\mathcal{E}_{\mathsf{Vesta}}(\mathbb{F}_{p_{\mathsf{Vesta}}}) = p_{\mathsf{Pallas}}$, both prime, cofactor 1.
- Both $p - 1$ carry exactly 2^{32}; the Action circuit uses 2^{11} rows. Both primes $\equiv 1 \bmod 3$: an order-3 endomorphism.
- Deployed: circuit over $\mathbb{F}_{p_{\mathsf{Pallas}}}$; commitments on Vesta, scalar field $\mathbb{F}_{p_{\mathsf{Pallas}}}$; one half of the cycle, no recursion.
- Bitcoin's secp256k1: two-adicity 1 (base), 6 (scalar); largest domain 2 (or 64) points.

Say

Write the two primes and point at the shared prefix. Both are 2^{254} plus a tail; the tails differ. The cycle is the fact that the Pallas curve over the Pallas base field has exactly p_{Vesta} points, and the Vesta curve over its base field exactly p_{Pallas} — both prime, cofactor 1. Each curve's scalar field is the other's base field.

Two properties of $p - 1$ are what the proof system wanted. Both carry exactly 2^{32}, so power-of-two evaluation domains go up to 2^{32}; the Action circuit uses 2^{11} rows. Both primes are 1 mod 3, so an order-3 endomorphism speeds scalar multiplication.

Deployed is one half of the cycle. The circuit's native field is $\mathbb{F}_{p_{\mathsf{Pallas}}}$, so Pallas points are native to it; its commitments live on Vesta, whose scalar field is that same $\mathbb{F}_{p_{\mathsf{Pallas}}}$. Recursion is not deployed; verification amortises by batching instead.

The Bitcoin contrast is secp256k1's two-adicity. Its $p - 1$ has two-adicity 1, its scalar field 6: the largest power-of-two domain has 2 (or 64) points — no room for a 2^{11}-row circuit, and no need for one. Say that this last number is our own computation, not a volume figure.

Ask

Why not build the circuit over secp256k1? Its two-adicity is 1 (scalar field 6): the largest power-of-two domain has 2 (or 64) points, far short of 2^{11} rows.

Source: *Math Guide*, "Pallas and Vesta assembled"; "Base fields, scalar fields, and the Pasta cycle"; *Halo 2 Guide*, "A cycle of curves for efficient native recursion"; *Ironwood Guide*, "The Action circuit and the Halo 2 proof"; protocol specification, "Pallas and Vesta"

Board 197 — Sinsemilla — the hash

optional — exists

Write

$$Q(D) := \mathsf{GroupHash}^{\mathbb{G}}(\texttt{z.cash:SinsemillaQ}, D),$$
$$P[m] := \mathsf{GroupHash}^{\mathbb{G}}(\texttt{z.cash:SinsemillaS}, \mathrm{LE}_{32}(m)), \qquad 0 \le m < 2^{10}$$
$$\mathsf{Sinsemilla}_D(M) := \mathsf{Acc}_\ell, \qquad \mathsf{Acc}_0 := Q(D), \qquad \mathsf{Acc}_{i+1} := (\mathsf{Acc}_i \dot{+} P[m_{i+1}]) \dot{+} \mathsf{Acc}_i$$

- Chunk width $k = 10$; at most $c = 253$ chunks (2530 bits); zero-padded on the right to a multiple of 10 bits.

- Table 1024 points, shared by every domain; $Q(D)$ per domain: 1025 generators, all GroupHash outputs.
- Per chunk: one lookup, two incomplete additions ($\dot{+}$). A 510-bit message: 51 rounds.

Say

Write the two generator families first, then the accumulator. The per-domain point $Q(D)$ and the 1024-entry table $P[m]$ are all GroupHash outputs — 1025 nothing-up-my-sleeve generators. The table is shared by every domain; only $Q(D)$ changes with the domain.

The hash is a walk. Start the accumulator at $Q(D)$; for each 10-bit chunk, add the chunk's table point to the accumulator, then add the old accumulator to the result. The message is zero-padded on the right to a multiple of 10 bits, and there are at most 253 chunks, 2530 bits.

The cost model is the point. Each chunk costs one table lookup and two incomplete additions; a 510-bit message costs 51 rounds, not 510 bit operations. Inside the circuit the table is a lookup argument.

The Bitcoin contrast is SHA-256's home ground. SHA-256 is tuned for a CPU's word operations; inside a circuit every one of its bits is a constraint. Sinsemilla is tuned for the circuit that must recompute it — and would be a poor CPU hash.

Ask

How many rounds does a 510-bit message cost? Fifty-one: one lookup and two incomplete additions per 10-bit chunk.

Source: *Ironwood Guide*, "Sinsemilla: hash, commitment, and short forms", Remark "Cost inside a proof"; protocol specification, "Sinsemilla Hash Function"

Board 198 — Sinsemilla — the commitment and its short forms

optional — exists

Write

$$\mathsf{SinsemillaCommit}_r(D, M) := \mathsf{Sinsemilla}_{D\|\texttt{-M}}(M) + [r]\,H_D,$$
$$\mathsf{ShortCommit}_r(D, M) := \mathsf{Extract}(\mathsf{SinsemillaCommit}_r(D, M)),$$
$$\mathsf{SinsemillaHash}(D, M) := \mathsf{Extract}(\mathsf{Sinsemilla}_D(M)), \qquad H_D := \mathsf{GroupHash}^{\mathbb{G}}(D\,\|\,\texttt{-r},\ \varepsilon)$$

- Instances: NoteCommit — this commitment in its own domain, a point; the chain stores cmx, its x.
- Key commitment $\mathsf{Commit}^{\mathsf{ivk}}$ — ShortCommit over $\mathrm{LE}_{255}(\mathsf{Extract}(\mathsf{ak}))\,\|\,\mathrm{LE}_{255}(\mathsf{nk})$, 510 bits, 51 rounds.
- Tree node hash — SinsemillaHash over 520 bits, 52 rounds.

Say

The commitment is the hash plus one blinding term. Hash the message in the domain's -M sub-domain, then add $[r]\,H_D$, where H_D is a GroupHash point in the domain's own -r sub-domain. The short forms take the x-coordinate.

Hiding needs no assumption. For uniform r the term $[r]\,H_D$ is uniform on the group, so the commitment reveals nothing about M.

Binding rests on the two bases being unrelated. The point H_D sits in its own sub-domain, unrelated to Q and the table as far as anyone knows — Pedersen's split of message base from blinding base.

Three instances carried the workshop. The note commitment NoteCommit is this commitment in its own domain: a point, of which the chain stores cmx. The key commitment $\mathsf{Commit}^{\mathsf{ivk}}$ is the short commitment over Extract(ak) and nk, 510 bits, 51 rounds. The tree's node hash is SinsemillaHash over 520 bits, 52 rounds.

The Bitcoin bridge comes next. Confidential Transactions' $[v]V + [r]R$ is the one-slot Pedersen ancestor of this shape.

Ask

Which side of the commitment needs an assumption, hiding or binding? Binding: hiding is unconditional for uniform r; binding needs H_D unrelated to Q and the table.

Source: *Ironwood Guide*, "Sinsemilla: hash, commitment, and short forms"; "Note commitments"; "The Merkle hash and the tree"; protocol specification, "Sinsemilla commitments"

Board 199 — Sinsemilla beside Confidential Transactions

optional — exists

Write

$$\text{CT: } [v]\,V + [r]\,R; \quad \text{Pedersen hash: } \textstyle\sum_j [m_j]\,P_j; \quad \text{Sinsemilla: } [2^\ell]\,Q(D) + \sum_i [2^{\ell-i}]\,P[m_i].$$

- One chunk: $[2]\,Q(D) + P[m_1]$ — a lookup, not $[m_1]\,V$.
- Pedersen's coefficient has become Sinsemilla's index.

Say

Write the three forms on one line and read them left to right. Confidential Transactions' $[v]V + [r]R$ is the one-slot Pedersen commitment. The Pedersen *hash* extends it to a message with one independent generator per chunk — a table that grows with the message length.

Sinsemilla keeps the reduction and swaps the roles. The discrete-log collision reduction stays. But a chunk value now *selects* a point from one shared 1024-entry table, and the chunk's position supplies a fixed power-of-two weight. One table serves every domain and every length up to 253 chunks.

The one-chunk case shows the swap. A one-chunk Sinsemilla is $[2]\,Q(D) + P[m_1]$: a lookup, not $[m_1]\,V$. Pedersen's coefficient has become Sinsemilla's index.

The Bitcoin contrast is what checks the opening. In Elements the opening of $[v]V + [r]R$ is range-proved by a Bulletproof; here NoteCommit's opening is recomputed inside the Action proof, which is what the lookup table is tuned for.

Ask

What is a one-chunk Sinsemilla? The point $[2]\,Q(D) + P[m_1]$ — the chunk indexes the table instead of scaling a generator.

Source: *Crypto Guide*, "Sinsemilla: an algebraic hash-based commitment"; *Ironwood Guide*, "Sinsemilla: hash, commitment, and short forms", Proposition "Collision resistance of the Sinsemilla instances"; Remark "Cost inside a proof"

Board 200 — Sinsemilla — why a collision is a discrete logarithm

optional — exists

Write

$$\mathsf{Sinsemilla}_D(M) = [2^\ell]\,Q(D) + \sum_{i=1}^{\ell} [2^{\ell-i}]\,P[m_i], \qquad \chi_j(m) := \sum_{i=1}^{\ell} 2^{\ell-i}\,[m_i = j],$$

$$\mathsf{Sinsemilla}_D(M) = \mathsf{Sinsemilla}_D(M') \implies \sum_{j=0}^{2^{10}-1} [\chi_j(m) - \chi_j(m')]\,P[j] = O.$$

- Take $M \neq M'$ of equal length; χ_j taken modulo p_{Vesta}.
- The bound $\ell \leq 253$ keeps $2^{253} < p_{\mathsf{Vesta}}$: nothing wraps; $\chi(m)$ is injective.

Say

Unfold the accumulator, assuming no addition exception. Every chunk lands on a table generator with a power-of-two weight, and $Q(D)$ carries 2^ℓ. A repeated chunk value collects its weights into χ_j, taken modulo p_{Vesta}. Take two distinct messages of equal length.

A collision is a relation among the generators. Subtract the two unfolded sums: the Q terms cancel, and what remains is a linear relation among the $P[j]$ with coefficients $\chi_j(m) - \chi_j(m')$.

The relation is non-trivial because χ is injective. The binary digits of χ_j mark exactly where chunk value j occurs, and $\ell \leq 253$ keeps $2^{253} < p_{\mathsf{Vesta}}$, so nothing wraps modulo the group order. Distinct messages give distinct vectors, so some coefficient is non-zero — and a non-trivial relation among independently sampled random-oracle generators is found only by discrete logarithms.

The Bitcoin contrast is the kind of assumption. A SHA-256 collision would falsify a heuristic; a Sinsemilla collision would compute a discrete logarithm on Pallas — the same kind of assumption that guards every Bitcoin key, on a different curve.

Ask

Why does $\ell \leq 253$ matter to the proof? It keeps $2^{253} < p_{\mathsf{Vesta}}$, so the weight vector $\chi(m)$ never wraps and stays injective.

Source: *Ironwood Guide*, "Sinsemilla: hash, commitment, and short forms", Proposition "Collision resistance of the Sinsemilla instances"; *Crypto Guide*, "Sinsemilla: an algebraic hash-based commitment"

Board 201 — Sinsemilla — the collision argument's fine print

optional — exists

Write

$$\mathsf{Sinsemilla}_D(M) = -\mathsf{Sinsemilla}_D(M') \implies [2^{\ell+1}]\,Q(D) + \sum_j [\chi_j(m) + \chi_j(m')]\,P[j] = O$$

- The coefficient $2^{\ell+1} \neq 0 \bmod p_{\mathsf{Vesta}}$: non-trivial whether or not any chunk differs.
- Blinded forms: subtracting two openings adds $[r - r']\,H_D$ (or $[r + r']\,H_D$).
- Fixed length is load-bearing: every deployed use fixes its input length.

Say

Taking x-coordinates adds a second case. The short forms identify P with $-P$, so a short-form collision is either the $+$ case from the previous board or this $-$ case. Its Q-coefficient is $2^{\ell+1}$, non-zero modulo p_{Vesta}, so the relation is non-trivial whether or not any chunk differs.

Blinding does not weaken it. Subtracting two openings of the commitment adds $[r - r']\,H_D$, or $[r + r']\,H_D$ in the $-$ case; the message coefficients or the Q-coefficient keep the relation non-trivial. Two distinct notes cannot share cm, nor cmx.

The addition exception is not a loophole. An undefined incomplete addition is an equality between accumulator points — itself a relation among the same generators.

Fixed length is load-bearing. Zero-padding makes two messages of different bit-lengths collide with no relation at all, so every deployed use fixes its input length.

The Bitcoin contrast is one hash at every length. A Bitcoin hash is used at every length; here each domain hashes one fixed length, and the theorem is stated for exactly that.

Ask

What breaks if the input length is not fixed? Zero-padding lets two messages of different bit-lengths collide with no relation among the generators at all.

Source: *Ironwood Guide*, "Sinsemilla: hash, commitment, and short forms", Proposition "Collision resistance of the Sinsemilla instances"; Remark "Fixed length"

Board 202 — The seal — three rules, in full

optional — conserved

Write

1. **No issuance.** Coinbase: *empty* Orchard component.
2. **No circulation.** Every Orchard-pool Action: $\mathsf{enableCrossAddress} = 0$ — enforced by the Action-circuit verifying key, selected by branch.
3. **No entry.** Per transaction: $v^{\mathsf{Orchard}}_{\mathsf{balance}} \geq 0$.

All three bind every transaction *regardless of its version* (ZIP-258).

Say

Three consensus rules seal the Orchard pool to spend-only. Number them as you write them. No issuance: a coinbase transaction carries an empty Orchard component. No circulation: every Orchard-pool Action has enableCrossAddress $= 0$. No entry: per transaction, $v_{\text{balance}}^{\text{Orchard}} \geq 0$ — net value may leave the pool, never enter.

All three bind regardless of transaction version. The second is enforced by the Action-circuit verifying key, which consensus selects by branch, so a v5 transaction cannot bypass it; ZIP-229 states the version-independence explicitly.

The second rule is the deepest, because it lives in the circuit. An Orchard-pool spend can pay only its own receiver. The motive is recoverability: lead-byte-`0x02` notes cannot be recovered after a quantum break, so consensus stops their circulation and funnels the value out, while owners keep spending.

Bitcoin retires no output type. An early P2PK coin is spendable today on the terms it was created with. Zcash retires a pool by sealing its doors and metering its exit.

Ask

Which of the three rules lives in the circuit? No circulation: the verifying key, selected by branch, enforces enableCrossAddress $= 0$.

Do not say

Do not say the seal applies only to v6 transactions: all three rules bind every transaction regardless of its version.

Source: *Ironwood Guide*, "Chain state and pool rules"

Board 203 — The cross-address restriction, A10, in full

optional — conserved

Write

With $\delta :=$ disableCrossAddress and $x(\cdot), y(\cdot)$ the affine coordinates of a Pallas point:

$$\delta \cdot (x(\mathsf{g}_{\mathsf{d}_{\text{old}}}) - x(\mathsf{g}_{\mathsf{d}_{\text{new}}})) = 0, \qquad \delta \cdot (y(\mathsf{g}_{\mathsf{d}_{\text{old}}}) - y(\mathsf{g}_{\mathsf{d}_{\text{new}}})) = 0,$$
$$\delta \cdot (x(\mathsf{pk}_{\mathsf{d}_{\text{old}}}) - x(\mathsf{pk}_{\mathsf{d}_{\text{new}}})) = 0, \qquad \delta \cdot (y(\mathsf{pk}_{\mathsf{d}_{\text{old}}}) - y(\mathsf{pk}_{\mathsf{d}_{\text{new}}})) = 0.$$

- Public inputs: anchor; cv (x, y); nf; rk (x, y); cmx; enableSpends; enableOutputs; δ — ten field elements; no normative instance position for δ.
- Same gate shape as A3: $v_{\text{old}} \cdot (\text{root} - \text{anchor}) = 0$.
- Flags byte: `0b001` spends, `0b010` outputs, `0b100` cross-address (Ironwood; reserved in Orchard); canonical `0b011` Orchard pool, `0b111` Ironwood.

Say

A10 is four equalities in the base field. With δ the disable flag, each equality says: if $\delta \neq 0$, one coordinate of the old receiver equals the same coordinate of the new one. Four rows cover x and y of g_{d} and of pk_{d}, so with $\delta \neq 0$ the receivers are equal as full points, not merely in x.

The flag is one more public input. Write them: anchor, the two coordinates of cv, nf, the two of rk, cmx, enableSpends, enableOutputs, and δ; no normative text fixes its position in the encoded instance. The gate shape is A3's $v_{\text{old}} \cdot (\text{root} - \text{anchor}) = 0$ on four more rows.

On the wire the polarity is reversed. Bit 2 of the Ironwood flags byte is enableCrossAddress — bits `0b001` spends, `0b010` outputs, `0b100` cross-address; in the Orchard byte bit 2 stays reserved and 0 — and the canonical bytes are `0b011` for the Orchard pool and `0b111` for Ironwood. Bit $2 = 0$ is what a v5 signer treating it as reserved-zero already produces, so hardware signers keep signing Orchard-pool unshielding unchanged.

The Bitcoin contrast is how a rule narrows. A soft fork narrows which scripts are valid; here the narrowing is a new verifying key selected by branch, and the wire change is one formerly reserved bit.

Ask

Why four equalities and not two? Equality in x alone would identify P with $-P$; x and y of both $\mathsf{g_d}$ and $\mathsf{pk_d}$ make the receivers equal as full points.

Do not say

Do not read δ as the wire flag: the wire carries enableCrossAddress, the instance carries its negation disableCrossAddress.

Source: *Ironwood Guide*, "The Action statement", Remark "Meaning of A10"; "Bundle flags"

Board 204 — Spending under the restriction

optional — conserved

Write

$$(\mathsf{addr}_{\text{old}},\ v_{\text{old}}) \longrightarrow (\mathsf{addr}_{\text{new}} = \mathsf{addr}_{\text{old}},\ v_{\text{new}} = 0), \qquad v^{\mathsf{Orchard}}_{\text{balance}} = v_{\text{old}} > 0.$$

- Pay anyone by *exiting*: value leaves through a positive $v^{\mathsf{Orchard}}_{\text{balance}}$; the output slot holds a fabricated zero-valued note to the spent note's own receiver.
- Self-sends stay possible; wallets forbid sends to *external* Orchard-pool receivers (ZIP-326).

Say

An Action spends one note and creates one, and the restriction fixes the created note's receiver. Under A10 the created note must pay the spent note's own receiver. So an owner pays anyone by exiting: the real value leaves through a positive $v^{\mathsf{Orchard}}_{\text{balance}}$, and the output slot holds a fabricated zero-valued note to the spent note's own receiver — which is also what keeps the Orchard tree growing.

Self-sends survive by construction. Paying an Orchard-pool receiver requires a spend at that receiver, hence its spending key. Wallet rules close the gap by forbidding sends to external Orchard-pool receivers — that is ZIP-326.

The nearest Bitcoin concept is a covenant. A covenant restricting where an output may go is something Bitcoin script does not offer; here it is one gate in a fixed circuit.

Ask

How does an Orchard-pool owner pay Bob? By exiting: the value leaves as a positive $v^{\mathsf{Orchard}}_{\text{balance}}$, and the Action's output is a zero-valued note to the spent note's own receiver.

Do not say

Do not attribute the fabricated zero-valued output to the two-Action minimum: it is there because every Action creates one note and the restriction fixes that note's receiver.

Source: *Ironwood Guide*, "The Action statement", Remark "Effect on the Orchard pool"; "Actions, bundles, and dummy notes"

Board 205 — The fabricated output, and the restriction's standing

optional

Write

- Fabricated output: C_{enc} = random bytes.
- Inside a PCZT: recipient, value, rseed explicit ⇒ signer classifies it as a tolerable dummy.
- *Specified*: the flag's encoding, its consensus rule, the verifying-key selection by branch (ZIP-229, ZIP-258); "equal receivers" and $\delta = 1 -$ enableCrossAddress (protocol specification).
- *Designed but unspecified*: δ's instance position, the dummy-spend treatment.
- ZIP-2006: a Reserved stub.

Say

The fabrication is thorough. The fabricated output's C_{enc} is random bytes. A real encryption would let any ivk holder — including a later quantum adversary who recovered ivk from a published address — decrypt it and link the Action's nullifier to the address. That is ZIP-326's reasoning.

Between construction and signing the pretence is dropped. Inside a PCZT the fabricated output carries its recipient, value and rseed explicitly, so a signer can classify it as a tolerable dummy rather than an opaque payment.

Its standing is split. The restriction is deployed and enforced by consensus, yet its owning ZIP-2006 is a Reserved stub. Specified: the flag's encoding, its consensus rule, the verifying-key selection by branch, in ZIP-229 and ZIP-258; the semantics of "equal receivers" and the flag's negation into the statement, in the protocol specification. Designed but unspecified: the flag's position in the instance, the dummy-spend treatment.

The Bitcoin contrast is what a signer can see. A Bitcoin signer sees every output in the clear; a shielded signer needs the format to tell it which output is a dummy.

Ask

Why is the fabricated C_{enc} random rather than a real encryption of the zero note? A real encryption would let an ivk holder — a future one included — link the Action's nullifier to the address.

Source: *Ironwood Guide*, "Actions, bundles, and dummy notes"; "The Action statement", Remark "Status"

Board 206 — The turnstile and migration

optional — conserved

Write

$$\mathsf{IronwoodPoolBalance} := -\sum_{\text{tx}} v^{\mathsf{Ironwood}}_{\text{balance}}(\text{tx}) \;\geq\; 0, \qquad v^{\mathsf{Orchard}}_{\text{balance}} \geq 0 \text{ per transaction.}$$

- Migrating transaction: Orchard-pool spends with $v^{\mathsf{Orchard}}_{\text{balance}} > 0 \rightarrow$ transparent funds (fees and transparent outputs paid here) $\rightarrow v^{\mathsf{Ironwood}}_{\text{balance}} < 0$. Both fields cleartext.
- Pool balances: zero before the upgrade, never negative, summing to at most MAX_MONEY (ZIP-209).
- Consensus caps nothing migration-specific; ZIP-318 recommends migration outputs of at most 10 000 ZEC.
- Ironwood tree capacity 2^{32}; history tree gains three Ironwood fields.

Say

The turnstile is the instrument that retired Sprout. It is a one-way, publicly metered gate, and its consensus content is exactly the two rules on the board: the seal's third, and the Ironwood pool-balance row under ZIP-209's rule — zero before the upgrade, never negative, all pool balances summing to at most MAX_MONEY.

A migrating transaction has a fixed shape. It spends Orchard-pool notes with $v^{\mathsf{Orchard}}_{\text{balance}} > 0$ into its transparent funds — where fees and transparent outputs are paid too — and absorbs them with $v^{\mathsf{Ironwood}}_{\text{balance}} < 0$. Both fields are cleartext: the amount crossing is public in every transaction.

Consensus caps nothing migration-specific. ZIP-318 — a Reserved stub at the pin, a Draft at the later recheck — recommends migration outputs of at most 10 000 ZEC. The Ironwood tree has its twin's 2^{32} capacity; the history tree gains three Ironwood fields.

The Bitcoin contrast is the ledger row itself. Bitcoin keeps no per-type ledger row because every amount is already public; the pool balance is the shielded ledger's one public number per pool.

Ask

Where does a migrating transaction pay its fee? From the transparent funds it passes through between the Orchard exit and the Ironwood entry.

Do not say

Do not say the pool is auditable: the balance row is the one public number per pool, and what it does is meter the exit. Do not present 10 000 ZEC as a consensus cap: it is a ZIP-318 recommendation.

Source: *Ironwood Guide*, "Chain state and pool rules"; ZIP-258, "Changes to ZIP 209"; "Changes to ZIP 221"; ZIP-318, "Amount selection, canonical quantization"

Board 207 — The quantum footnote — what Part I left out

optional

Write

- One external ivk opens neither the internal-scope ivk nor any scope whose address the adversary never learns.
- The rcm derivation's collision bound: classical $O(q^2/N)$ → QROM $O(q^3/N)$ (Zhandry, compressed oracle). At $q = 2^{60}$, $N \approx 2^{254}$: 2^{-134} classically, 2^{-74} in the QROM.
- Recovery Statement, key half: BindKeys — ask, ak, nk from sk; plus $\mathsf{r}_{\mathsf{ivk}}$ from sk by PRFexpand, or from a quantum key qk = one BLAKE3 compression of qsk; signatures of knowledge over the sighash.

Say

The retroactive break has an edge. One external ivk does not open the separate internal-scope ivk, where change lands (Part D), nor any scope whose address the adversary never learns. The attack is keyed by the address, not by the ciphertext.

The binding bound has a price. The classical collision bound $O(q^2/N)$ on the rcm derivation lifts, by Zhandry's compressed-oracle technique, to $O(q^3/N)$ in the quantum random-oracle model. At $q = 2^{60}$ and $N \approx 2^{254}$ that is about 2^{-134} classically against 2^{-74}: the lift costs a factor q.

The Recovery Statement's key half is BindKeys **plus the** $\mathsf{r}_{\mathsf{ivk}}$ **branch.** The name BindKeys covers the derivations of ask, ak, nk from sk. The $\mathsf{r}_{\mathsf{ivk}}$ branch derives $\mathsf{r}_{\mathsf{ivk}}$ from sk by PRFexpand, or — where no single sk serves, FROST-generated or hardware-held keys — from a quantum key qk, one BLAKE3 compression of a secret qsk, with signatures of knowledge over the sighash tying the claimed keys to the spend.

Bitcoin's exposure is of Part I's table's kind, with two differences. A forged signature under an exposed public key must be accepted. But a forged RedPallas signature alone spends nothing here — theft also needs the note's witness or a proof forgery — and counterfeit, not theft, is what the pool's exit caps.

Ask

Does a recovered external ivk reach the change? No: change lands under the separate internal-scope ivk, which the external one does not open.

Do not say

Do not say change is found through ovk: on this board the change is reached only through the internal-scope ivk, which the external ivk does not open. Do not turn the 2^{-74} into a timing claim: it is a bound at one query budget, not a clock.

Source: *Ironwood Guide*, "Encryption to the recipient"; ZIP-2005, "Effects of discrete-logarithm-breaking attacks after the switch to the Recovery Protocol"; "QROM lift of the security arguments"; "Proposed Recovery Statement"; "Effects of discrete-logarithm-breaking attacks before the switch to the Recovery Protocol"